



Développement d'Applications Flutter

Cours : Développement d'Applications Flutter

Module 1 : Introduction à Flutter et à Dart

Leçon 1 : Comprendre Flutter et son écosystème

Flutter est un **framework open-source** développé par **Google** pour créer des applications **natives** de haute performance pour **iOS, Android, Web** et **desktop** à partir d'une seule base de code. Il utilise le langage de programmation **Dart**, également développé par Google, qui offre une syntaxe moderne et efficace.

Écosystème Flutter :

- **Widgets** : Les blocs de construction de l'interface utilisateur en Flutter. Tout est un widget, des éléments visuels aux structures de layout.
- **Flutter SDK** : Comprend les outils nécessaires pour développer, compiler et déployer des applications Flutter.
- **Pub.dev** : Le gestionnaire de packages de Flutter, permettant d'ajouter facilement des **packages** et **plugins** tiers pour étendre les fonctionnalités.
- **Flutter DevTools** : Un ensemble d'outils de développement pour le **debugging**, l'**inspection des widgets**, et l'**analyse des performances**.
- **Communauté et Documentation** : Une vaste communauté active et une documentation exhaustive facilitant l'apprentissage et le développement.

Avantages de Flutter :

- **Performance native** grâce au moteur de rendu **Skia**.
- **Hot Reload** permettant de voir instantanément les modifications sans redémarrer l'application.
- **Personnalisation** complète de l'interface utilisateur grâce aux widgets hautement configurables.
- **Support multiplateforme** réduisant le temps et les coûts de développement.

Leçon 2 : Installation de Flutter et configuration de l'environnement de développement

Étapes d'installation de Flutter :

1. **Téléchargement du SDK Flutter :**

- Rendez-vous sur le site officiel flutter.dev et téléchargez le **Flutter SDK** adapté à votre système d'exploitation (**Windows, macOS, Linux**).
- 2. **Extraction et configuration du PATH :**
 - Extrayez le contenu de l'archive téléchargée.
 - Ajoutez le répertoire `flutter/bin` à la variable d'environnement **PATH** pour pouvoir exécuter les commandes Flutter depuis n'importe quel terminal.
- 3. **Installation des dépendances :**
 - **Windows :** Installez **Git** et assurez-vous qu'il est accessible via le PATH.
 - **macOS :** Assurez-vous que **Xcode** est installé pour le développement iOS.
 - **Linux :** Installez les dépendances nécessaires via votre gestionnaire de paquets.
- 4. **Vérification de l'installation avec flutter doctor :**
 - Ouvrez un terminal et exécutez la commande `flutter doctor` pour vérifier l'installation et identifier les éventuels problèmes à résoudre.

Configuration de l'environnement de développement :

1. **Choix d'un IDE :**
 - **Visual Studio Code :** Léger et extensible avec des extensions Flutter et Dart.
 - **Android Studio :** Offre des outils intégrés pour Flutter et une intégration complète avec le SDK Android.
2. **Installation des extensions/plugins :**
 - Installez les extensions **Flutter** et **Dart** dans votre IDE choisi pour bénéficier de fonctionnalités telles que le **highlighting syntaxique**, le **complétion de code**, et les **outils de débogage**.
3. **Configuration des émulateurs/simulateurs :**
 - **Android :** Configurez un **émulateur Android** via **Android Virtual Device (AVD) Manager**.
 - **iOS :** Utilisez le **simulator iOS** fourni par **Xcode**.
4. **Création d'un projet Flutter de test :**
 - Exécutez la commande `flutter create nom_du_projet` pour créer un nouveau projet Flutter.
 - Ouvrez le projet dans votre IDE et lancez l'application sur un émulateur ou un appareil physique pour vérifier que tout fonctionne correctement.

Leçon 3 : Notions de base du langage Dart

Introduction à Dart :

- **Dart** est un langage de programmation orienté objet, **fortement typé** et **optimisé** pour le développement d'applications **client-side**.

Principes fondamentaux :

1. Syntaxe de base :

- **Variables** : Déclaration avec `var`, `final`, et `const`.
- `var nom = 'Flutter';`
- `final age = 10;`
- `const pi = 3.14;`
- **Types de données** : `int`, `double`, `String`, `bool`, `List`, `Map`, etc.

2. Fonctions :

- Définition et utilisation des fonctions.
- `int addition(int a, int b) {`
- `return a + b;`
- `}`
- **Fonctions anonymes et closures.**

3. Classes et Objets :

- Création de **classes** avec **constructeurs**, **méthodes**, et **attributs**.
- `class Personne {`
- `String nom;`
- `int age;`
- `Personne(this.nom, this.age);`
- `void saluer() {`
- `print('Bonjour, je m'appelle $nom et j'ai $age`
- `ans.');`
- `}`
- `}`
- **Héritage et polymorphisme.**

4. Gestion des erreurs :

- Utilisation de `try`, `catch`, et `finally` pour gérer les exceptions.
- `try {`
- `// Code susceptible de générer une exception`
- `} catch (e) {`
- `print('Erreur : $e');`
- `} finally {`
- `// Code exécuté en toutes circonstances`
- `}`

5. Asynchronisme :

- Utilisation de **`async`** et **`await`** pour gérer les opérations asynchrones.
- `Future<void> fetchData() async {`
- `var data = await obtenirDonnees();`
- `print(data);`
- `}`

Concepts avancés :

- **Generics** : Types génériques pour créer des classes et des fonctions réutilisables.
- `class Boite<T> {`
- `T contenu;`
- `Boite(this.contenu);`
- `}`
- **Extensions** : Ajouter des fonctionnalités aux classes existantes sans les modifier.
- `extension StringExtensions on String {`
- `bool estPalindrome() {`

- `return this == this.split('').reversed.join('');`
- `}`
- `}`

Leçon 4 : Syntaxe et structures de contrôle en Dart

Structures de contrôle de flux :

1. Conditionnelles :

- **if et else :**
- `if (age > 18) {`
- `print('Adulte');`
- `} else {`
- `print('Mineur');`
- `}`
- **Opérateur ternaire :**
- `String statut = age > 18 ? 'Adulte' : 'Mineur';`

2. Boucles :

- **for classique :**
- `for (int i = 0; i < 10; i++) {`
- `print(i);`
- `}`
- **for-in pour itérer sur des collections :**
- `List<String> noms = ['Alice', 'Bob', 'Charlie'];`
- `for (var nom in noms) {`
- `print(nom);`
- `}`
- **while et do-while :**
- `int compteur = 0;`
- `while (compteur < 5) {`
- `print(compteur);`
- `compteur++;`
- `}`

3. Switch-case :

- 4. `switch (jour) {`
- 5. `case 'Lundi':`
- 6. `print('Début de la semaine');`
- 7. `break;`
- 8. `case 'Vendredi':`
- 9. `print('Presque le week-end');`
- 10. `break;`
- 11. `default:`
- 12. `print('Jour standard');`
- 13. `}`

Fonctionnalités avancées :

- **Break et Continue :**
 - **break** pour sortir d'une boucle ou d'un switch.
 - **continue** pour sauter à l'itération suivante d'une boucle.
- **Labels :**

- Utilisés pour identifier les boucles imbriquées.
- `outerLoop:`
- `for (var i = 0; i < 3; i++) {`
- `for (var j = 0; j < 3; j++) {`
- `if (i == j) {`
- `continue outerLoop;`
- `}`
- `print('$i, $j');`
- `}`
- `}`

Leçon 5 : Gestion des packages et des dépendances dans Flutter

Introduction à Pub.dev :

- **Pub.dev** est le dépôt officiel de packages pour **Dart** et **Flutter**. Il permet de trouver, partager et utiliser des packages tiers pour enrichir vos applications.

Ajouter des dépendances :

1. **Modifier le fichier `pubspec.yaml` :**
 - Sous la section **dependencies**, ajoutez le nom du package et sa version.
 - `dependencies:`
 - `flutter:`
 - `sdk: flutter`
 - `http: ^0.13.4`
2. **Installer les packages :**
 - Exécutez la commande **`flutter pub get`** dans le terminal pour télécharger et installer les packages ajoutés.

Gestion des versions :

- **Spécifier des versions :**
 - Utilisez des **contraintes de version** pour contrôler les mises à jour des packages.
 - `dependencies:`
 - `http: ^0.13.0`
 - `^` permet d'accepter les mises à jour compatibles.
- **Mise à jour des packages :**
 - **`flutter pub upgrade`** pour mettre à jour les packages vers leurs dernières versions compatibles.

Utilisation des packages :

- **Importer le package** dans votre fichier Dart.
- `import 'package:http/http.dart' as http;`
- **Exploiter les fonctionnalités** fournies par le package.
- `Future<void> fetchData() async {`

- ```
final response = await
http.get(Uri.parse('https://api.example.com/data'));
```
- ```
if (response.statusCode == 200) {
```
- ```
 print('Données récupérées : ${response.body}');
```
- ```
} else {
```
- ```
 print('Erreur : ${response.statusCode}');
```
- ```
}
```
- ```
}
```

### Création de packages personnalisés :

- **Structurer le package** avec les dossiers **lib**, **test**, etc.
- **Publier le package** sur **Pub.dev** pour le partager avec la communauté.

### Gestion des dépendances transitive :

- Les packages que vous ajoutez peuvent avoir leurs propres dépendances. **Pub** gère automatiquement ces dépendances transitive pour assurer la compatibilité et éviter les conflits de versions.

## Leçon 6 : Comparaison entre Flutter et autres frameworks mobiles

### Flutter vs. React Native :

- **Langage** : Flutter utilise **Dart**, tandis que React Native utilise **JavaScript**.
- **Performance** : Flutter compile en **code machine natif**, offrant des performances supérieures.
- **Widgets vs. Composants** : Flutter propose une vaste bibliothèque de **widgets** personnalisables, tandis que React Native s'appuie sur des **composants** natifs.
- **Hot Reload** : Les deux frameworks offrent des fonctionnalités de **hot reload**, mais Flutter est souvent loué pour sa rapidité et sa fiabilité.

### Flutter vs. Xamarin :

- **Langage** : Flutter utilise **Dart**, Xamarin utilise **C#**.
- **Intégration** : Xamarin est intégré dans l'écosystème **.NET**, facilitant la réutilisation du code backend.
- **Performance** : Flutter a tendance à offrir une meilleure performance graphique grâce à son moteur de rendu propriétaire.

### Flutter vs. Native Development :

- **Codebase** : Flutter permet une **base de code unique** pour plusieurs plateformes, réduisant le temps de développement.
- **Performance** : Les applications natives peuvent offrir une performance légèrement supérieure, mais Flutter est très proche grâce à son moteur de rendu.

- **Accès aux fonctionnalités** : Flutter offre un accès complet aux fonctionnalités natives via des **plugins**, mais certaines fonctionnalités spécifiques peuvent nécessiter du code natif supplémentaire.

### Flutter vs. Ionic :

- **Technologie sous-jacente** : Ionic est basé sur **HTML, CSS et JavaScript**, fonctionnant dans un **WebView**, tandis que Flutter compile en **code natif**.
- **Performance** : Flutter offre généralement de meilleures performances graphiques et une expérience utilisateur plus fluide.
- **Expérience utilisateur** : Flutter permet une personnalisation plus poussée des interfaces grâce à ses widgets natifs.

**Conclusion** : Flutter se distingue par sa **performance élevée**, sa **richesse en widgets personnalisables**, et sa **capacité à maintenir une base de code unique** pour plusieurs plateformes. Cependant, le choix du framework dépend des besoins spécifiques du projet, des compétences de l'équipe de développement, et des exigences en matière de performance et de fonctionnalités.

## Module 2 : Fondamentaux de la Conception d'Interfaces Utilisateur avec Flutter

### Leçon 1 : Principes de base des widgets dans Flutter

Les **widgets** sont les éléments fondamentaux de l'interface utilisateur en Flutter. Chaque composant visuel, depuis un simple **texte** jusqu'à des structures de layout complexes, est représenté par un widget.

#### Types de widgets :

- **Widgets de base** : Text, Image, Icon.
- **Widgets de layout** : Container, Row, Column, Stack.
- **Widgets interactifs** : Button, GestureDetector, InkWell.
- **Widgets d'entrée** : TextField, Checkbox, Switch.

#### Composition des widgets :

- **Immuabilité** : Les widgets sont immuables, ce qui signifie qu'ils ne changent pas une fois créés. Pour mettre à jour l'interface, Flutter reconstruit les widgets.
- **Hierarchie des widgets** : Les widgets sont organisés en une **arborescence**, où les widgets parents peuvent contenir des widgets enfants.

#### Types de widgets :

- **StatelessWidget** :

- Utilisé pour des widgets qui ne changent pas d'état.
- Exemple :

```
class MonWidget extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
 return Text('Hello Flutter');
 }
}
```
- **StatefulWidget :**
  - Utilisé pour des widgets qui maintiennent un état mutable.
  - Exemple :

```
class Compteur extends StatefulWidget {
 @override
 _CompteurState createState() => _CompteurState();
}

class _CompteurState extends State<Compteur> {
 int _compteur = 0;

 void _incrémenter() {
 setState(() {
 _compteur++;
 });
 }

 @override
 Widget build(BuildContext context) {
 return Column(
 children: [
 Text('Compteur : $_compteur'),
 ElevatedButton(
 onPressed: _incrémenter,
 child: Text('Incrémenter'),
),
],
);
 }
}
```

### Principe de composition :

- Flutter encourage la **composition** des widgets plutôt que l'héritage, permettant de créer des interfaces complexes en combinant des widgets simples.
- Exemple de composition :

```
class CarteProfil extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
 return Card(
 child: Column(
 children: [
 Image.network('https://example.com/image.jpg'),
 Text('Nom Utilisateur'),
 Text('Description du profil'),
],
),
);
 }
}
```

- ],
- ),
- );
- }
- }

### Optimisation des widgets :

- Utilisez des **widgets légers** et évitez de reconstruire l'arborescence des widgets inutilement.
- Utilisez des **Keys** pour préserver l'état des widgets lors de la reconstruction.

## Leçon 2 : Création et personnalisation des layouts

La création de **layouts** en Flutter repose sur l'utilisation de widgets de layout qui définissent la disposition des éléments à l'écran.

### Principaux widgets de layout :

#### 1. Container :

- Sert de **boîte de décoration** avec des propriétés telles que la **marge**, le **padding**, les **bordures**, et les **couleurs de fond**.
- ```
Container(
```
- ```
 margin: EdgeInsets.all(10),
```
- ```
  padding: EdgeInsets.symmetric(horizontal: 20, vertical:
```
- ```
 10),
```
- ```
  decoration: BoxDecoration(
```
- ```
 color: Colors.blue,
```
- ```
    borderRadius: BorderRadius.circular(10),
```
- ```
),
```
- ```
  child: Text('Contenu personnalisé'),
```
- ```
)
```

#### 2. Row et Column :

- **Row** dispose les enfants horizontalement, tandis que **Column** les dispose verticalement.
- ```
Row(
```
- ```
 mainAxisAlignment: MainAxisAlignment.spaceEvenly,
```
- ```
  children: [
```
- ```
 Icon(Icons.star),
```
- ```
    Icon(Icons.star),
```
- ```
 Icon(Icons.star),
```
- ```
  ],
```
- ```
)
```
- ```
Column(
```
- ```
 crossAxisAlignment: CrossAxisAlignment.start,
```
- ```
  children: [
```
- ```
 Text('Titre'),
```
- ```
    Text('Sous-titre'),
```
- ```
],
```
- ```
)
```

3. Stack :

- Permet de superposer les widgets les uns sur les autres.
- `Stack`(
 - `children: [`
 - `Image.network('https://example.com/background.jpg'),`
 - `Positioned`(
 - `bottom: 10,`
 - `right: 10,`
 - `child: Text('Overlay Text'),`
 - `),`
 - `],`
 - `)`

4. Expanded et Flexible :

- Utilisés dans des **Row** et **Column** pour contrôler la flexibilité et l'espace occupé par les enfants.
- `Row`(
 - `children: [`
 - `Expanded`(
 - `flex: 2,`
 - `child: Container(color: Colors.red),`
 - `),`
 - `Expanded`(
 - `flex: 1,`
 - `child: Container(color: Colors.green),`
 - `),`
 - `],`
 - `)`

5. Padding :

- Ajoute de l'espace autour d'un widget.
- `Padding`(
 - `padding: EdgeInsets.all(16.0),`
 - `child: Text('Avec padding'),`
 - `)`

Personnalisation des layouts :

1. Alignement :

- Utilisez les propriétés comme **mainAxisAlignment** et **crossAxisAlignment** dans **Row** et **Column** pour aligner les enfants.
- `Column`(
 - `mainAxisAlignment: MainAxisAlignment.center,`
 - `crossAxisAlignment: CrossAxisAlignment.start,`
 - `children: [`
 - `Text('Aligné en haut'),`
 - `Text('Aligné à gauche'),`
 - `],`
 - `)`

2. SizedBox :

- Utilisé pour définir des dimensions fixes ou ajouter des espaces.
- `SizedBox`(
 - `width: 100,`
 - `height: 50,`
 - `child: ElevatedButton`(

```
o      onPressed: () {},  
o      child: Text('Bouton'),  
o    ),  
o  )
```

3. ListView :

```
o  Pour créer des listes déroulantes verticales ou horizontales.  
o  ListView(  
o    children: [  
o      ListTile(title: Text('Élément 1')),  
o      ListTile(title: Text('Élément 2')),  
o      ListTile(title: Text('Élément 3')),  
o    ],  
o  )
```

4. GridView :

```
o  Pour créer des grilles d'éléments.  
o  GridView.count(  
o    crossAxisCount: 2,  
o    children: List.generate(4, (index) {  
o      return Center(  
o        child: Text('Item $index'),  
o      );  
o    })),  
o  )
```

Responsive Design :

- Utilisez des widgets comme **MediaQuery** et **LayoutBuilder** pour créer des interfaces adaptatives.
- ```
LayoutBuilder(
 builder: (context, constraints) {
 if (constraints.maxWidth > 600) {
 return _buildLargeLayout();
 } else {
 return _buildSmallLayout();
 }
 },
)
```

## Thématisation et Styles :

- Appliquez des styles cohérents à l'aide de **Themes** pour garantir une apparence uniforme.
- ```
ThemeData(  
  primarySwatch: Colors.blue,  
  textTheme: TextTheme(  
    headline6: TextStyle(fontSize: 20.0, fontWeight:  
      FontWeight.bold),  
  ),  
)
```

Leçon 3 : Utilisation des widgets de texte, image et icône

Widgets de Texte :

- **Text** : Affiche du texte avec des options de style.
- ```
Text(
 'Bonjour Flutter',
 style: TextStyle(
 fontSize: 24,
 fontWeight: FontWeight.bold,
 color: Colors.blue,
),
)
```
- **RichText** : Permet de styliser différentes parties du texte de manière indépendante.
- ```
RichText(  
  text: TextSpan(  
    text: 'Hello ',  
    style: TextStyle(color: Colors.black),  
    children: <TextSpan>[  
      TextSpan(text: 'Flutter', style: TextStyle(fontWeight:  
FontWeight.bold)),  
    ],  
  ),  
)
```

Widgets d'Image :

- **Image.asset** : Affiche une image depuis les assets de l'application.
- ```
Image.asset('assets/images/logo.png')
```
- **Image.network** : Affiche une image depuis une URL.
- ```
Image.network('https://example.com/image.jpg')
```
- **Image.file** : Affiche une image depuis un fichier local.
- ```
Image.file(File('/path/to/image.png'))
```
- **Image avec des options de redimensionnement et de couverture.**
- ```
Image.network(  
  'https://example.com/image.jpg',  
  fit: BoxFit.cover,  
  width: 100,  
  height: 100,  
)
```

Widgets d'Icône :

- **Icon** : Affiche une icône matérielle.
- ```
Icon(
 Icons.home,
 color: Colors.blue,
 size: 30.0,
)
```

- **IconButton** : Bouton interactif avec une icône.
- ```
IconButton(  
  icon: Icon(Icons.favorite),  
  color: Colors.red,  
  onPressed: () {  
    print('Icône cliquée');  
  },  
)
```
- **Custom Icons** : Utilisez des polices d'icônes personnalisées ou des **SVG** avec des packages comme **flutter_svg**.
- ```
SvgPicture.asset('assets/icons/custom_icon.svg')
```

### Personnalisation et Thématisation :

- **Couleurs et tailles** : Ajustez les propriétés des widgets pour correspondre au thème de l'application.
- **Animations** : Intégrez des animations pour améliorer l'interaction utilisateur, comme des transitions d'icônes ou des changements de couleur dynamique.

### Accessibilité :

- **Labels** : Ajoutez des descriptions pour les lecteurs d'écran.
- ```
Icon(  
  Icons.info,  
  semanticLabel: 'Informations supplémentaires',  
)
```

Exemples d'utilisation combinée :

- Créez des composants réutilisables combinant texte, images et icônes pour construire des interfaces riches et interactives.
- ```
Row(
 children: [
 Icon(Icons.person, color: Colors.blue),
 SizedBox(width: 10),
 Text('Nom d\'utilisateur', style: TextStyle(fontSize: 18)),
],
)
```

## Leçon 4 : Widgets de conteneur et de mise en forme

### Container :

- **Boîte polyvalente** pour la mise en forme et la disposition des widgets enfants.
- ```
Container(  
  width: 150,  
  height: 150,
```

- padding: EdgeInsets.all(20),
- margin: EdgeInsets.symmetric(vertical: 10),
- decoration: BoxDecoration(
 - color: Colors.lightBlueAccent,
 - borderRadius: BorderRadius.circular(15),
 - boxShadow: [
 - BoxShadow(
 - color: Colors.grey.withOpacity(0.5),
 - spreadRadius: 5,
 - blurRadius: 7,
 - offset: Offset(0, 3),
-],
-),
- child: Text('Contenu dans un Container'),
-)

BoxDecoration :

- **Personnalisation avancée de l'apparence des conteneurs.**
- BoxDecoration(
 - color: Colors.white,
 - border: Border.all(color: Colors.blue, width: 2),
 - borderRadius: BorderRadius.circular(10),
 - gradient: LinearGradient(
 - colors: [Colors.blue, Colors.green],
 - begin: Alignment.topLeft,
 - end: Alignment.bottomRight,
 -),
 - image: DecorationImage(
 - image: AssetImage('assets/images/background.png'),
 - fit: BoxFit.cover,
 -),
 -)

Padding et Margin :

- **Padding** : Espace à l'intérieur du widget.
- Padding(
 - padding: EdgeInsets.all(16.0),
 - child: Text('Avec padding'),
 -)
- **Margin** : Espace à l'extérieur du widget.
- Container(
 - margin: EdgeInsets.symmetric(horizontal: 20),
 - child: Text('Avec margin'),
 -)

SizedBox :

- Définit des dimensions fixes ou ajoute des espaces entre les widgets.
- `SizedBox(`
- `width: 50,`
- `height: 50,`
- `child: ElevatedButton(`
- `onPressed: () {},`
- `child: Text('Bouton'),`
- `),`
- `)`

Align et Center :

- **Align** : Positionne un widget enfant selon une **fraction de l'espace disponible**.
- `Align(`
- `alignment: Alignment.bottomRight,`
- `child: Text('Aligné en bas à droite'),`
- `)`
- **Center** : Centre un widget enfant dans l'espace disponible.
- `Center(`
- `child: Text('Centré'),`
- `)`

Transform :

- Applique des transformations géométriques telles que la **rotation**, la **mise à l'échelle** et la **translation**.
- `Transform.rotate(`
- `angle: 0.1, // en radians`
- `child: Icon(Icons.rotate_right, size: 50),`
- `)`

Clip :

- **Clipping** des widgets pour créer des formes personnalisées.
- `ClipRect(`
- `borderRadius: BorderRadius.circular(15),`
- `child: Image.network('https://example.com/image.jpg'),`
- `)`

Exemples pratiques :

- **Card** : Utilisez des containers décorés pour créer des cartes d'information.
- `Card(`
- `elevation: 4,`
- `shape: RoundedRectangleBorder(`
- `borderRadius: BorderRadius.circular(10),`

```
• ),  
• child: Padding(  
•   padding: EdgeInsets.all(16),  
•   child: Column(  
•     children: [  
•       Text('Titre de la carte', style: TextStyle(fontSize: 20,  
fontWeight: FontWeight.bold)),  
•       SizedBox(height: 10),  
•       Text('Description de la carte'),  
•     ],  
•   ),  
• ),  
• )
```

Responsivité et Adaptabilité :

- Combinez les widgets de conteneur avec des widgets de layout pour créer des interfaces adaptatives.
- Utilisez des propriétés comme **constraints** et **Flex** pour ajuster la taille et la disposition des widgets en fonction de l'écran.

Leçon 5 : Gestion de l'interaction utilisateur : boutons et gestes

Widgets interactifs :

- Les interactions utilisateur sont essentielles pour une expérience utilisateur fluide. Flutter propose divers widgets pour gérer les interactions telles que les **boutons** et les **gestes**.

Boutons :

1. **ElevatedButton** :

- Bouton avec une élévation, idéal pour les actions principales.
- `ElevatedButton(`
- `onPressed: () {`
- `print('ElevatedButton cliqué');`
- `},`
- `child: Text('Cliquez-moi'),`
- `)`

2. **TextButton** :

- Bouton sans élévation, souvent utilisé dans les dialogues ou les barres d'outils.
- `TextButton(`
- `onPressed: () {`
- `print('TextButton cliqué');`
- `},`
- `child: Text('Lien'),`
- `)`

3. **OutlinedButton** :

- Bouton avec une bordure, utilisé pour les actions secondaires.
- `OutlinedButton` (
 - `onPressed: () {`
 - `print('OutlinedButton cliqué');`
 - `},`
 - `child: Text('Bouton bordé'),`
 - `)`

4. **IconButton :**

- Bouton affichant uniquement une icône.
- `IconButton` (
 - `icon: Icon(Icons.settings),`
 - `onPressed: () {`
 - `print('IconButton cliqué');`
 - `},`
 - `)`

5. **FloatingActionButton :**

- Bouton flottant, souvent utilisé pour des actions primaires dans les écrans.
- `FloatingActionButton` (
 - `onPressed: () {`
 - `print('FloatingActionButton cliqué');`
 - `},`
 - `child: Icon(Icons.add),`
 - `)`

Personnalisation des boutons :

- **Styles :** Utilisez les propriétés comme **style** pour personnaliser l'apparence des boutons.
- `ElevatedButton` (
 - `style: ElevatedButton.styleFrom(`
 - `primary: Colors.green,`
 - `padding: EdgeInsets.symmetric(horizontal: 30, vertical: 15),`
 - `textStyle: TextStyle(fontSize: 18),`
 - `),`
 - `onPressed: () {},`
 - `child: Text('Stylisé'),`
 - `)`

Gestion des gestes :

1. **GestureDetector :**

- Détecte divers types de gestes tels que les **tap**, **double tap**, **long press**, **swipe**, etc.
- `GestureDetector` (
 - `onTap: () {`
 - `print('Widget tapé');`
 - `},`
 - `onDoubleTap: () {`
 - `print('Widget double tapé');`
 - `},`
 - `onLongPress: () {`
 - `print('Widget pressé longtemps');`

```
o   },
o   child: Container(
o     color: Colors.blue,
o     width: 100,
o     height: 100,
o     child: Center(child: Text('Gestes')),
o   ),
o )
```

2. InkWell :

- o Offre un effet visuel de **ripple** lors des interactions, souvent utilisé avec des **Material widgets**.

```
o InkWell(
o   onTap: () {
o     print('InkWell tapé');
o   },
o   child: Container(
o     padding: EdgeInsets.all(20),
o     child: Text('Cliquez-moi avec effet'),
o   ),
o )
```

3. Draggable et DragTarget :

- o Permettent de créer des interactions de **glisser-déposer**.

```
o Draggable<String>(
o   data: 'Données draggées',
o   child: Container(
o     color: Colors.red,
o     width: 100,
o     height: 100,
o     child: Center(child: Text('Draggable')),
o   ),
o   feedback: Material(
o     child: Container(
o       color: Colors.red.withOpacity(0.5),
o       width: 100,
o       height: 100,
o       child: Center(child: Text('En cours de glisser')),
o     ),
o   ),
o )
o
o DragTarget<String>(
o   onAccept: (data) {
o     print('Données reçues : $data');
o   },
o   builder: (context, candidateData, rejectedData) {
o     return Container(
o       color: Colors.green,
o       width: 150,
o       height: 150,
o       child: Center(child: Text('Drag Target')),
o     );
o   },
o )
```

Gestion des formulaires :

- Utilisez des widgets interactifs comme **TextField**, **Checkbox**, et **Radio** pour créer des formulaires.
- ```
TextField(
 decoration: InputDecoration(
 labelText: 'Nom',
 border: OutlineInputBorder(),
),
 onChanged: (value) {
 print('Valeur saisie : $value');
 },
)
```

### Feedback utilisateur :

- Fournissez des retours visuels ou auditifs pour les interactions, comme des **snackbars**, **dialogs**, ou des **animations**.

### Exemples pratiques :

- **Formulaire d'inscription :**
- ```
Column(  
  children: [  
    TextField(  
      decoration: InputDecoration(labelText: 'Email'),  
    ),  
    SizedBox(height: 10),  
    TextField(  
      decoration: InputDecoration(labelText: 'Mot de passe'),  
      obscureText: true,  
    ),  
    SizedBox(height: 20),  
    ElevatedButton(  
      onPressed: () {  
        print('Inscription soumise');  
      },  
      child: Text('S\'inscrire'),  
    ),  
  ],  
)
```

Accessibilité :

- Assurez-vous que les widgets interactifs sont **accessibles** en ajoutant des **labels** et en gérant correctement le **focus**.

Leçon 6 : Introduction aux animations de base avec Flutter

Les **animations** améliorent l'expérience utilisateur en rendant les interactions plus fluides et engageantes. Flutter offre une variété de moyens pour implémenter des animations.

Types d'animations :

1. Animations implicites :

- Utilisent des widgets qui gèrent automatiquement les transitions d'état.
- Exemples : **AnimatedContainer**, **AnimatedOpacity**, **AnimatedAlign**.
- `AnimatedContainer(`
- `duration: Duration(seconds: 1),`
- `width: _largeur ? 200 : 100,`
- `height: _hauteur ? 200 : 100,`
- `color: _couleur ? Colors.blue : Colors.red,`
- `child: FlutterLogo(),`
- `)`

2. Animations explicites :

- Donnent un contrôle plus précis sur les animations en utilisant des **AnimationController** et des **Animation**.
- Nécessitent généralement un **StatefulWidget**.
- `class AnimationExplicite extends StatefulWidget {`
- `@override`
- `_AnimationExpliciteState createState() =>`
- `_AnimationExpliciteState();`
- `}`
- `class _AnimationExpliciteState extends`
- `State<AnimationExplicite> with`
- `SingleTickerProviderStateMixin {`
- `AnimationController _controller;`
- `Animation<double> _animation;`
- `@override`
- `void initState() {`
- `super.initState();`
- `_controller = AnimationController(`
- `duration: Duration(seconds: 2),`
- `vsync: this,`
- `);`
- `_animation = CurvedAnimation(parent: _controller,`
- `curve: Curves.easeIn);`
- `_controller.forward();`
- `}`
- `@override`
- `void dispose() {`
- `_controller.dispose();`
- `super.dispose();`
- `}`
- `@override`
- `Widget build(BuildContext context) {`
- `return FadeTransition(`
- `opacity: _animation,`
- `child: FlutterLogo(size: 100),`

```
○      );  
○    }  
○  }
```

Widgets d'animation courants :

1. **AnimatedContainer :**

- Anime les propriétés de **Container** comme la **taille**, la **couleur**, et la **bordure**.
- `AnimatedContainer(`
- `duration: Duration(seconds: 1),`
- `width: _toggle ? 200 : 100,`
- `height: _toggle ? 200 : 100,`
- `color: _toggle ? Colors.blue : Colors.green,`
- `child: FlutterLogo(),`
- `)`

2. **AnimatedOpacity :**

- Anime l'opacité d'un widget.
- `AnimatedOpacity(`
- `duration: Duration(seconds: 1),`
- `opacity: _visible ? 1.0 : 0.0,`
- `child: Text('Texte animé'),`
- `)`

3. **AnimatedPositioned :**

- Anime la position d'un widget au sein d'un **Stack**.
- `Stack(`
- `children: [`
- `AnimatedPositioned(`
- `duration: Duration(seconds: 1),`
- `left: _left ? 50 : 150,`
- `top: _top ? 50 : 150,`
- `child: FlutterLogo(size: 50),`
- `),`
- `],`
- `)`

4. **AnimatedAlign :**

- Anime l'alignement d'un widget enfant.
- `AnimatedAlign(`
- `duration: Duration(seconds: 1),`
- `alignment: _align ? Alignment.topLeft :`
- `Alignment.bottomRight,`
- `child: FlutterLogo(size: 50),`
- `)`

AnimationController et Animation :

- **AnimationController :**
 - Contrôle la durée et le déroulement de l'animation.
 - Nécessite une **classe mixin** comme **SingleTickerProviderStateMixin**.
- **Animation :**
 - Définit la valeur de l'animation à chaque instant.
 - Peut être couplée avec des **Curves** pour des transitions plus naturelles.

Curves :

- Flutter offre une variété de **courbes** prédéfinies pour les animations, comme **Curves.easeIn**, **Curves.bounceOut**, **Curves.linear**, etc.
- CurvedAnimation(
 - parent: _controller,
 - curve: Curves.easeInOut,
 -)

Animations basées sur les gestes :

- Combinez les animations avec des gestes pour des interactions dynamiques.
- GestureDetector(
 - onTap: () {
 - setState(() {
 - _toggle = !_toggle;
 - });
 - },
 - child: AnimatedContainer(
 - duration: Duration(seconds: 1),
 - width: _toggle ? 200 : 100,
 - height: _toggle ? 200 : 100,
 - color: _toggle ? Colors.blue : Colors.red,
 - child: FlutterLogo(),
 -),
 -)

Exemples pratiques :

1. **Animation de transition entre deux états :**
2. class ToggleAnimation extends StatefulWidget {
3. @override
4. _ToggleAnimationState createState() => _ToggleAnimationState();
5. }
- 6.
7. class _ToggleAnimationState extends State<ToggleAnimation> {
8. bool _isToggled = false;
- 9.
10. @override
11. Widget build(BuildContext context) {
12. return GestureDetector(
 - 13. onTap: () {
 - 14. setState(() {
 - 15. _isToggled = !_isToggled;
 - 16. });
 - 17. },
 - 18. child: AnimatedContainer(
 - 19. duration: Duration(milliseconds: 500),
 - 20. width: _isToggled ? 200 : 100,
 - 21. height: _isToggled ? 100 : 200,
 - 22. color: _isToggled ? Colors.green : Colors.orange,

```
23.         alignment: _isToggled ? Alignment.center :
           Alignment.topLeft,
24.         child: Text(
25.             _isToggled ? 'Toggled' : 'Untoggled',
26.             style: TextStyle(color: Colors.white),
27.         ),
28.     ),
29. );
30. }
31. }

32. Animation de rotation :
33. class RotationAnimation extends StatefulWidget {
34.     @override
35.     _RotationAnimationState createState() =>
       _RotationAnimationState();
36. }
37.
38. class _RotationAnimationState extends State<RotationAnimation>
   with SingleTickerProviderStateMixin {
39.     AnimationController _controller;
40.
41.     @override
42.     void initState() {
43.         super.initState();
44.         _controller = AnimationController(
45.             duration: Duration(seconds: 2),
46.             vsync: this,
47.         )..repeat();
48.     }
49.
50.     @override
51.     void dispose() {
52.         _controller.dispose();
53.         super.dispose();
54.     }
55.
56.     @override
57.     Widget build(BuildContext context) {
58.         return RotationTransition(
59.             turns: _controller,
60.             child: FlutterLogo(size: 100),
61.         );
62.     }
63. }
```

Optimisation des animations :

- Utilisez des animations **fluides** et évitez les animations trop complexes qui pourraient nuire aux performances.
- Privilégiez les animations **légères** et **économes en ressources**.

Conclusion : Les animations enrichissent l'interface utilisateur en ajoutant du dynamisme et en guidant l'utilisateur à travers les interactions. En maîtrisant les

différentes techniques d'animation offertes par Flutter, vous pouvez créer des applications visuellement attrayantes et réactives.

Leçon 7 : Conception d'interfaces réactives avec Flexbox et Grid

Conception réactive :

- Créer des interfaces qui s'adaptent automatiquement à différentes tailles d'écran et orientations.
- Utiliser des **layouts flexibles** pour garantir une expérience utilisateur cohérente sur tous les appareils.

Flexbox avec Row et Column :

1. Row :

- Dispose les widgets enfants horizontalement.
- **Propriétés clés :**
 - **mainAxisAlignment** : Contrôle l'alignement le long de l'axe principal (horizontal).
 - **crossAxisAlignment** : Contrôle l'alignement le long de l'axe secondaire (vertical).
 - **mainAxisSize** : Définit la taille de la Row le long de l'axe principal.
- Exemple :
- ```
Row(
 mainAxisAlignment: MainAxisAlignment.spaceBetween,
 crossAxisAlignment: CrossAxisAlignment.center,
 children: [
 Icon(Icons.menu),
 Text('Titre'),
 Icon(Icons.search),
],
)
```

##### 2. Column :

- Dispose les widgets enfants verticalement.
- **Propriétés clés :**
  - **mainAxisAlignment** : Contrôle l'alignement le long de l'axe principal (vertical).
  - **crossAxisAlignment** : Contrôle l'alignement le long de l'axe secondaire (horizontal).
  - **mainAxisSize** : Définit la taille de la Column le long de l'axe principal.
- Exemple :
- ```
Column(  
  mainAxisAlignment: MainAxisAlignment.center,  
  crossAxisAlignment: CrossAxisAlignment.start,  
  children: [  
    Text('Bienvenue'),  
    Text('à Flutter'),  
  ],  
)
```

```
○ ],  
○ )
```

Flex et Expanded :

- **Flexible :**

- Permet aux enfants de Row ou Column de s'adapter à l'espace disponible.
- ```
Row(
 children: [
 Flexible(
 flex: 2,
 child: Container(color: Colors.red, height: 100),
),
 Flexible(
 flex: 1,
 child: Container(color: Colors.green, height: 100),
),
],
)
```

- **Expanded :**

- Une version simplifiée de Flexible qui prend tout l'espace disponible.
- ```
Column(  
  children: [  
    Expanded(  
      child: Container(color: Colors.blue),  
    ),  
    Expanded(  
      child: Container(color: Colors.yellow),  
    ),  
  ],  
)
```

Grid avec GridView :

1. **GridView.count :**

- Crée une grille avec un nombre fixe de colonnes.
- ```
GridView.count(
 crossAxisCount: 3,
 children: List.generate(9, (index) {
 return Center(
 child: Text('Item $index'),
);
 }),
)
```

2. **GridView.builder :**

- Crée une grille de manière dynamique avec un constructeur.
- ```
GridView.builder(  
  gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(  
    crossAxisCount: 4,  
    mainAxisSpacing: 10,  
    crossAxisSpacing: 10,  
  ),  
  itemBuilder: (context, index) {  
    return Container(  

```

```
o         color: Colors.teal,  
o         child: Center(child: Text('Item $index')),  
o       );  
o     },  
o     itemCount: 20,  
o   )
```

3. SliverGrid :

```
o   Pour des grilles intégrées dans des CustomScrollView avec des  
o   comportements personnalisés.  
o   CustomScrollView(  
o     slivers: [  
o       SliverGrid(  
o         gridDelegate:  
o         SliverGridDelegateWithMaxCrossAxisExtent(  
o           maxCrossAxisExtent: 200,  
o           mainAxisSpacing: 10,  
o           crossAxisSpacing: 10,  
o           childAspectRatio: 1,  
o         ),  
o         delegate: SliverChildBuilderDelegate(  
o           (BuildContext context, int index) {  
o             return Container(  
o               color: Colors.orange,  
o               child: Center(child: Text('Item $index')),  
o             );  
o           },  
o           childCount: 50,  
o         ),  
o       ),  
o     ],  
o   )
```

Responsive Design avec LayoutBuilder et MediaQuery :

• LayoutBuilder :

```
o   Permet de construire des widgets en fonction des contraintes parentales.  
o   LayoutBuilder(  
o     builder: (context, constraints) {  
o       if (constraints.maxWidth > 600) {  
o         return _buildLargeScreen();  
o       } else {  
o         return _buildSmallScreen();  
o       }  
o     },  
o   )
```

• MediaQuery :

```
o   Fournit des informations sur la taille de l'écran et les préférences de  
o   l'utilisateur.  
o   Container(  
o     width: MediaQuery.of(context).size.width * 0.8,  
o     height: MediaQuery.of(context).size.height * 0.5,  
o     color: Colors.purple,  
o   )
```

Adaptation des widgets selon l'écran :

- **Utiliser des proportions** plutôt que des dimensions fixes pour garantir que l'interface s'adapte à différentes tailles d'écran.
- `Container(`
 - `width: MediaQuery.of(context).size.width * 0.5,`
 - `height: MediaQuery.of(context).size.height * 0.3,`
 - `color: Colors.blue,`
 - `)`
- **Orientation :**
 - Adapter la disposition des widgets en fonction de l'orientation de l'appareil.
 - `if (MediaQuery.of(context).orientation == Orientation.portrait) {`
 - `return Column(`
 - `children: [/* Widgets en mode portrait */],`
 - `);`
 - `} else {`
 - `return Row(`
 - `children: [/* Widgets en mode paysage */],`
 - `);`
 - `}`

Utilisation de packages pour le responsive design :

- **flutter_layout_grid** : Pour créer des grilles flexibles et responsives.
- **responsive_builder** : Pour faciliter la création d'interfaces adaptatives.

Exemples pratiques :

1. Interface adaptative avec Row et Column :

```
2. LayoutBuilder(  
3.   builder: (context, constraints) {  
4.     if (constraints.maxWidth > 600) {  
5.       return Row(  
6.         children: [  
7.           Expanded(child: _buildSidebar()),  
8.           Expanded(child: _buildContent()),  
9.         ],  
10.        );  
11.     } else {  
12.       return Column(  
13.         children: [  
14.           _buildHeader(),  
15.           _buildContent(),  
16.         ],  
17.        );  
18.     }  
19.   },  
20. )
```

21. Grille responsive avec GridView :

```
22. GridView.builder(  
23.   gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(  
24.     crossAxisCount: 2,  
25.     mainAxisSpacing: 10,  
26.     crossAxisSpacing: 10,  
27.   ),  
28.   children: [
```

```
23.   gridDelegate: SliverGridDelegateWithMaxCrossAxisExtent (
24.     maxCrossAxisExtent: 200,
25.     childAspectRatio: 3 / 2,
26.     crossAxisSpacing: 20,
27.     mainAxisSpacing: 20,
28.   ),
29.   itemBuilder: (context, index) {
30.     return Container(
31.       alignment: Alignment.center,
32.       color: Colors.teal[100 * (index % 9)],
33.       child: Text('Item $index'),
34.     );
35.   },
36.   itemCount: 20,
37. )
```

Conclusion : La conception d'interfaces réactives est cruciale pour garantir une expérience utilisateur optimale sur une variété d'appareils et de tailles d'écran. En maîtrisant les widgets de layout de Flutter et en adoptant des pratiques de design adaptatif, vous pouvez créer des applications flexibles, esthétiques et performantes.

Module 3 : Navigation et Gestion de l'État

Leçon 1 : Concepts de navigation dans Flutter : routes et transitions

La **navigation** est essentielle pour structurer le flux d'une application et permettre aux utilisateurs de se déplacer entre différents **écrans** ou **pages**.

Système de navigation de Flutter :

- Basé sur le **Navigator** et les **routes**.
- **Navigator** gère une **pile de routes** où chaque route représente un écran.

Définir des routes :

1. Routes nommées :

- Déclarez les routes dans le fichier **MaterialApp**.
- `MaterialApp(`
- `initialRoute: '/',`
- `routes: {`
- `'/': (context) => Accueil(),`
- `'/second': (context) => SecondPage(),`
- `},`
- `)`
- Naviguez vers une route nommée.
- `Navigator.pushNamed(context, '/second');`

2. Routes anonymes :

- Utilisez la méthode **MaterialPageRoute** pour naviguer vers une nouvelle page.
- `Navigator.push(`

- context,
- MaterialPageRoute(builder: (context) => SecondPage()),
-);

Navigation de base :

1. Pousser une nouvelle route :

- Ajoute une nouvelle route au sommet de la pile.
- Navigator.push(
 - context,
 - MaterialPageRoute(builder: (context) => NouvellePage()),
-);

2. Retourner à la route précédente :

- Supprime la route actuelle de la pile.
- Navigator.pop(context);

3. Remplacer la route actuelle :

- Remplace la route en cours par une nouvelle route.
- Navigator.pushReplacement(
 - context,
 - MaterialPageRoute(builder: (context) => NouvellePage()),
-);

4. Effacer la pile de navigation :

- Supprime toutes les routes précédentes et ajoute une nouvelle route.
- Navigator.pushAndRemoveUntil(
 - context,
 - MaterialPageRoute(builder: (context) => NouvellePage()),
 - (Route<dynamic> route) => false,
-);

Transitions personnalisées :

1. Transition par défaut :

- **MaterialPageRoute** utilise une transition de glissement vers la droite sur Android et une transition de **page tournante** sur iOS.

2. Transitions personnalisées avec **PageRouteBuilder** :

- Créez des transitions personnalisées en définissant les animations.
- Navigator.push(
 - context,
 - PageRouteBuilder(
 - pageBuilder: (context, animation, secondaryAnimation) => NouvellePage(),
 - transitionsBuilder: (context, animation, secondaryAnimation, child) {
 - var begin = Offset(0.0, 1.0);
 - var end = Offset.zero;
 - var curve = Curves.ease;
 -
 - var tween = Tween(begin: begin, end: end).chain(CurveTween(curve: curve));
 - var offsetAnimation = animation.drive(tween);
 -
 - return SlideTransition(

```
o         position: offsetAnimation,  
o         child: child,  
o       );  
o     },  
o   ),  
o );
```

3. Animations intégrées :

- o Utilisez des animations intégrées comme **FadeTransition**, **ScaleTransition**, et **RotationTransition**.
- o `transitionsBuilder: (context, animation, secondaryAnimation, child) {`
- o `return FadeTransition(`
- o `opacity: animation,`
- o `child: child,`
- o `);`
- o }

Navigation avec arguments :

- Passez des données entre les routes en utilisant les arguments.
- `// Pousser avec arguments`
- `Navigator.pushNamed(`
- `context,`
- `'/second',`
- `arguments: 'Données passées',`
- `);`
-
- `// Recevoir les arguments`
- `class SecondPage extends StatelessWidget {`
- `@override`
- `Widget build(BuildContext context) {`
- `final args = ModalRoute.of(context).settings.arguments as`
- `String;`
- `return Scaffold(`
- `appBar: AppBar(title: Text('Second Page')),`
- `body: Center(child: Text(args)),`
- `);`
- `}`
- `}`

Navigation imbriquée :

- Gérez la navigation au sein de sections spécifiques de l'application, comme les onglets ou les **Drawer**.
- `Navigator.of(context).push(`
- `MaterialPageRoute(builder: (context) => DetailsPage()),`
- `);`

Navigation avec BottomNavigationBar :

- Intégrez la navigation avec une barre de navigation inférieure pour basculer entre différents écrans.

```
• class HomePage extends StatefulWidget {
•   @override
•   _HomePageState createState() => _HomePageState();
• }
•
• class _HomePageState extends State<HomePage> {
•   int _selectedIndex = 0;
•   static List<Widget> _widgetOptions = <Widget>[
•     Page1(),
•     Page2(),
•     Page3(),
•   ];
•
•   void _onItemTapped(int index) {
•     setState(() {
•       _selectedIndex = index;
•     });
•   }
•
•   @override
•   Widget build(BuildContext context) {
•     return Scaffold(
•       body: Center(
•         child: _widgetOptions.elementAt(_selectedIndex),
•       ),
•       bottomNavigationBar: BottomNavigationBar(
•         items: const <BottomNavigationBarItem>[
•           BottomNavigationBarItem(icon: Icon(Icons.home), label:
'Accueil'),
•           BottomNavigationBarItem(icon: Icon(Icons.business),
label: 'Business'),
•           BottomNavigationBarItem(icon: Icon(Icons.school),
label: 'École'),
•         ],
•         currentIndex: _selectedIndex,
•         onTap: _onItemTapped,
•       ),
•     );
•   }
• }
```

Navigation avec Drawer :

- Utilisez un **Drawer** pour offrir un menu de navigation latéral.
- ```
• Scaffold(
• appBar: AppBar(title: Text('Page avec Drawer')),
• drawer: Drawer(
```

```
• child: ListView(
• children: [
• DrawerHeader(child: Text('Menu')),
• ListTile(
• title: Text('Accueil'),
• onTap: () {
• Navigator.pushNamed(context, '/');
• },
•),
• ListTile(
• title: Text('Paramètres'),
• onTap: () {
• Navigator.pushNamed(context, '/settings');
• },
•),
•],
•),
•),
• body: Center(child: Text('Contenu principal')),
•)
```

### Deep Linking :

- Permet aux utilisateurs de naviguer directement vers des pages spécifiques via des URLs.
- Implémentation avancée nécessitant la configuration des routes et la gestion des liens profonds.

### Gestion des erreurs de navigation :

- Gérez les cas où une route n'existe pas en définissant une **page 404**.
- MaterialApp(  
• onUnknownRoute: (settings) {  
• return MaterialPageRoute(builder: (context) => Page404());  
• },  
• )

### Bonnes pratiques :

- **Maintenez une gestion claire des routes** pour éviter les conflits et faciliter la maintenance.
- **Utilisez des routes nommées** pour une navigation plus lisible et structurée.
- **Gérez les transitions** de manière cohérente avec le design de l'application.
- **Passez les données de manière sécurisée** et évitez de surcharger les routes avec trop d'informations.

**Conclusion :** La navigation est un aspect fondamental du développement d'applications Flutter. En maîtrisant les concepts de routes, les transitions, et la gestion des états de

navigation, vous pouvez créer des flux d'utilisateur intuitifs et efficaces, garantissant une expérience utilisateur optimale.

### Leçon 2 : Gestion de l'état local avec `setState` et les `StatefulWidget`

La gestion de l'état est cruciale pour maintenir et mettre à jour les données dynamiques au sein d'une application Flutter. **Flutter** propose plusieurs méthodes pour gérer l'état, dont `setState` et les **`StatefulWidget`** pour la gestion d'état local.

#### **StatefulWidget vs StatelessWidget :**

- **StatelessWidget** : Utilisé pour des widgets qui ne changent pas d'état après leur création.
- **StatefulWidget** : Utilisé pour des widgets qui peuvent changer d'état au fil du temps.

#### **Structure d'un StatefulWidget :**

```
class MonStatefulWidget extends StatefulWidget {
 @override
 _MonStatefulWidgetState createState() => _MonStatefulWidgetState();
}

class _MonStatefulWidgetState extends State<MonStatefulWidget> {
 // Variables d'état
 int _compteur = 0;

 // Méthode pour modifier l'état
 void _incrémenter() {
 setState(() {
 _compteur++;
 });
 }

 @override
 Widget build(BuildContext context) {
 return Column(
 children: [
 Text('Compteur : $_compteur'),
 ElevatedButton(
 onPressed: _incrémenter,
 child: Text('Incrémenter'),
),
],
);
 }
}
```

#### **Utilisation de `setState` :**

- **setState** notifie Flutter que l'état interne du widget a changé, déclenchant une reconstruction de l'interface utilisateur.
- Il est crucial d'appeler **setState** uniquement dans le **State** de **StatefulWidget**.

### Exemple de **setState** :

```
class ToggleSwitch extends StatefulWidget {
 @override
 _ToggleSwitchState createState() => _ToggleSwitchState();
}

class _ToggleSwitchState extends State<ToggleSwitch> {
 bool _estActif = false;

 void _toggle() {
 setState(() {
 _estActif = !_estActif;
 });
 }

 @override
 Widget build(BuildContext context) {
 return Switch(
 value: _estActif,
 onChanged: (value) {
 _toggle();
 },
);
 }
}
```

### Gestion de l'état local :

- **Variables d'état** : Définissez les variables qui représentent l'état interne du widget.
- **Mises à jour de l'état** : Utilisez **setState** pour modifier ces variables et mettre à jour l'interface utilisateur.

### Cycle de vie d'un **StatefulWidget** :

1. **createState** : Création de l'objet d'état.
2. **initState** : Initialisation de l'état.
3. **build** : Construction de l'interface utilisateur.
4. **setState** : Mise à jour de l'état et reconstruction de l'interface.
5. **dispose** : Nettoyage des ressources.

### Bonnes pratiques :

- **Minimiser l'utilisation de **setState**** : Évitez les appels inutiles pour optimiser les performances.

- **Encapsuler l'état** : Gardez l'état local aussi proche que possible des widgets qui en dépendent.
- **Immutabilité** : Lorsque cela est possible, utilisez des objets immuables pour représenter l'état.

### Exemple pratique :

Créer un widget de **compteur** simple utilisant **StatefulWidget** et **setState**.

```
class CompteurSimple extends StatefulWidget {
 @override
 _CompteurSimpleState createState() => _CompteurSimpleState();
}

class _CompteurSimpleState extends State<CompteurSimple> {
 int _compteur = 0;

 void _incrémenter() {
 setState(() {
 _compteur++;
 });
 }

 @override
 Widget build(BuildContext context) {
 return Column(
 mainAxisAlignment: MainAxisAlignment.center,
 children: [
 Text(
 'Vous avez appuyé sur le bouton cette quantité de fois : ',
),
 Text(
 '$_compteur',
 style: Theme.of(context).textTheme.headline4,
),
 SizedBox(height: 20),
 ElevatedButton(
 onPressed: _incrémenter,
 child: Text('Incrémenter'),
),
],
);
 }
}
```

### Limitations de **setState** :

- **Propager l'état** : **setState** est limité à la gestion de l'état local. Pour gérer l'état global ou partagé entre plusieurs widgets, des solutions plus avancées sont nécessaires.

**Conclusion** : La gestion de l'état local avec **setState** et **StatefulWidget** est simple et efficace pour des scénarios simples où l'état est limité à un widget spécifique. Cependant,

pour des applications plus complexes nécessitant une gestion d'état partagée ou globale, il est recommandé d'utiliser des solutions de gestion d'état plus robustes.

### Leçon 3 : Approfondissement sur Provider pour la gestion de l'état global

La gestion de l'état global est cruciale pour les applications Flutter de taille moyenne à grande, où plusieurs widgets doivent accéder et modifier des données partagées.

**Provider** est une bibliothèque recommandée pour gérer efficacement l'état global dans Flutter.

#### Introduction à Provider :

- **Provider** est une solution de gestion d'état basée sur l'injection de dépendances, facilitant le partage de données entre widgets sans passer par une chaîne complexe de paramètres.
- Il est maintenu par la communauté Flutter et est bien intégré dans l'écosystème Flutter.

#### Installation de Provider :

1. **Ajouter la dépendance** dans le fichier `pubspec.yaml`.
2. `dependencies:`
3. `provider: ^6.0.0`
4. **Installer les packages** avec la commande `flutter pub get`.

#### Concepts clés de Provider :

##### 1. **ChangeNotifier** :

- Un modèle de classe qui notifie les écouteurs lorsque l'état change.
- `class Compteur with ChangeNotifier {`
- `int _valeur = 0;`
- `int get valeur => _valeur;`
- `void incrementer() {`
- `_valeur++;`
- `notifyListeners();`
- `}`
- `}`

##### 2. **ChangeNotifierProvider** :

- Fournit une instance de **ChangeNotifier** à l'arborescence des widgets.
- `void main() {`
- `runApp(`
- `ChangeNotifierProvider(`
- `create: (context) => Compteur(),`
- `child: MonApp(),`
- `),`
- `);`
- `}`

##### 3. **Consumer** :

- Un widget qui écoute les changements dans le modèle et reconstruit automatiquement quand une notification est reçue.
- `Consumer<Compteur>({`
- `builder: (context, compteur, child) {`
- `return Text('Valeur : ${compteur.valeur}');`
- `},`
- `)`

#### 4. **Provider.of :**

- Permet d'accéder au modèle depuis n'importe quel widget descendant.
- `ElevatedButton({`
- `onPressed: () {`
- `Provider.of<Compteur>(context, listen:`
- `false).incrémenter();`
- `},`
- `child: Text('Incrémenter'),`
- `)`

### Implémentation pratique :

#### 1. Définir le modèle :

```
2. class Auth with ChangeNotifier {
3. String _token;
4. String get token => _token;
5.
6. bool get isAuthenticated => _token != null;
7.
8. void login(String token) {
9. _token = token;
10. notifyListeners();
11. }
12.
13. void logout() {
14. _token = null;
15. notifyListeners();
16. }
17. }
```

#### 18. Fournir le modèle :

```
19. void main() {
20. runApp(
21. MultiProvider(
22. providers: [
23. ChangeNotifierProvider(create: (_) => Auth()),
24. ChangeNotifierProvider(create: (_) => Compteur()),
25.],
26. child: MyApp(),
27.),
28.);
29. }
```

#### 30. Consommer le modèle :

```
31. class Accueil extends StatelessWidget {
32. @override
33. Widget build(BuildContext context) {
34. return Scaffold(
35. appBar: AppBar(title: Text('Accueil')),
```

```
36. body: Center(
37. child: Column(
38. mainAxisAlignment: MainAxisAlignment.center,
39. children: [
40. Consumer<Compteur>(
41. builder: (context, compteur, child) =>
Text('Compteur : ${compteur.valeur}'),
42.),
43. SizedBox(height: 20),
44. ElevatedButton(
45. onPressed: () {
46. Provider.of<Compteur>(context, listen:
false).incrementer();
47. },
48. child: Text('Incrémenter'),
49.),
50.],
51.),
52.),
53.);
54. }
55. }
```

### Avantages de Provider :

- **Simplicité** : Facile à comprendre et à implémenter pour les développeurs Flutter.
- **Performance** : Optimise la reconstruction des widgets en ne reconstruisant que les widgets qui en ont besoin.
- **Flexibilité** : Peut être combiné avec d'autres solutions de gestion d'état si nécessaire.

### Utilisation avancée :

#### 1. Selector :

- Sélectionnez une partie spécifique du modèle pour éviter les reconstructions inutiles.
- `Selector<Compteur, int>(  
○ selector: (context, compteur) => compteur.valeur,  
○ builder: (context, valeur, child) {  
○ return Text('Valeur sélectionnée : $valeur');  
○ },  
○ )`

#### 2. ProxyProvider :

- Permet de créer un provider dépendant d'un autre provider.
- `ProxyProvider<Auth, DataService>(  
○ update: (context, auth, previous) =>  
DataService(auth.token),  
○ )`

#### 3. Multiple Providers :

- Utilisez **MultiProvider** pour fournir plusieurs modèles à l'arborescence des widgets.
- `MultiProvider(  
○ child: Text('Hello World'),  
○ )`

```
o providers: [
o ChangeNotifierProvider(create: (_) => Auth()),
o ChangeNotifierProvider(create: (_) => Compteur()),
o],
o child: MyApp(),
o)
```

### Bonnes pratiques :

- **Encapsulation** : Gardez la logique métier séparée de la présentation en utilisant des modèles dédiés.
- **Évitez la duplication** : Utilisez **MultiProvider** pour fournir plusieurs modèles sans dupliquer le code.
- **Optimisez les reconstructions** : Utilisez **Consumer** et **Selector** pour cibler précisément les widgets à reconstruire.

### Exemple avancé :

Créer une application de **todo list** avec **Provider**.

```
// Modèle Todo
class Todo with ChangeNotifier {
 String titre;
 bool estTermine;

 Todo({this.titre, this.estTermine = false});

 void toggleTermine() {
 estTermine = !estTermine;
 notifyListeners();
 }
}

// Modèle ListeTodo
class ListeTodo with ChangeNotifier {
 List<Todo> _todos = [];

 List<Todo> get todos => _todos;

 void ajouterTodo(String titre) {
 _todos.add(Todo(titre: titre));
 notifyListeners();
 }

 void supprimerTodo(int index) {
 _todos.removeAt(index);
 notifyListeners();
 }
}

// Fournir ListeTodo
void main() {
 runApp(
 ChangeNotifierProvider(

```

```

 create: (_) => ListeTodo(),
 child: MyApp(),
),
);
 }

// Consommer ListeTodo
class TodoPage extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Todo List'),
),
 body: Consumer<ListeTodo>(
 builder: (context, listeTodo, child) {
 return ListView.builder(
 itemCount: listeTodo.todos.length,
 itemBuilder: (context, index) {
 final todo = listeTodo.todos[index];
 return ListTile(
 title: Text(
 todo.titre,
 style: TextStyle(
 decoration: todo.estTermine ?
TextDecoration.lineThrough : null,
),
),
 trailing: Checkbox(
 value: todo.estTermine,
 onChanged: (value) {
 todo.toggleTermine();
 },
),
 onLongPress: () {
 listeTodo.supprimerTodo(index);
 },
);
 },
);
 },
),
 floatingActionButton: FloatingActionButton(
 onPressed: () {
 _afficherDialogAjouter(context);
 },
 child: Icon(Icons.add),
),
);
 }

void _afficherDialogAjouter(BuildContext context) {
 String nouveauTitre = '';
 showDialog(
 context: context,
 builder: (context) => AlertDialog(
 title: Text('Ajouter une tâche'),

```

```
content: TextField(
 onChanged: (value) {
 nouveauTitre = value;
 },
 decoration: InputDecoration(hintText: 'Titre de la tâche'),
) ,
actions: [
 TextButton(
 onPressed: () {
 Provider.of<ListeTodo>(context, listen:
false).ajouterTodo(nouveauTitre);
 Navigator.pop(context);
 },
 child: Text('Ajouter'),
),
],
) ,
) ;
}
```

**Conclusion : Provider** est une solution puissante et flexible pour gérer l'état global dans les applications Flutter. En comprenant et en maîtrisant les concepts clés de Provider, vous pouvez créer des applications maintenables, évolutives et performantes, facilitant la gestion des données partagées et améliorant l'architecture globale de votre projet.

### Leçon 4 : Modèle de BLoC (Business Logic Component) et son intégration

Le modèle **BLoC (Business Logic Component)** est une architecture de gestion d'état qui sépare la logique métier de la présentation, facilitant ainsi la maintenance et les tests des applications Flutter. **BLoC** utilise des **streams** pour gérer les flux de données et les événements.

#### Concepts clés de BLoC :

1. **Streams :**
  - Flux de données asynchrones permettant d'écouter et de réagir aux changements.
  - Utilisés pour émettre des **données** et des **états**.
2. **Sinks :**
  - Points d'entrée des données dans le stream, permettant d'envoyer des événements ou des actions.
3. **Bloc :**
  - Gestionnaire qui reçoit les événements via les **sinks**, traite la logique métier, et émet les états via les **streams**.

#### Installation des dépendances :

1. **Ajouter les packages nécessaires :**
2. `dependencies:`

3. flutter\_bloc: ^8.0.0
4. bloc: ^8.0.0
5. **Installer les packages** avec la commande **flutter pub get**.

### Structure d'un BLoC :

#### 1. Définir les événements :

- Représentent les actions ou les interactions utilisateur.
- `abstract class CompteurEvent {}`
- 
- `class Incrementer extends CompteurEvent {}`
- `class Decrementer extends CompteurEvent {}`

#### 2. Définir les états :

- Représentent les différentes configurations ou données de l'interface utilisateur.
- `abstract class CompteurState {}`
- 
- `class CompteurInitial extends CompteurState {`
- `final int valeur = 0;`
- `}`
- 
- `class CompteurModifie extends CompteurState {`
- `final int valeur;`
- 
- `CompteurModifie(this.valeur);`
- `}`

#### 3. Créer le BLoC :

- Gère les événements et émet les états.
- `class CompteurBloc extends Bloc<CompteurEvent, CompteurState> {`
- `CompteurBloc() : super(CompteurInitial()) {`
- `on<Incrementer>((event, emit) {`
- `if (state is CompteurInitial) {`
- `emit(CompteurModifie(1));`
- `} else if (state is CompteurModifie) {`
- `final currentValue = (state as`
- `CompteurModifie).valeur;`
- `emit(CompteurModifie(currentValue + 1));`
- `}`
- `});`
- 
- `on<Decrementer>((event, emit) {`
- `if (state is CompteurModifie) {`
- `final currentValue = (state as`
- `CompteurModifie).valeur;`
- `emit(CompteurModifie(currentValue - 1));`
- `}`
- `});`
- `}`
- `}`

### Intégration avec Flutter :

### 1. Fournir le BLoC :

- Utilisez **BlocProvider** pour rendre le BLoC accessible aux widgets descendants.
- ```
void main() {  
  runApp(  
    BlocProvider(  
      create: (context) => CompteurBloc(),  
      child: MyApp(),  
    ),  
  );  
}
```

2. Consommer le BLoC :

- Utilisez **BlocBuilder** pour reconstruire les widgets en fonction des états.
- ```
class CompteurPage extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(title: Text('BLoC Compteur')),
 body: Center(
 child: BlocBuilder<CompteurBloc, CompteurState>(
 builder: (context, state) {
 int valeur = 0;
 if (state is CompteurInitial) {
 valeur = state.valeur;
 } else if (state is CompteurModifie) {
 valeur = state.valeur;
 }
 return Text(
 'Compteur : $valeur',
 style: TextStyle(fontSize: 24),
);
 },
),
),
 floatingActionButton: Column(
 mainAxisAlignment: MainAxisAlignment.end,
 children: [
 FloatingActionButton(
 onPressed: () {
 context.read<CompteurBloc>().add(Incrementer());
 },
 child: Icon(Icons.add),
),
 SizedBox(height: 10),
 FloatingActionButton(
 onPressed: () {
 context.read<CompteurBloc>().add(Decrementer());
 },
 child: Icon(Icons.remove),
),
],
),
);
 }
}
```

- }

### Gestion des événements complexes :

- Les BLoC permettent de gérer des flux d'événements complexes, comme les appels API, les transformations de données, ou les interactions utilisateur multiples.

### Bonnes pratiques :

- **Séparation des responsabilités** : Gardez la logique métier dans le BLoC et la présentation dans les widgets.
- **Testabilité** : Les BLoC facilitent les tests unitaires en isolant la logique métier.
- **Modularité** : Créez des BLoC séparés pour différentes fonctionnalités de l'application.

### Exemple avancé :

Créer une application de **gestion de tâches** utilisant BLoC.

```
// Définir les événements
abstract class TacheEvent {}

class AjouterTache extends TacheEvent {
 final String titre;

 AjouterTache(this.titre);
}

class SupprimerTache extends TacheEvent {
 final int index;

 SupprimerTache(this.index);
}

// Définir les états
abstract class TacheState {}

class TacheInitial extends TacheState {
 final List<String> taches = [];
}

class TacheMisAJour extends TacheState {
 final List<String> taches;

 TacheMisAJour(this.taches);
}

// Créer le BLoC
class TacheBloc extends Bloc<TacheEvent, TacheState> {
 TacheBloc() : super(TacheInitial()) {
 on<AjouterTache>((event, emit) {
```

## Développement d'Applications Flutter

```
 final currentState = state;
 if (currentState is TacheInitial) {
 final nouvellesTaches =
List<String>.from(currentState.taches)..add(event.titre);
 emit(TacheMisAJour(nouvellesTaches));
 } else if (currentState is TacheMisAJour) {
 final nouvellesTaches =
List<String>.from(currentState.taches)..add(event.titre);
 emit(TacheMisAJour(nouvellesTaches));
 }
 });

 on<SupprimerTache>((event, emit) {
 final currentState = state;
 if (currentState is TacheMisAJour) {
 final nouvellesTaches =
List<String>.from(currentState.taches)..removeAt(event.index);
 emit(TacheMisAJour(nouvellesTaches));
 }
 });
}

// Intégrer le BLoC dans Flutter
void main() {
 runApp(
 BlocProvider(
 create: (context) => TacheBloc(),
 child: MyApp(),
),
);
}

// Consommer le BLoC
class TachePage extends StatelessWidget {
 final TextEditingController _controller = TextEditingController();

 @override
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(title: Text('Gestion de Tâches')),
 body: Column(
 children: [
 Padding(
 padding: EdgeInsets.all(16),
 child: Row(
 children: [
 Expanded(
 child: TextField(
 controller: _controller,
 decoration: InputDecoration(labelText: 'Nouvelle
Tâche'),
),
),
 IconButton(
 icon: Icon(Icons.add),
 onPressed: () {
```

## Développement d'Applications Flutter

```
 if (_controller.text.isNotEmpty) {
context.read<TacheBloc>().add(AjouterTache(_controller.text));
 _controller.clear();
 }
 },
),
],
),
),
Expanded(
 child: BlocBuilder<TacheBloc, TacheState>(
 builder: (context, state) {
 if (state is TacheInitial || state is TacheMisAJour) {
 final taches = state is TacheMisAJour ? state.taches
: [];

 return ListView.builder(
 itemCount: taches.length,
 itemBuilder: (context, index) {
 return ListTile(
 title: Text(taches[index]),
 trailing: IconButton(
 icon: Icon(Icons.delete),
 onPressed: () {
context.read<TacheBloc>().add(SupprimerTache(index));
 },
),
);
 },
);
 }
 return Center(child: Text('Aucune tâche'));
 },
),
),
],
),
);
}
```

**Conclusion :** Le modèle **BLoC** offre une approche structurée pour gérer la logique métier et l'état dans les applications Flutter. En séparant clairement la logique de présentation et en utilisant des **streams** pour gérer les flux de données, BLoC améliore la maintenabilité, la testabilité et la scalabilité des applications. Bien que plus complexe que des solutions comme **Provider**, BLoC est particulièrement adapté aux applications nécessitant une gestion d'état robuste et des interactions complexes.

## Leçon 5 : Utilisation de Riverpod pour la gestion avancée de l'état

**Introduction à Riverpod :** Riverpod est une bibliothèque de gestion d'état pour Flutter, créée par le même développeur que Provider, mais avec des améliorations significatives.

Elle offre une meilleure sécurité de type, une gestion de la dépendance plus flexible, et élimine certaines limitations de Provider.

### Avantages de Riverpod :

- **Sécurité de type améliorée.**
- **Pas de contexte nécessaire** pour accéder aux providers.
- **Facilité de test** grâce à l'isolation des providers.
- **Support pour les providers asynchrones.**

### Installation de Riverpod :

1. **Ajouter les dépendances** dans le fichier `pubspec.yaml`.
2. `dependencies:`
3. `flutter_riverpod: ^2.0.0`
4. **Installer les packages** avec la commande `flutter pub get`.

### Concepts clés de Riverpod :

1. **Provider :**
  - Déclarez un provider pour exposer une valeur ou un état.
  - `final compteurProvider = StateProvider<int>((ref) => 0);`
2. **ConsumerWidget :**
  - Un widget qui peut lire les providers.
  - `class CompteurWidget extends ConsumerWidget {`
  - `@override`
  - `Widget build(BuildContext context, WidgetRef ref) {`
  - `final compteur = ref.watch(compteurProvider);`
  - `return Text('Compteur : $compteur');`
  - `}`
  - `}`
3. **WidgetRef :**
  - Remplace **BuildContext** pour lire ou modifier les providers.
  - `ElevatedButton(`
  - `onPressed: () {`
  - `ref.read(compteurProvider.notifier).state++;`
  - `},`
  - `child: Text('Incrémenter'),`
  - `)`

### Types de providers :

1. **Provider :**
  - Fournit une valeur statique ou calculée.
  - `final helloProvider = Provider((ref) => 'Bonjour Riverpod');`
2. **StateProvider :**
  - Gère un état mutable simple.
  - `final compteurProvider = StateProvider<int>((ref) => 0);`
3. **FutureProvider :**

- Gère des opérations asynchrones.
- `final dataProvider = FutureProvider<String>((ref) async {`
- `await Future.delayed(Duration(seconds: 2));`
- `return 'Données chargées';`
- `});`

#### 4. StreamProvider :

- Gère des flux de données asynchrones.
- `final streamProvider = StreamProvider<int>((ref) async* {`
- `for (int i = 0; i < 10; i++) {`
- `await Future.delayed(Duration(seconds: 1));`
- `yield i;`
- `}`
- `});`

#### 5. ChangeNotifierProvider :

- Fournit une instance de **ChangeNotifier**.
- `class AuthNotifier extends ChangeNotifier {`
- `String _token;`
- `String get token => _token;`
- 
- `void login(String token) {`
- `_token = token;`
- `notifyListeners();`
- `}`
- 
- `void logout() {`
- `_token = null;`
- `notifyListeners();`
- `}`
- `}`
- 
- `final authProvider = ChangeNotifierProvider((ref) =>`
- `AuthNotifier());`

### Utilisation de Riverpod :

#### 1. Envelopper l'application avec ProviderScope :

```
2. void main() {
3. runApp(
4. ProviderScope(
5. child: MyApp(),
6.),
7.);
8. }
```

#### 9. Lire un provider :

- Utilisez **ref.watch** pour écouter les changements.
- `final compteur = ref.watch(compteurProvider);`

#### 10. Modifier un provider :

- Utilisez **ref.read** pour accéder au notifier et modifier l'état.
- `ref.read(compteurProvider.notifier).state++;`

### Gestion des dépendances :

- **Riverpod** facilite la déclaration et l'utilisation des dépendances entre providers.

## Développement d'Applications Flutter

- `final repositoryProvider = Provider<Repository>((ref) => Repository());`
- 
- `final dataProvider = FutureProvider<String>((ref) async {`
- `final repository = ref.read(repositoryProvider);`
- `return repository.fetchData();`
- `});`

### Providers asynchrones :

- **FutureProvider** et **StreamProvider** gèrent les opérations asynchrones, simplifiant l'intégration des appels API et des flux de données en temps réel.
- `class DataService {`
- `Future<String> fetchData() async {`
- `await Future.delayed(Duration(seconds: 2));`
- `return 'Données asynchrones';`
- `}`
- `}`
- 
- `final dataServiceProvider = Provider((ref) => DataService());`
- 
- `final asyncDataProvider = FutureProvider<String>((ref) async {`
- `final service = ref.read(dataServiceProvider);`
- `return service.fetchData();`
- `});`

### Consommer des providers asynchrones :

- Utilisez **AsyncValue** pour gérer les états de chargement, de succès et d'erreur.
- `class AsyncDataWidget extends ConsumerWidget {`
- `@override`
- `Widget build(BuildContext context, WidgetRef ref) {`
- `final asyncData = ref.watch(asyncDataProvider);`
- 
- `return asyncData.when(`
- `data: (data) => Text('Données : $data'),`
- `loading: () => CircularProgressIndicator(),`
- `error: (err, stack) => Text('Erreur : $err'),`
- `);`
- `}`
- `}`

### Testing avec Riverpod :

- **ProviderScope** peut être personnalisé pour injecter des mocks ou des instances spécifiques lors des tests.
- `testWidgets('Test CompteurWidget', (WidgetTester tester) async {`
- `await tester.pumpWidget(`

```
• ProviderScope(
• overrides: [
• compteurProvider.overrideWithValue(StateController(5)),
•],
• child: MaterialApp(
• home: CompteurWidget(),
•),
•),
•);
•
• expect(find.text('Compteur : 5'), findsOneWidget);
• });
```

### Exemple pratique :

Créer une application de **gestion de favoris** utilisant Riverpod.

```
// Définir le provider
final favorisProvider = StateProvider<List<String>>((ref) => []);

// Widget pour afficher les favoris
class FavorisPage extends ConsumerWidget {
 @override
 Widget build(BuildContext context, WidgetRef ref) {
 final favoris = ref.watch(favorisProvider).state;

 return Scaffold(
 appBar: AppBar(title: Text('Favoris')),
 body: ListView.builder(
 itemCount: favoris.length,
 itemBuilder: (context, index) {
 return ListTile(
 title: Text(favoris[index]),
 trailing: IconButton(
 icon: Icon(Icons.remove_circle),
 onPressed: () {
 ref.read(favorisProvider).state.removeAt(index);
 ref.read(favorisProvider).state =
List.from(ref.read(favorisProvider).state);
 },
),
);
 },
),
 floatingActionButton: FloatingActionButton(
 onPressed: () {
 _afficherDialogAjouter(context, ref);
 },
 child: Icon(Icons.add),
),
);
 }

 void _afficherDialogAjouter(BuildContext context, WidgetRef ref) {
```

```
String nouveauFavori = '';
showDialog(
 context: context,
 builder: (context) => AlertDialog(
 title: Text('Ajouter un Favori'),
 content: TextField(
 onChanged: (value) {
 nouveauFavori = value;
 },
 decoration: InputDecoration(hintText: 'Nom du favori'),
),
 actions: [
 TextButton(
 onPressed: () {
 if (nouveauFavori.isNotEmpty) {
 ref.read(favorisProvider).state =
[...ref.read(favorisProvider).state, nouveauFavori];
 }
 Navigator.pop(context);
 },
 child: Text('Ajouter'),
),
],
),
);
}

// Intégrer le widget dans l'application
class MyApp extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
 return MaterialApp(
 title: 'Gestion de Favoris avec Riverpod',
 home: FavorisPage(),
);
 }
}
```

**Conclusion : Riverpod** offre une approche moderne et puissante pour la gestion de l'état dans les applications Flutter, surpassant certaines limitations de Provider. Avec une syntaxe claire, une meilleure gestion des dépendances et une intégration facile avec les opérations asynchrones, Riverpod est une excellente option pour les développeurs cherchant une solution de gestion d'état robuste et flexible. En maîtrisant Riverpod, vous pouvez créer des applications Flutter évolutives, maintenables et hautement performantes.

## Leçon 6 : Structurer et organiser une application pour une navigation fluide

Une bonne **structure** et organisation de l'application sont essentielles pour garantir une **navigation fluide**, une **maintenabilité** et une **évolutivité** optimales. Cette leçon explore

les meilleures pratiques pour structurer une application Flutter, organiser les fichiers et gérer la navigation de manière efficace.

### Architecture de l'application :

1. **MVC (Model-View-Controller) :**
  - **Modèle :** Gère les données et la logique métier.
  - **Vue :** Interface utilisateur.
  - **Contrôleur :** Intermédiaire entre le modèle et la vue.
2. **MVVM (Model-View-ViewModel) :**
  - **Modèle :** Données et logique métier.
  - **Vue :** Interface utilisateur.
  - **ViewModel :** Gère l'état et les interactions de la vue.
3. **Clean Architecture :**
  - Sépare l'application en couches distinctes (présentation, domaine, données) pour une meilleure organisation et testabilité.

### Organisation des fichiers :

1. **Structure de base :**
  - **lib/**
    - **main.dart** : Point d'entrée de l'application.
    - **models/** : Classes de données et modèles.
    - **views/** : Widgets de l'interface utilisateur.
    - **controllers/** ou **viewmodels/** : Gestionnaires de logique.
    - **services/** : Services pour les appels API, la gestion de base de données, etc.
    - **providers/** : Providers pour la gestion d'état.
    - **widgets/** : Widgets réutilisables.
    - **utils/** : Fonctions utilitaires et constantes.

2. **Exemple de structure :**

```
3. lib/
4. |— main.dart
5. |— models/
6. | |— utilisateur.dart
7. | |— tache.dart
8. |— views/
9. | |— accueil.dart
10. | |— detail_tache.dart
11. |— controllers/
12. | |— tache_controller.dart
13. |— services/
14. | |— api_service.dart
15. |— providers/
16. | |— compteur_provider.dart
17. |— widgets/
18. | |— header.dart
19. | |— custom_button.dart
20. |— utils/
21. | |— constantes.dart
```

22.       └─ helpers.dart

### Gestion de la navigation :

#### 1. Définir un routeur centralisé :

- Centralisez la définition des routes pour une gestion simplifiée.
- ```
class AppRouter {
```
- ```
static const String accueil = '/';
```
- ```
static const String detailTache = '/detailTache';
```
- ```
Route<dynamic> onGenerateRoute(RouteSettings settings) {
```
- ```
switch (settings.name) {
```
- ```
case accueil:
```
- ```
return MaterialPageRoute(builder: (context) =>
```
- ```
Accueil());
```
- ```
case detailTache:
```
- ```
final tache = settings.arguments as Tache;
```
- ```
return MaterialPageRoute(builder: (context) =>
```
- ```
DetailTache(tache: tache));
```
- ```
default:
```
- ```
return MaterialPageRoute(builder: (context) =>
```
- ```
Page404());
```
- ```
}
```
- ```
}
```
- }

2. Intégrer le routeur dans MaterialApp :

```
3. void main() {
4.   runApp(MyApp());
5. }
6.
7. class MyApp extends StatelessWidget {
8.   final AppRouter _appRouter = AppRouter();
9.
10.   @override
11.   Widget build(BuildContext context) {
12.     return MaterialApp(
13.       title: 'Application Structurée',
14.       onGenerateRoute: _appRouter.onGenerateRoute,
15.       initialRoute: AppRouter.accueil,
16.     );
17.   }
18. }
```

19. Naviguer avec des routes nommées :

```
20. Navigator.pushNamed(
21.   context,
22.   AppRouter.detailTache,
23.   arguments: tacheInstance,
24. );
```

Gestion de l'état et navigation :

- Intégrez la gestion d'état avec la navigation pour assurer que l'état est correctement partagé et mis à jour entre les différentes routes.
- Exemple avec **Provider** ou **Riverpod** :

```
• // Dans le routeur
• case AppRouter.detailTache:
•   final tache = settings.arguments as Tache;
•   return MaterialPageRoute(
•     builder: (context) => ChangeNotifierProvider(
•       create: (_) => TacheDetailViewModel(tache),
•       child: DetailTache(),
•     ),
•   );
```

Utilisation des Navigation 2.0 :

- Pour des besoins de navigation avancés, utilisez **Router** et **RouteInformationParser** pour une gestion plus fine des routes et des URL.

Modularisation de l'application :

- Divisez l'application en **modules** ou **features** pour une meilleure organisation et évolutivité.

```
• lib/
• |— features/
• |   |— authentication/
• |   |   |— models/
• |   |   |— views/
• |   |   |— controllers/
• |   |   |— services/
• |   |— gestion_taches/
• |   |   |— models/
• |   |   |— views/
• |   |   |— controllers/
• |   |   |— services/
• |— shared/
• |   |— widgets/
• |   |— utils/
• |— main.dart
```

Gestion des dépendances entre modules :

- Utilisez des **providers** ou des **injections de dépendances** pour gérer les interactions entre les différents modules.
- ```
final authServiceProvider = Provider<AuthService>((ref) =>
AuthService());
```
- ```
final tacheServiceProvider = Provider<TacheService>((ref) {
  final authService = ref.read(authServiceProvider);
  return TacheService(authService);
});
```

Bonnes pratiques :

- **Séparation claire des responsabilités** : Assurez-vous que chaque module gère une seule responsabilité pour faciliter la maintenance.
- **Nomination cohérente des fichiers et dossiers** : Utilisez des conventions de nommage claires et cohérentes pour faciliter la navigation dans le projet.
- **Réutilisation des widgets** : Créez des widgets réutilisables dans le dossier **widgets** pour éviter la duplication de code.
- **Documentation** : Documentez la structure de l'application et les responsabilités de chaque module pour faciliter la collaboration et la compréhension du code.

Exemple pratique :

Créer une application avec une page d'accueil et une page de détails structurées.

```
// main.dart
void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  final AppRouter _appRouter = AppRouter();

  @override
  Widget build(BuildContext context) {
    return ProviderScope(
      child: MaterialApp(
        title: 'Navigation Structurée',
        onGenerateRoute: _appRouter.onGenerateRoute,
        initialRoute: AppRouter.accueil,
      ),
    );
  }
}

// app_router.dart
class AppRouter {
  static const String accueil = '/';
  static const String detailTache = '/detailTache';

  Route<dynamic> onGenerateRoute(RouteSettings settings) {
    switch (settings.name) {
      case accueil:
        return MaterialPageRoute(builder: (context) => Accueil());
      case detailTache:
        final tache = settings.arguments as Tache;
        return MaterialPageRoute(builder: (context) =>
DetailTache(tache: tache));
      default:
        return MaterialPageRoute(builder: (context) => Page404());
    }
  }
}
```

```
// models/tache.dart
class Tache {
  final String titre;
  final String description;

  Tache({required this.titre, required this.description});
}

// views/accueil.dart
class Accueil extends StatelessWidget {
  final List<Tache> _taches = [
    Tache(titre: 'Tâche 1', description: 'Description 1'),
    Tache(titre: 'Tâche 2', description: 'Description 2'),
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Accueil')),
      body: ListView.builder(
        itemCount: _taches.length,
        itemBuilder: (context, index) {
          final tache = _taches[index];
          return ListTile(
            title: Text(tache.titre),
            onTap: () {
              Navigator.pushNamed(
                context,
                AppRouter.detailTache,
                arguments: tache,
              );
            },
          ),
        ),
      ),
    );
  }
}

// views/detail_tache.dart
class DetailTache extends StatelessWidget {
  final Tache tache;

  DetailTache({required this.tache});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Détails de la Tâche')),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          children: [
            Text(tache.titre, style: TextStyle(fontSize: 24,
fontWeight: FontWeight.bold)),
            SizedBox(height: 10),
          ],
        ),
      ),
    );
  }
}
```

```
        Text(tache.description),
      ],
    ),
  ),
);
}
}

// views/page_404.dart
class Page404 extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Page non trouvée')),
      body: Center(child: Text('Erreur 404 : Page non trouvée')),
    );
  }
}
```

Conclusion : Une bonne structuration et organisation de l'application sont essentielles pour une navigation fluide et une maintenance simplifiée. En adoptant une architecture claire, en organisant les fichiers de manière logique et en utilisant des outils de gestion de navigation efficaces comme **AppRouter**, vous pouvez développer des applications Flutter robustes, évolutives et faciles à gérer. La combinaison de bonnes pratiques de structuration avec des solutions de gestion d'état appropriées garantit une expérience utilisateur optimale et une base de code maintenable.

Leçon 7 : Optimisation des performances et gestion des ressources

Introduction à l'optimisation des performances : L'optimisation des performances est cruciale pour garantir une expérience utilisateur fluide et réactive. Une application Flutter bien optimisée consomme moins de ressources, démarre plus rapidement et répond mieux aux interactions utilisateur.

Principes de base :

- **Minimiser les reconstructions inutiles :** Évitez de reconstruire l'arborescence des widgets sans nécessité.
- **Optimiser les images et les assets :** Utilisez des formats adaptés et compressez les images pour réduire la charge.
- **Gérer efficacement l'état :** Utilisez des solutions de gestion d'état appropriées pour éviter les recalculs et les reconstructions excessives.

Techniques d'optimisation :

1. **Réduction des reconstructions de widgets :**
 - Utilisez des widgets **const** lorsque c'est possible pour éviter les reconstructions.
 - `const Text('Texte constant')`

- Utilisez **shouldRebuild** dans les widgets personnalisés pour contrôler les reconstructions.
- 2. **Optimisation des ListViews :**
 - Utilisez **ListView.builder** ou **ListView.separated** pour des listes dynamiques et performantes.
 - ```
ListView.builder(
 itemCount: 1000,
 itemBuilder: (context, index) {
 return ListTile(title: Text('Item $index'));
 },
)
```
- 3. **Gestion efficace des images :**
  - Utilisez des images **lazy-loaded** avec des packages comme **cached\_network\_image**.
  - ```
CachedNetworkImage(  
  imageUrl: 'https://example.com/image.jpg',  
  placeholder: (context, url) =>  
    CircularProgressIndicator(),  
  errorWidget: (context, url, error) => Icon(Icons.error),  
)
```
 - Compressez et redimensionnez les images pour réduire la taille des fichiers.
- 4. **Utilisation de const :**
 - Déclarez les widgets **const** pour améliorer les performances en permettant à Flutter de les réutiliser sans les reconstruire.
 - ```
const Icon(Icons.star)
```
- 5. **Optimisation de la gestion d'état :**
  - Utilisez des solutions de gestion d'état efficaces comme **Provider**, **Riverpod**, ou **BLoC** pour minimiser les reconstructions.
  - Gardez l'état local quand c'est possible pour réduire la portée des reconstructions.
- 6. **Lazy Loading et Pagination :**
  - Implémentez le **lazy loading** pour charger les données au fur et à mesure que l'utilisateur fait défiler la liste.
  - ```
ListView.builder(  
  controller: _scrollController,  
  itemCount: _items.length,  
  itemBuilder: (context, index) {  
    if (index == _items.length - 1) {  
      // Charger plus de données  
    }  
    return ListTile(title: Text(_items[index]));  
  },  
)
```
- 7. **Utilisation de RepaintBoundary :**
 - Encapsulez les parties de l'interface utilisateur qui changent souvent dans des **RepaintBoundary** pour limiter les zones de redessin.
 - ```
RepaintBoundary(
 child: Container(
 color: Colors.blue,
 child: Text('Contenu isolé'),
),
)
```

- ),
- )

### 8. Profilage et Debugging :

- Utilisez **Flutter DevTools** pour profiler les performances de l'application, identifier les goulots d'étranglement, et optimiser le rendu.
  - **Timeline** : Analysez les frames et identifiez les problèmes de performance.
  - **Widget Inspector** : Inspectez la hiérarchie des widgets et leur état.
  - **Memory Profiler** : Surveillez l'utilisation de la mémoire pour éviter les fuites.

## Gestion des ressources :

### 1. Libération des ressources :

- Utilisez la méthode **dispose** pour nettoyer les ressources comme les contrôleurs de texte, les animations, et les stream subscriptions.
- @override
- void dispose() {
- \_controller.dispose();
- super.dispose();
- }

### 2. Optimisation du build :

- Minimisez les opérations lourdes dans la méthode **build**. Effectuez les calculs intensifs en dehors du build ou en utilisant des **compute** pour les tâches en arrière-plan.
- Future<void> \_chargerDonnees() async {
- final result = await compute(traiterDonnees, data);
- setState(() {
- \_resultat = result;
- });
- }

### 3. Gestion des animations :

- Utilisez des animations **légères** et optimisez les frames pour éviter les ralentissements.
- Évitez les animations complexes qui nécessitent beaucoup de ressources de rendu.

### 4. Utilisation de Const Constructors :

- Déclarez les constructeurs **const** pour les widgets immuables afin de réduire les reconstructions.
- const Padding(
- padding: EdgeInsets.all(8.0),
- child: Text('Texte constant'),
- )

### 5. Minimisation des dépendances :

- Évitez d'ajouter des packages inutiles qui augmentent la taille de l'application et consomment des ressources.
- Analysez régulièrement les dépendances pour supprimer celles qui ne sont plus nécessaires.

### Optimisation du rendu :

#### 1. Utilisation des layers :

- Comprenez et optimisez l'utilisation des **layers** pour améliorer le rendu graphique.
- Évitez les widgets qui génèrent trop de layers, comme les widgets trop imbriqués.

#### 2. Caching :

- Utilisez le **caching** pour les opérations coûteuses et les images pour améliorer les performances.
- `final imageCache = PaintingBinding.instance.imageCache;`
- `imageCache.maximumSize = 100;`
- `imageCache.maximumSizeBytes = 100 << 20;`

### Optimisation des performances réseau :

#### 1. Minimiser les requêtes réseau :

- Cachez les réponses des API pour éviter les requêtes redondantes.
- Utilisez des stratégies de **caching** efficaces avec des packages comme **dio\_cache\_interceptor**.

#### 2. Compression des données :

- Compressez les données échangées avec les serveurs pour réduire le temps de chargement.
- Utilisez des formats de données légers comme **JSON** ou **Protocol Buffers**.

### Optimisation des builds :

#### 1. Tree Shaking :

- Flutter élimine le code inutilisé lors de la compilation, réduisant la taille de l'application finale.
- Assurez-vous d'utiliser des **imports** spécifiques pour tirer parti du tree shaking.
- `// Mauvais`
- `import 'package:flutter/material.dart';`
- 
- `// Bon`
- `import 'package:flutter/material.dart' show Text, Icon;`

#### 2. Minification et Obfuscation :

- Utilisez la minification et l'obfuscation pour réduire la taille de l'application et protéger le code source.
- `flutter build apk --release --obfuscate --split-debug-info=/<répertoire>`

### Exemple pratique :

Optimiser une **ListView** avec des images lourdes en utilisant **cached\_network\_image** et **const widgets**.

```
class ListeOptimisee extends StatelessWidget {
 final List<String> _imageUrls = [
 'https://example.com/image1.jpg',
 'https://example.com/image2.jpg',
 // Plus d'URLs
];

 @override
 Widget build(BuildContext context) {
 return ListView.builder(
 itemCount: _imageUrls.length,
 itemBuilder: (context, index) {
 return ListTile(
 leading: CachedNetworkImage(
 imageUrl: _imageUrls[index],
 placeholder: (context, url) => const
CircularProgressIndicator(),
 errorWidget: (context, url, error) => const
Icon(Icons.error),
),
 title: const Text('Titre de l\'image'),
 subtitle: const Text('Description de l\'image'),
);
 },
);
 }
}
```

**Conclusion :** L'optimisation des performances et la gestion efficace des ressources sont essentielles pour développer des applications Flutter performantes et réactives. En appliquant les techniques d'optimisation décrites dans cette leçon, vous pouvez améliorer significativement la fluidité, la réactivité et l'efficacité de votre application, tout en garantissant une utilisation optimale des ressources matérielles et réseau. Une application bien optimisée non seulement offre une meilleure expérience utilisateur, mais est également plus facile à maintenir et à évoluer.

## Leçon 7 : Tests et assurance qualité dans Flutter

**Introduction aux tests :** Les tests sont essentiels pour garantir la **qualité**, la **fiabilité** et la **maintenabilité** des applications Flutter. Flutter supporte différents types de tests : **tests unitaires**, **tests widgets**, et **tests d'intégration**.

**Types de tests :**

### 1. Tests unitaires :

- Testent des unités de code isolées, comme des fonctions ou des classes.
- Utilisés pour vérifier la logique métier et les calculs.
- Exemple :
- ```
void main() {
```
- ```
 test('Addition devrait retourner la somme de deux
```
- ```
    nombres', () {
```
- ```
 final result = addition(2, 3);
```

```
o expect(result, 5);
o });
o }
o
o int addition(int a, int b) => a + b;
```

### 2. Tests widgets :

- o Testent les widgets individuels et leur interaction.
- o Utilisés pour vérifier l'interface utilisateur et le comportement des widgets.
- o Exemple :

```
o void main() {
o testWidgets('Bouton incrémente le compteur',
o (WidgetTester tester) async {
o await tester.pumpWidget(
o ProviderScope(
o child: MaterialApp(home: CompteurPage()),
o),
o);
o
o expect(find.text('Compteur : 0'), findsOneWidget);
o
o await tester.tap(find.byType(ElevatedButton));
o await tester.pump();
o
o expect(find.text('Compteur : 1'), findsOneWidget);
o });
o }
```

### 3. Tests d'intégration :

- o Testent l'application entière ou de grandes parties de celle-ci.
- o Utilisés pour vérifier les interactions entre les différents modules et la navigation.
- o Exemple :

```
o void main() {
o testWidgets('Naviguer de la page d\'accueil à la page de
o détails', (WidgetTester tester) async {
o await tester.pumpWidget(
o ProviderScope(
o child: MaterialApp(
o initialRoute: '/',
o routes: {
o '/': (context) => Accueil(),
o '/detail': (context) => DetailPage(),
o },
o),
o),
o);
o
o expect(find.text('Accueil'), findsOneWidget);
o
o await tester.tap(find.text('Aller à détail'));
o await tester.pumpAndSettle();
o
o expect(find.text('Page de Détail'), findsOneWidget);
o });
o }
```

### Configuration des tests :

#### 1. Ajouter les dépendances de test :

- Ajoutez les packages `flutter_test` et `mockito` pour les tests unitaires et les mocks.
- `dev_dependencies:`
- `flutter_test:`
- `sdk: flutter`
- `mockito: ^5.0.0`

#### 2. Structure des fichiers de test :

- Placez les tests dans le répertoire `test/`, en imitant la structure des fichiers de l'application.
- `test/`
- `├─ unit/`
- `│ └─ addition_test.dart`
- `├─ widgets/`
- `│ └─ compteur_test.dart`
- `└─ integration/`
- `└─ navigation_test.dart`

### Tests unitaires avancés :

#### 1. Tester les services :

- Utilisez **mockito** pour créer des mocks des services et tester leur interaction.
- ```
class MockApiService extends Mock implements ApiService {}
```
- ```
void main() {
```
- ```
  group('CompteurBloc', () {
```
- ```
 late CompteurBloc bloc;
```
- ```
    late MockApiService mockApiService;
```
- ```
 setUp(() {
```
- ```
      mockApiService = MockApiService();
```
- ```
 bloc = CompteurBloc(apiService: mockApiService);
```
- ```
    });
```
- ```
 test('Initial state est CompteurInitial', () {
```
- ```
      expect(bloc.state, CompteurInitial());
```
- ```
 });
```
- ```
    test('Ajouter un compteur émet CompteurModifie', () {
```
- ```
 bloc.add(Incrementer());
```
- ```
      expectLater(
```
- ```
 bloc.stream,
```
- ```
        emitsInOrder([CompteurModifie(1)]),
```
- ```
);
```
- ```
    });
```
- ```
 });
```
- ```
}
```

2. Tester les modèles :

- Assurez-vous que les modèles fonctionnent comme prévu.
- ```
void main() {
```

```
o test('Modèle Tache devrait stocker les données
correctement', () {
o final tache = Tache(titre: 'Test', description:
'Description test');
o expect(tache.titre, 'Test');
o expect(tache.description, 'Description test');
o });
o }
```

### Tests widgets avancés :

#### 1. Tester l'interaction utilisateur :

```
o Simulez les interactions et vérifiez les changements dans l'interface.
o void main() {
o testWidgets('Écrire dans le TextField et vérifier la mise
à jour', (WidgetTester tester) async {
o await tester.pumpWidget(
o ProviderScope(
o child: MaterialApp(home: FormulairePage()),
o),
o);
o
o await tester.enterText(find.byType(TextField),
'Nouvelle Tâche');
o expect(find.text('Nouvelle Tâche'), findsOneWidget);
o });
o }
```

#### 2. Tester les animations :

```
o Vérifiez que les animations se déroulent comme prévu.
o void main() {
o testWidgets('Animation de fade fonctionne', (WidgetTester
tester) async {
o await tester.pumpWidget(
o MaterialApp(home: FadeWidget()),
o);
o
o expect(find.byType(FadeTransition), findsOneWidget);
o
o // Simuler l'animation
o await tester.pump(Duration(milliseconds: 500));
o
o // Vérifier l'état après l'animation
o expect(find.text('Fade In'), findsOneWidget);
o });
o }
```

### Tests d'intégration avancés :

#### 1. Utiliser Flutter Driver :

```
o Automatiser les tests d'intégration avec Flutter Driver pour simuler des
interactions utilisateur sur l'application entière.
```

#### 2. Tester la navigation complète :

- Vérifiez que la navigation entre plusieurs pages fonctionne comme prévu dans des scénarios réels.
- ```
void main() {  
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();  
    
  testWidgets('Navigation de Accueil à Détails',  
    (WidgetTester tester) async {  
    await tester.pumpWidget(  
      ProviderScope(  
        child: MaterialApp(  
          initialRoute: '/',  
          routes: {  
            '/': (context) => Accueil(),  
            '/details': (context) => DetailPage(),  
          },  
        ),  
      ),  
    );  
      
    // Vérifier la présence de la page d'accueil  
    expect(find.text('Accueil'), findsOneWidget);  
      
    // Cliquer sur un bouton pour naviguer  
    await tester.tap(find.text('Aller à Détails'));  
    await tester.pumpAndSettle();  
      
    // Vérifier la présence de la page de détails  
    expect(find.text('Détails'), findsOneWidget);  
  });  
}
```

Bonnes pratiques de tests :

- **Isoler les tests** : Chaque test doit être indépendant des autres pour éviter les interférences.
- **Couvrir tous les cas d'utilisation** : Assurez-vous que les tests couvrent les différents scénarios et cas limites.
- **Automatiser les tests** : Intégrez les tests dans un pipeline CI/CD pour garantir une qualité continue.

Outils complémentaires :

- **Mockito** : Pour créer des mocks et des stubs dans les tests unitaires.
- **Flutter Test** : Fournit des utilitaires pour tester les widgets et les interactions.
- **Integration Test** : Pour automatiser les tests d'intégration et simuler des interactions réelles.

Exemple avancé :

Créer un test widget pour vérifier l'interaction d'un formulaire avec validation.

```
// Widget du formulaire
```

Développement d'Applications Flutter

```
class FormulairePage extends StatefulWidget {
  @override
  _FormulairePageState createState() => _FormulairePageState();
}

class _FormulairePageState extends State<FormulairePage> {
  final _formKey = GlobalKey<FormState>();
  String _nom = '';

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Formulaire')),
      body: Form(
        key: _formKey,
        child: Column(
          children: [
            TextFormField(
              decoration: InputDecoration(labelText: 'Nom'),
              validator: (value) {
                if (value == null || value.isEmpty) {
                  return 'Veuillez entrer un nom';
                }
                return null;
              },
              onSave: (value) {
                _nom = value!;
              },
            ),
            ElevatedButton(
              onPressed: () {
                if (_formKey.currentState!.validate()) {
                  _formKey.currentState!.save();
                  // Action après la validation
                }
              },
              child: Text('Soumettre'),
            ),
          ],
        ),
      ),
    );
  }
}

// Test du formulaire
void main() {
  testWidgets('Validation du formulaire fonctionne', (WidgetTester
  tester) async {
    await tester.pumpWidget(
      ProviderScope(
        child: MaterialApp(home: FormulairePage()),
      ),
    );

    // Taper du texte dans le champ
    await tester.enterText(find.byType(TextFormField), 'John Doe');
```

```
await tester.tap(find.text('Soumettre'));
await tester.pump();

// Vérifier qu'aucune erreur n'est affichée
expect(find.text('Veuillez entrer un nom'), findsNothing);
});

testWidgets('Validation du formulaire échoue avec champ vide',
(WidgetTester tester) async {
  await tester.pumpWidget(
    ProviderScope(
      child: MaterialApp(home: FormulairePage()),
    ),
  );

  // Taper du texte vide et soumettre
  await tester.enterText(find.byType(TextFormField), '');
  await tester.tap(find.text('Soumettre'));
  await tester.pump();

  // Vérifier qu'une erreur est affichée
  expect(find.text('Veuillez entrer un nom'), findsOneWidget);
});
}
```

Conclusion : Les tests sont indispensables pour assurer la qualité et la fiabilité des applications Flutter. En maîtrisant les différents types de tests et en suivant les bonnes pratiques, vous pouvez identifier et corriger les bugs plus tôt, améliorer la maintenabilité du code, et garantir une expérience utilisateur optimale. L'intégration des tests dans votre flux de développement et l'utilisation d'outils adaptés renforcent la robustesse et la stabilité de vos applications.

Module 4 : Interactions avec les APIs et les Bases de Données

Leçon 1 : Introduction aux appels API REST et HTTP en Flutter

Les **APIs REST** sont essentielles pour communiquer avec des services externes et récupérer des données dynamiques dans une application Flutter. Cette leçon couvre les bases des appels API en utilisant le package **http**.

Concepts clés :

- **API REST** : Architecture permettant la communication entre clients et serveurs via des requêtes HTTP.
- **Endpoints** : URL spécifiques où les ressources peuvent être accessibles.
- **Verbes HTTP** : **GET**, **POST**, **PUT**, **DELETE**, etc., utilisés pour effectuer différentes opérations sur les ressources.

Installation du package `http` :

1. **Ajouter la dépendance** dans le fichier `pubspec.yaml` :
2. `dependencies:`
3. `http: ^0.13.4`
4. **Installer les packages** avec la commande `flutter pub get`.

Effectuer un appel GET :

```
import 'package:http/http.dart' as http;
import 'dart:convert';

Future<void> fetchData() async {
  final response = await
http.get(Uri.parse('https://api.example.com/data'));

  if (response.statusCode == 200) {
    final data = json.decode(response.body);
    print('Données récupérées : $data');
  } else {
    throw Exception('Erreur lors du chargement des données');
  }
}
```

Effectuer un appel POST :

```
Future<void> sendData(Map<String, dynamic> payload) async {
  final response = await http.post(
    Uri.parse('https://api.example.com/data'),
    headers: {'Content-Type': 'application/json'},
    body: json.encode(payload),
  );

  if (response.statusCode == 201) {
    print('Données envoyées avec succès');
  } else {
    throw Exception('Erreur lors de l\'envoi des données');
  }
}
```

Gestion des erreurs :

- **Codes de statut HTTP** : Vérifiez les codes de statut pour gérer les réponses positives et les erreurs.
- **Exceptions** : Utilisez des blocs `try-catch` pour capturer et gérer les exceptions.

Exemple avec gestion des erreurs :

```
Future<void> fetchDataWithErrorHandling() async {
  try {
    final response = await
http.get(Uri.parse('https://api.example.com/data'));
  }
```

```
if (response.statusCode == 200) {  
  final data = json.decode(response.body);  
  print('Données récupérées : $data');  
} else {  
  print('Erreur : ${response.statusCode}');  
}  
} catch (e) {  
  print('Exception capturée : $e');  
}  
}
```

Bonnes pratiques :

- **Sécurité** : Utilisez HTTPS pour sécuriser les communications.
- **Optimisation** : Implémentez le **caching** des réponses pour améliorer les performances.
- **Pagination** : Gérez la pagination pour les grandes quantités de données.

Leçon 2 : Traitement des réponses JSON et gestion des erreurs

Le traitement des réponses **JSON** est une compétence essentielle pour manipuler les données récupérées via les APIs. Cette leçon couvre la sérialisation et la désérialisation des données JSON, ainsi que la gestion des erreurs.

Sérialisation et Désérialisation :

- **Désérialisation** : Convertir les données JSON en objets Dart.
- **Sérialisation** : Convertir les objets Dart en données JSON.

Exemple de modèle Dart :

```
class Utilisateur {  
  final int id;  
  final String nom;  
  final String email;  
  
  Utilisateur({required this.id, required this.nom, required  
    this.email});  
  
  factory Utilisateur.fromJson(Map<String, dynamic> json) {  
    return Utilisateur(  
      id: json['id'],  
      nom: json['nom'],  
      email: json['email'],  
    );  
  }  
  
  Map<String, dynamic> toJson() {  
    return {  
      'id': id,  
      'nom': nom,  
    };  
  }  
}
```

```
        'email': email,  
      };  
    }  
  }
```

Désérialiser une réponse JSON :

```
Future<List<Utilisateur>> fetchUtilisateurs() async {  
  final response = await  
  http.get(Uri.parse('https://api.example.com/utilisateurs'));  
  
  if (response.statusCode == 200) {  
    final List<dynamic> jsonData = json.decode(response.body);  
    return jsonData.map((json) => Utilisateur.fromJson(json)).toList();  
  } else {  
    throw Exception('Erreur lors du chargement des utilisateurs');  
  }  
}
```

Sérialiser un objet Dart :

```
Future<void> ajouterUtilisateur(Utilisateur utilisateur) async {  
  final response = await http.post(  
    Uri.parse('https://api.example.com/utilisateurs'),  
    headers: {'Content-Type': 'application/json'},  
    body: json.encode(utilisateur.toJson()),  
  );  
  
  if (response.statusCode == 201) {  
    print('Utilisateur ajouté avec succès');  
  } else {  
    throw Exception('Erreur lors de l\'ajout de l\'utilisateur');  
  }  
}
```

Gestion des erreurs avancée :

- **Validation des données** : Assurez-vous que les données JSON contiennent les champs nécessaires.
- **Gestion des exceptions spécifiques** : Gérez les types d'exceptions spécifiques pour une meilleure robustesse.

Exemple avec validation des données :

```
factory Utilisateur.fromJson(Map<String, dynamic> json) {  
  if (json['id'] == null || json['nom'] == null || json['email'] ==  
  null) {  
    throw Exception('Données JSON invalides');  
  }  
  return Utilisateur(  
    id: json['id'],  
    nom: json['nom'],  
    email: json['email'],  
  );  
}
```

```
);  
}
```

Bonnes pratiques :

- **Validation des réponses** : Toujours valider les données reçues avant de les utiliser.
- **Gestion centralisée des erreurs** : Implémentez une gestion des erreurs centralisée pour uniformiser les réponses aux erreurs.

Leçon 3 : Utilisation de Firebase comme backend

Introduction à Firebase : Firebase est une plateforme de développement d'applications mobiles et web proposée par Google, offrant une gamme de services backend comme l'authentification, la base de données en temps réel, le stockage de fichiers, et bien plus.

Configuration de Firebase dans Flutter :

1. **Créer un projet Firebase** :
 - Rendez-vous sur [Firebase Console](#).
 - Créez un nouveau projet ou utilisez un projet existant.
2. **Ajouter une application Flutter à Firebase** :
 - Sélectionnez l'icône Android/iOS pour ajouter une application.
 - Suivez les instructions pour télécharger le fichier `google-services.json` (Android) ou `GoogleService-Info.plist` (iOS) et l'ajouter à votre projet Flutter.
3. **Ajouter les dépendances Firebase dans `pubspec.yaml`** :
4. `dependencies:`
5. `firebase_core: ^2.0.0`
6. `firebase_auth: ^4.0.0`
7. `cloud_firestore: ^4.0.0`
8. `firebase_storage: ^11.0.0`
9. **Initialiser Firebase dans Flutter** :
10. `import 'package:firebase_core/firebase_core.dart';`
11. `import 'package:flutter/material.dart';`
12. `import 'firebase_options.dart'; // Généré par Firebase CLI`
- 13.
14. `void main() async {`
15. `WidgetsFlutterBinding.ensureInitialized();`
16. `await Firebase.initializeApp(`
17. `options: DefaultFirebaseOptions.currentPlatform,`
18. `);`
19. `runApp(MyApp());`
20. `}`
- 21.
22. `class MyApp extends StatelessWidget {`
23. `@override`
24. `Widget build(BuildContext context) {`
25. `return MaterialApp(`
26. `title: 'Flutter Firebase',`
27. `home: Accueil(),`

```
28.    );  
29.  }  
30. }
```

Authentification avec Firebase :

```
import 'package:firebase_auth/firebase_auth.dart';  
import 'package:flutter/material.dart';  
  
class AuthService {  
  final FirebaseAuth _auth = FirebaseAuth.instance;  
  
  // Inscription  
  Future<User?> inscrire(String email, String motDePasse) async {  
    try {  
      UserCredential result = await  
_auth.createUserWithEmailAndPassword(  
        email: email,  
        password: motDePasse,  
      );  
      return result.user;  
    } catch (e) {  
      print(e.toString());  
      return null;  
    }  
  }  
  
  // Connexion  
  Future<User?> connecter(String email, String motDePasse) async {  
    try {  
      UserCredential result = await _auth.signInWithEmailAndPassword(  
        email: email,  
        password: motDePasse,  
      );  
      return result.user;  
    } catch (e) {  
      print(e.toString());  
      return null;  
    }  
  }  
  
  // Déconnexion  
  Future<void> deconnecter() async {  
    await _auth.signOut();  
  }  
}
```

Utilisation de Cloud Firestore :

```
import 'package:cloud_firestore/cloud_firestore.dart';  
import 'package:flutter/material.dart';  
  
class FirestoreService {  
  final FirebaseFirestore _db = FirebaseFirestore.instance;
```

Développement d'Applications Flutter

```
// Ajouter un document
Future<void> ajouterUtilisateur(Map<String, dynamic> data) async {
  await _db.collection('utilisateurs').add(data);
}

// Lire des documents
Stream<List<Utilisateur>> getUtilisateurs() {
  return _db.collection('utilisateurs').snapshots().map((snapshot) =>
    snapshot.docs.map((doc) =>
      Utilisateur.fromFirestore(doc)).toList());
}

// Mettre à jour un document
Future<void> mettreAJourUtilisateur(String id, Map<String, dynamic>
data) async {
  await _db.collection('utilisateurs').doc(id).update(data);
}

// Supprimer un document
Future<void> supprimerUtilisateur(String id) async {
  await _db.collection('utilisateurs').doc(id).delete();
}
}

class Utilisateur {
  final String id;
  final String nom;
  final String email;

  Utilisateur({required this.id, required this.nom, required
this.email});

  factory Utilisateur.fromFirestore(DocumentSnapshot doc) {
    Map data = doc.data() as Map<String, dynamic>;
    return Utilisateur(
      id: doc.id,
      nom: data['nom'] ?? '',
      email: data['email'] ?? '',
    );
  }

  Map<String, dynamic> toMap() {
    return {
      'nom': nom,
      'email': email,
    };
  }
}
```

Utilisation de Firebase Storage :

```
import 'package:firebase_storage/firebase_storage.dart';
import 'package:image_picker/image_picker.dart';
import 'dart:io';

class StorageService {
```

```
final FirebaseStorage _storage = FirebaseStorage.instance;

// Télécharger un fichier
Future<String?> telechargerImage(File fichier, String chemin) async {
  try {
    Reference ref = _storage.ref().child(chemin);
    UploadTask uploadTask = ref.putFile(fichier);
    TaskSnapshot snapshot = await uploadTask;
    return await snapshot.ref.getDownloadURL();
  } catch (e) {
    print(e.toString());
    return null;
  }
}

// Supprimer un fichier
Future<void> supprimerImage(String chemin) async {
  try {
    Reference ref = _storage.ref().child(chemin);
    await ref.delete();
  } catch (e) {
    print(e.toString());
  }
}
```

Bonnes pratiques :

- **Sécurité** : Configurez les règles de sécurité Firebase pour restreindre l'accès aux données sensibles.
- **Optimisation des requêtes** : Limitez les champs retournés par les requêtes pour améliorer les performances.
- **Gestion des erreurs** : Implémentez une gestion robuste des erreurs pour une meilleure expérience utilisateur.

Leçon 4 : Connexion à une base de données locale avec SQLite

Introduction à SQLite : SQLite est une base de données relationnelle légère et intégrée, idéale pour le stockage local des données dans les applications Flutter.

Installation du package `sqflite` :

1. **Ajouter les dépendances** dans le fichier `pubspec.yaml` :
2. `dependencies:`
3. `sqflite: ^2.0.0`
4. `path: ^1.8.0`
5. **Installer les packages** avec la commande `flutter pub get`.

Configuration de SQLite :

```
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';
```

```
class DatabaseHelper {
  static final DatabaseHelper _instance = DatabaseHelper._internal();
  factory DatabaseHelper() => _instance;
  DatabaseHelper._internal();

  static Database? _database;

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('app.db');
    return _database!;
  }

  Future<Database> _initDB(String filePath) async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, filePath);

    return await openDatabase(
      path,
      version: 1,
      onCreate: _createDB,
    );
  }

  Future<void> _createDB(Database db, int version) async {
    await db.execute('''
      CREATE TABLE utilisateurs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nom TEXT NOT NULL,
        email TEXT NOT NULL
      )
    ''');
  }
}
```

Opérations CRUD avec SQLite :

```
class Utilisateur {
  final int? id;
  final String nom;
  final String email;

  Utilisateur({this.id, required this.nom, required this.email});

  factory Utilisateur.fromMap(Map<String, dynamic> json) =>
    Utilisateur(
      id: json['id'],
      nom: json['nom'],
      email: json['email'],
    );

  Map<String, dynamic> toMap() => {
    'id': id,
    'nom': nom,
    'email': email,
  }
}
```

```
    };  
  }  
  
  class UtilisateurDao {  
    final DatabaseHelper _dbHelper = DatabaseHelper();  
  
    Future<int> ajouterUtilisateur(Utilisateur utilisateur) async {  
      final db = await _dbHelper.database;  
      return await db.insert('utilisateurs', utilisateur.toMap());  
    }  
  
    Future<List<Utilisateur>> getUtilisateurs() async {  
      final db = await _dbHelper.database;  
      final List<Map<String, dynamic>> maps = await  
db.query('utilisateurs');  
  
      return List.generate(maps.length, (i) {  
        return Utilisateur.fromMap(maps[i]);  
      });  
    }  
  
    Future<int> mettreAJourUtilisateur(Utilisateur utilisateur) async {  
      final db = await _dbHelper.database;  
      return await db.update(  
        'utilisateurs',  
        utilisateur.toMap(),  
        where: 'id = ?',  
        whereArgs: [utilisateur.id],  
      );  
    }  
  
    Future<int> supprimerUtilisateur(int id) async {  
      final db = await _dbHelper.database;  
      return await db.delete(  
        'utilisateurs',  
        where: 'id = ?',  
        whereArgs: [id],  
      );  
    }  
  }  
}
```

Utilisation dans Flutter :

```
import 'package:flutter/material.dart';  
  
class UtilisateurPage extends StatefulWidget {  
  @override  
  _UtilisateurPageState createState() => _UtilisateurPageState();  
}  
  
class _UtilisateurPageState extends State<UtilisateurPage> {  
  final UtilisateurDao _dao = UtilisateurDao();  
  List<Utilisateur> _utilisateurs = [];  
  
  @override  
  void initState() {
```

```
super.initState();
_chargerUtilisateurs();
}

Future<void> _chargerUtilisateurs() async {
  final utilisateurs = await _dao.getUtilisateurs();
  setState(() {
    _utilisateurs = utilisateurs;
  });
}

Future<void> _ajouterUtilisateur() async {
  final utilisateur = Utilisateur(nom: 'Nouveau Utilisateur', email:
'email@example.com');
  await _dao.ajouterUtilisateur(utilisateur);
  _chargerUtilisateurs();
}

Future<void> _supprimerUtilisateur(int id) async {
  await _dao.supprimerUtilisateur(id);
  _chargerUtilisateurs();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('Utilisateurs SQLite'),
    ),
    body: ListView.builder(
      itemCount: _utilisateurs.length,
      itemBuilder: (context, index) {
        final utilisateur = _utilisateurs[index];
        return ListTile(
          title: Text(utilisateur.nom),
          subtitle: Text(utilisateur.email),
          trailing: IconButton(
            icon: Icon(Icons.delete),
            onPressed: () => _supprimerUtilisateur(utilisateur.id!),
          ),
        );
      },
    ),
    floatingActionButton: FloatingActionButton(
      onPressed: _ajouterUtilisateur,
      child: Icon(Icons.add),
    ),
  );
}
```

Bonnes pratiques :

- **Gestion des transactions** : Utilisez des transactions pour garantir l'intégrité des données lors d'opérations multiples.

- **Optimisation des requêtes** : Indexez les colonnes fréquemment utilisées pour améliorer les performances des requêtes.
- **Sécurité des données** : Chiffrez les données sensibles stockées localement.

Leçon 5 : Stockage des données locales avec Hive et SharedPreferences

Introduction à Hive : Hive est une base de données NoSQL rapide et légère pour Flutter, idéale pour le stockage local de données structurées.

Installation du package hive et hive_flutter :

1. **Ajouter les dépendances** dans le fichier `pubspec.yaml` :
2. `dependencies:`
3. `hive: ^2.2.3`
4. `hive_flutter: ^1.1.0`
5. **Installer les packages** avec la commande `flutter pub get`.

Configuration de Hive :

```
import 'package:hive/hive.dart';
import 'package:hive_flutter/hive_flutter.dart';

class Utilisateur {
  final String nom;
  final String email;

  Utilisateur({required this.nom, required this.email});

  factory Utilisateur.fromMap(Map<String, dynamic> map) {
    return Utilisateur(
      nom: map['nom'],
      email: map['email'],
    );
  }

  Map<String, dynamic> toMap() {
    return {
      'nom': nom,
      'email': email,
    };
  }
}

Future<void> main() async {
  await Hive.initFlutter();
  await Hive.openBox('utilisateurs');
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
```

```
        title: 'Hive Stockage Local',  
        home: UtilisateurHivePage(),  
      );  
    }  
  }  
}
```

Utilisation de Hive :

```
class UtilisateurHivePage extends StatefulWidget {  
  @override  
  _UtilisateurHivePageState createState() =>  
    _UtilisateurHivePageState();  
}  
  
class _UtilisateurHivePageState extends State<UtilisateurHivePage> {  
  final Box _box = Hive.box('utilisateurs');  
  
  Future<void> _ajouterUtilisateur() async {  
    final utilisateur = Utilisateur(nom: 'Utilisateur Hive', email:  
'hive@example.com');  
    await _box.add(utilisateur.toMap());  
    setState(() {});  
  }  
  
  Future<void> _supprimerUtilisateur(int index) async {  
    await _box.deleteAt(index);  
    setState(() {});  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    final utilisateurs = _box.values.toList();  
  
    return Scaffold(  
      appBar: AppBar(  
        title: Text('Utilisateurs Hive'),  
      ),  
      body: ListView.builder(  
        itemCount: utilisateurs.length,  
        itemBuilder: (context, index) {  
          final utilisateur = Utilisateur.fromMap(utilisateurs[index]);  
          return ListTile(  
            title: Text(utilisateur.nom),  
            subtitle: Text(utilisateur.email),  
            trailing: IconButton(  
              icon: Icon(Icons.delete),  
              onPressed: () => _supprimerUtilisateur(index),  
            ),  
          );  
        },  
      ),  
      floatingActionButton: FloatingActionButton(  
        onPressed: _ajouterUtilisateur,  
        child: Icon(Icons.add),  
      ),  
    );  
  }  
}
```

```
}  
}
```

Introduction à SharedPreferences : **SharedPreferences** est un moyen simple de stocker des données persistantes sous forme de paires clé-valeur, idéal pour les préférences utilisateur et les petites quantités de données.

Installation du package `shared_preferences` :

1. **Ajouter la dépendance** dans le fichier `pubspec.yaml` :
2. `dependencies:`
3. `shared_preferences: ^2.0.15`
4. **Installer les packages** avec la commande `flutter pub get`.

Utilisation de SharedPreferences :

```
import 'package:flutter/material.dart';  
import 'package:shared_preferences/shared_preferences.dart';  
  
class SharedPreferencesPage extends StatefulWidget {  
  @override  
  _SharedPreferencesPageState createState() =>  
  _SharedPreferencesPageState();  
}  
  
class _SharedPreferencesPageState extends State<SharedPreferencesPage>  
{  
  String _nom = '';  
  
  @override  
  void initState() {  
    super.initState();  
    _chargerNom();  
  }  
  
  Future<void> _chargerNom() async {  
    final prefs = await SharedPreferences.getInstance();  
    setState(() {  
      _nom = prefs.getString('nom') ?? 'Aucun';  
    });  
  }  
  
  Future<void> _sauvegarderNom(String nom) async {  
    final prefs = await SharedPreferences.getInstance();  
    await prefs.setString('nom', nom);  
    _chargerNom();  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    final TextEditingController _controller = TextEditingController();  
  
    return Scaffold(  
      appBar: AppBar(  

```

```
        title: Text('SharedPreferences'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          children: [
            Text(
              'Nom sauvegardé : $_nom',
              style: TextStyle(fontSize: 20),
            ),
            TextField(
              controller: _controller,
              decoration: InputDecoration(labelText: 'Entrez votre
nom'),
            ),
            ElevatedButton(
              onPressed: () {
                if (_controller.text.isNotEmpty) {
                  _sauvegarderNom(_controller.text);
                }
              },
              child: Text('Sauvegarder'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Bonnes pratiques :

- **Sélectionner la solution adaptée** : Utilisez **Hive** pour des données structurées complexes et **SharedPreferences** pour les préférences simples.
- **Gestion des types de données** : Assurez-vous de gérer correctement les types de données lors de la sérialisation et de la désérialisation.
- **Sécurité** : Chiffrez les données sensibles avant de les stocker localement.

Leçon 6 : Sécurisation des données et gestion des permissions

Introduction à la sécurisation des données : La sécurité des données est cruciale pour protéger les informations sensibles des utilisateurs. Cette leçon couvre les meilleures pratiques pour sécuriser les données stockées localement et gérer les permissions nécessaires.

Sécurisation des données locales :

1. **Chiffrement des données avec Hive** :
 - **Hive** supporte le chiffrement des boîtes pour protéger les données sensibles. ``dart import 'package:hive/hive.dart'; import

```
'package:hive_flutter/hive_flutter.dart'; import 'dart:convert'; import
'dart:typed_data';

class DatabaseHelper { static final DatabaseHelper _instance =
DatabaseHelper._internal(); factory DatabaseHelper() => _instance;
DatabaseHelper._internal();

static Database? _database;

Future get database async {

  if (_database != null) return _database!;
  _database = await _initDB('app_secure.db');
  return _database!;

}

Future _initDB(String filePath) async {

  final dbPath = await getDatabasesPath();
  final path = join(dbPath, filePath);

  // Générer une clé de chiffrement
  final key = utf8.encode('votre_clé_secrète_32_bytes'); // Doit
  être de 32 bytes
  final encryptedKey = base64Url.encode(key);

  return await openDatabase(
    path,
    version: 1,
    onCreate: (db, version) async {
      await db.execute('''
        CREATE TABLE utilisateurs (
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          nom TEXT NOT NULL,
          email TEXT NOT NULL
        )
      ''');
    },
    // Appliquer le chiffrement
    password: encryptedKey,
  );

} }
```

2. Utiliser le package `flutter_secure_storage` :

- Pour stocker les informations sensibles comme les tokens d'authentification. ``dart import 'package:flutter_secure_storage/flutter_secure_storage.dart';

```
class SecureStorageService { final FlutterSecureStorage _storage =
FlutterSecureStorage();

Future sauvegarderToken(String token) async {

await _storage.write(key: 'auth_token', value: token);

}

Future lireToken() async {

return await _storage.read(key: 'auth_token');

}

Future supprimerToken() async {

await _storage.delete(key: 'auth_token');

}} ``
```

Gestion des permissions :

1. Introduction aux permissions :

- Les applications peuvent nécessiter des permissions pour accéder à certaines fonctionnalités du dispositif, comme la caméra, le stockage, le microphone, etc.

2. Utiliser le package `permission_handler` :

- Ce package facilite la demande et la gestion des permissions.
- dependencies:
- `permission_handler: ^10.2.0`

3. Configuration pour Android :

- Ajoutez les permissions nécessaires dans le fichier **AndroidManifest.xml**.
- `<uses-permission android:name="android.permission.CAMERA" />`
- `<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />`

4. Configuration pour iOS :

- Ajoutez les descriptions des permissions dans le fichier **Info.plist**.
- `<key>NSCameraUsageDescription</key>`
- `<string>Cette application nécessite l'accès à la caméra.</string>`
- `<key>NSPhotoLibraryUsageDescription</key>`
- `<string>Cette application nécessite l'accès à la galerie de photos.</string>`

5. Demander des permissions dans Flutter :

6. `import 'package:permission_handler/permission_handler.dart';`
7. `import 'package:flutter/material.dart';`

```
8.
9. class PermissionPage extends StatelessWidget {
10.   Future<void> _demanderPermissions() async {
11.     var statusCamera = await Permission.camera.status;
12.     if (!statusCamera.isGranted) {
13.       await Permission.camera.request();
14.     }
15.
16.     var statusStorage = await Permission.storage.status;
17.     if (!statusStorage.isGranted) {
18.       await Permission.storage.request();
19.     }
20.   }
21.
22.   @override
23.   Widget build(BuildContext context) {
24.     return Scaffold(
25.       appBar: AppBar(title: Text('Gestion des Permissions')),
26.       body: Center(
27.         child: ElevatedButton(
28.           onPressed: _demanderPermissions,
29.           child: Text('Demander Permissions'),
30.         ),
31.       ),
32.     );
33.   }
34. }
```

Gestion des erreurs liées aux permissions :

- **Vérifier le statut des permissions** avant d'accéder à une fonctionnalité.
- **Informé l'utilisateur** en cas de refus et fournir des instructions pour activer les permissions via les paramètres.

Exemple de gestion des permissions avec feedback utilisateur :

```
import 'package:flutter/material.dart';
import 'package:permission_handler/permission_handler.dart';

class CameraAccessPage extends StatelessWidget {
  Future<void> _accéderCamera(BuildContext context) async {
    var status = await Permission.camera.status;
    if (status.isGranted) {
      // Accéder à la caméra
      Navigator.push(context, MaterialPageRoute(builder: (_) =>
CameraPage()));
    } else if (status.isDenied) {
      // Demander la permission
      if (await Permission.camera.request().isGranted) {
        Navigator.push(context, MaterialPageRoute(builder: (_) =>
CameraPage()));
      } else {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('Permission caméra refusée')),
        );
      }
    }
  }
}
```

```
    }  
  } else if (status.isPermanentlyDenied) {  
    // Ouvrir les paramètres  
    ScaffoldMessenger.of(context).showSnackBar(  
      SnackBar(  
        content: Text('Permission caméra définitivement refusée.  
Veuillez l\'activer dans les paramètres.'),  
        action: SnackBarAction(  
          label: 'Paramètres',  
          onPressed: () {  
            openAppSettings();  
          },  
        ),  
      ),  
    );  
  }  
}  
  
@override  
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(  
      title: Text('Accès Caméra'),  
    ),  
    body: Center(  
      child: ElevatedButton(  
        onPressed: () => _accéderCamera(context),  
        child: Text('Accéder à la Caméra'),  
      ),  
    ),  
  );  
}  
  
class CameraPage extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: Text('Caméra'),  
      ),  
      body: Center(  
        child: Text('Interface de la caméra'),  
      ),  
    );  
  }  
}
```

Bonnes pratiques :

- **Demander uniquement les permissions nécessaires** pour réduire les risques de refus.
- **Informez clairement l'utilisateur** sur la raison de la demande de permissions.
- **Gérer les permissions de manière proactive** pour éviter les interruptions dans l'expérience utilisateur.

Leçon 7 : Synchronisation des données entre les appareils

La synchronisation des données permet de maintenir la cohérence des informations entre plusieurs appareils et plateformes. Cette leçon explore les différentes méthodes et outils pour implémenter la synchronisation des données dans une application Flutter.

Concepts clés :

- **Synchronisation en temps réel** : Mise à jour instantanée des données sur tous les appareils.
- **Synchronisation différée** : Mise à jour des données lorsque l'appareil est connecté à Internet.
- **Conflits de données** : Gestion des divergences de données lors de la synchronisation.

Utiliser Cloud Firestore pour la synchronisation en temps réel :

```
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:flutter/material.dart';

class SynchronisationPage extends StatelessWidget {
  final FirebaseFirestore _firestore = FirebaseFirestore.instance;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Synchronisation en Temps Réel'),
      ),
      body: StreamBuilder<QuerySnapshot>(
        stream: _firestore.collection('messages').snapshots(),
        builder: (context, snapshot) {
          if (!snapshot.hasData) return CircularProgressIndicator();

          final messages = snapshot.data!.docs;

          return ListView.builder(
            itemCount: messages.length,
            itemBuilder: (context, index) {
              final message = messages[index].data() as Map<String,
dynamic>;

              return ListTile(
                title: Text(message['texte']),
                subtitle: Text(message['auteur']),
              );
            },
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () async {
          await _firestore.collection('messages').add({
            'texte': 'Nouveau message',

```

```
        'auteur': 'Utilisateur',
        'timestamp': FieldValue.serverTimestamp(),
      });
    },
    child: Icon(Icons.add),
  ),
);
}
}
```

Utiliser Firebase Realtime Database :

```
import 'package:firebase_database/firebase_database.dart';
import 'package:flutter/material.dart';

class RealtimeDatabasePage extends StatelessWidget {
  final DatabaseReference _dbRef =
    FirebaseDatabase.instance.ref('utilisateurs');

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Firebase Realtime Database'),
      ),
      body: StreamBuilder<DatabaseEvent>(
        stream: _dbRef.onValue,
        builder: (context, snapshot) {
          if (!snapshot.hasData) return CircularProgressIndicator();

          final data = snapshot.data!.snapshot.value as Map<dynamic,
dynamic>?;
          if (data == null) return Text('Aucune donnée');

          return ListView.builder(
            itemCount: data.length,
            itemBuilder: (context, index) {
              final key = data.keys.elementAt(index);
              final utilisateur = data[key];
              return ListTile(
                title: Text(utilisateur['nom']),
                subtitle: Text(utilisateur['email']),
              );
            },
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () async {
          await _dbRef.push().set({
            'nom': 'Nouvel Utilisateur',
            'email': 'nouvel@example.com',
          });
        },
        child: Icon(Icons.add),
      ),
    );
  }
}
```

```
);  
}  
}
```

Gestion des conflits de données :

- **Stratégies de résolution** : Implémentez des règles pour gérer les conflits, comme la priorité des données locales ou serveur.
- **Transactions** : Utilisez des transactions pour garantir l'intégrité des données lors des mises à jour concurrentes.

Synchronisation différée avec SQLite et Cloud Firestore :

- **Détection de la connectivité** : Utilisez le package `connectivity_plus` pour détecter l'état de la connexion.
- **Mise en cache des données** : Stockez les modifications localement et synchronisez-les lorsque l'appareil est connecté.

Exemple de synchronisation différée :

```
import 'package:connectivity_plus/connectivity_plus.dart';  
import 'package:hive/hive.dart';  
import 'package:cloud_firestore/cloud_firestore.dart';  
import 'package:flutter/material.dart';  
  
class SynchronisationDiffereePage extends StatefulWidget {  
  @override  
  _SynchronisationDiffereePageState createState() =>  
    _SynchronisationDiffereePageState();  
}  
  
class _SynchronisationDiffereePageState extends  
State<SynchronisationDiffereePage> {  
  final Box _box = Hive.box('modifications');  
  final FirebaseFirestore _firestore = FirebaseFirestore.instance;  
  late StreamSubscription<ConnectivityResult> _subscription;  
  
  @override  
  void initState() {  
    super.initState();  
    _subscription =  
Connectivity().onConnectivityChanged.listen((result) {  
    if (result != ConnectivityResult.none) {  
      _synchroniser();  
    }  
  });  
}  
  
Future<void> _synchroniser() async {  
  final modifications = _box.values.toList();  
  for (var modification in modifications) {  
    await _firestore.collection('utilisateurs').add(modification);  
  }  
}
```

Développement d'Applications Flutter

```
        await _box.clear();
        setState(() {});
    }

    Future<void> _ajouterModification(Map<String, dynamic> modification)
    async {
        final connectivity = await Connectivity().checkConnectivity();
        if (connectivity != ConnectivityResult.none) {
            await _firestore.collection('utilisateurs').add(modification);
        } else {
            await _box.add(modification);
        }
        setState(() {});
    }

    @override
    void dispose() {
        _subscription.cancel();
        super.dispose();
    }

    @override
    Widget build(BuildContext context) {
        final modifications = _box.values.toList();

        return Scaffold(
            appBar: AppBar(
                title: Text('Synchronisation Différée'),
            ),
            body: ListView.builder(
                itemCount: modifications.length,
                itemBuilder: (context, index) {
                    final modification = modifications[index] as Map<String,
dynamic>;
                    return ListTile(
                        title: Text(modification['nom']),
                        subtitle: Text(modification['email']),
                    );
                },
            ),
            floatingActionButton: FloatingActionButton(
                onPressed: () async {
                    final modification = {
                        'nom': 'Utilisateur Offline',
                        'email': 'offline@example.com',
                    };
                    await _ajouterModification(modification);
                },
                child: Icon(Icons.sync),
            ),
        );
    }
}
```

Bonnes pratiques :

- **Optimisation des synchronisations** : Synchronisez uniquement les données modifiées pour réduire la charge réseau.
- **Gestion des erreurs de synchronisation** : Implémentez des mécanismes de reprise en cas d'échec des synchronisations.
- **Notifications utilisateur** : Informez l'utilisateur lorsque la synchronisation est en cours ou a échoué.

Leçon 8 : Utilisation des notifications push avec Firebase Cloud Messaging

Introduction aux notifications push : Les **notifications push** permettent d'envoyer des messages aux utilisateurs même lorsque l'application est en arrière-plan ou fermée, améliorant l'engagement et l'interaction utilisateur.

Configuration de Firebase Cloud Messaging (FCM) :

1. **Ajouter FCM à votre projet Firebase** :
 - Suivez les étapes dans la [documentation officielle](#) pour ajouter FCM à votre application Flutter.
2. **Installer les dépendances nécessaires** dans `pubspec.yaml` :
3. `dependencies`:
4. `firebase_messaging: ^14.0.0`
5. **Configurer les plateformes** :
 - **Android** : Ajoutez les permissions nécessaires dans **AndroidManifest.xml**.
 - `<uses-permission android:name="android.permission.INTERNET"/>`
 - `<uses-permission android:name="android.permission.WAKE_LOCK"/>`
 - `<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>`
 - **iOS** : Configurez les capacités push notifications dans **Xcode** et demandez les permissions à l'utilisateur.

Initialisation de Firebase Messaging :

```
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:flutter/material.dart';

Future<void> _firebaseMessagingBackgroundHandler(RemoteMessage message)
async {
  await Firebase.initializeApp();
  print('Message reçu en arrière-plan : ${message.messageId}');
}

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();

  FirebaseMessaging.onBackgroundMessage(_firebaseMessagingBackgroundHandler);
}
```

Développement d'Applications Flutter

```
runApp(MyApp());
}

class MyApp extends StatefulWidget {
  @override
  _MyAppState createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  final FirebaseMessaging _messaging = FirebaseMessaging.instance;

  @override
  void initState() {
    super.initState();
    _initialiserFCM();
  }

  void _initialiserFCM() async {
    // Demander les permissions (iOS)
    NotificationSettings settings = await _messaging.requestPermission(
      alert: true,
      badge: true,
      sound: true,
    );

    if (settings.authorizationStatus == AuthorizationStatus.authorized)
    {
      print('Utilisateur a accordé les permissions');
    } else {
      print('Utilisateur a refusé les permissions');
    }

    // Obtenir le token FCM
    String? token = await _messaging.getToken();
    print('Token FCM : $token');

    // Gérer les messages lorsqu'ils arrivent en premier plan
    FirebaseMessaging.onMessage.listen((RemoteMessage message) {
      print('Message reçu en premier plan : ${message.messageId}');
      if (message.notification != null) {
        print('Titre : ${message.notification!.title}');
        print('Corps : ${message.notification!.body}');
      }
    });

    // Gérer les interactions avec les notifications
    FirebaseMessaging.onMessageOpenedApp.listen((RemoteMessage message)
    {
      print('Notification ouverte : ${message.messageId}');
      // Naviguer vers une page spécifique
    });
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Notifications Push Flutter',

```

```
        home: HomePage(),
      );
    }
  }

class HomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Accueil Notifications Push'),
      ),
      body: Center(
        child: Text('Attendez une notification push'),
      ),
    );
  }
}
```

Envoyer des notifications depuis Firebase Console :

1. **Accéder à Firebase Console.**
2. **Naviguer vers Cloud Messaging.**
3. **Créer une nouvelle notification** en définissant le titre, le corps et la cible (appareil ou groupe d'appareils).

Personnalisation des notifications :

- **Données personnalisées :** Envoyez des données supplémentaires avec les notifications pour une gestion plus précise.
- {
 - "notification": {
 - "title": "Nouveau Message",
 - "body": "Vous avez un nouveau message."
 - },
 - "data": {
 - "route": "/messages",
 - "id": "12345"
 - },
 - "to": "<FCM_TOKEN>"
- }

Gestion des notifications interactives :

- **Actions :** Ajoutez des actions personnalisées aux notifications pour permettre aux utilisateurs d'interagir directement.
- **Naviguer vers des écrans spécifiques** en fonction des données reçues avec la notification.

Bonnes pratiques :

- **Respect de la vie privée** : Informez les utilisateurs sur le type de notifications qu'ils recevront.
- **Optimisation des messages** : Envoyez des notifications pertinentes et évitez les abus pour maintenir l'engagement.
- **Test des notifications** : Testez les notifications dans différents scénarios pour garantir leur bon fonctionnement.

Leçon 9 : Optimisation des requêtes réseau et gestion des états asynchrones

Introduction à l'optimisation des requêtes réseau : L'optimisation des requêtes réseau améliore la performance de l'application et réduit la consommation de données. Cette leçon aborde les techniques pour optimiser les appels API et gérer efficacement les états asynchrones.

Techniques d'optimisation des requêtes :

1. Caching des réponses API :

- Utilisez des solutions de **caching** pour stocker temporairement les réponses et éviter les appels répétés.
- Exemple avec le package **dio_http_cache** :
- dependencies:
- dio: ^4.0.0
- dio_http_cache: ^0.3.0
- import 'package:dio/dio.dart';
- import 'package:dio_http_cache/dio_http_cache.dart';
-
- class ApiService {
- final Dio _dio = Dio();
-
- ApiService() {
-
- _dio.interceptors.add(DioCacheManager(CacheConfig()).interceptor);
- }
-
- Future<Response> fetchData() async {
- return await _dio.get(
- 'https://api.example.com/data',
- options: buildCacheOptions(Duration(days: 7)),
-);
- }
- }

2. Pagination et Lazy Loading :

- Implémentez la pagination pour charger les données en morceaux plutôt qu'en une seule requête. ``dart class PagedListPage extends StatefulWidget { @override _PagedListPageState createState() => _PagedListPageState(); }

```
class _PagedListPageState extends State { final ScrollController _scrollController = ScrollController(); final List _utilisateurs = []; int _page = 1; bool _isLoading = false;
```

```
@override void initState() {
```

```
  super.initState();  
  _chargerUtilisateurs();  
  _scrollController.addListener(() {  
    if (_scrollController.position.pixels ==  
        _scrollController.position.maxScrollExtent && !_isLoading) {  
      _chargerUtilisateurs();  
    }  
  });
```

```
}
```

```
Future _chargerUtilisateurs() async {
```

```
  setState(() {  
    _isLoading = true;  
  });
```

```
  // Remplacez par votre méthode d'appel API avec pagination  
  final nouveauxUtilisateurs = await  
  ApiService().fetchUtilisateurs(page: _page);
```

```
  setState(() {  
    _utilisateurs.addAll(nouveauxUtilisateurs);  
    _isLoading = false;  
    _page++;  
  });
```

```
}
```

```
@override void dispose() {
```

```
  _scrollController.dispose();  
  super.dispose();
```

```
}
```

```
@override Widget build(BuildContext context) {
```

```
  return Scaffold(  
    appBar: AppBar(title: Text('Liste Pagée')),  
    body: ListView.builder(  
      controller: _scrollController,  
      itemCount: _utilisateurs.length + 1,  
      itemBuilder: (context, index) {  
        if (index == _utilisateurs.length) {
```

```
        return _isLoading ? CircularProgressIndicator() :
        SizedBox.shrink();
      }
      final utilisateur = _utilisateurs[index];
      return ListTile(
        title: Text(utilisateur.nom),
        subtitle: Text(utilisateur.email),
      );
    },
  ),
);

}} ``
```

3. Minimisation des données échangées :

- Récupérez uniquement les champs nécessaires pour réduire la taille des réponses.
- Exemple avec Firestore :
- ```
Future<List<Utilisateur>> fetchUtilisateurs() async {
 final querySnapshot = await FirebaseFirestore.instance
 .collection('utilisateurs')
 .select(['nom', 'email']) // Sélectionner uniquement
 les champs nécessaires
 .get();
 return querySnapshot.docs.map((doc) =>
 Utilisateur.fromFirestore(doc)).toList();
}
```

### Gestion des états asynchrones avec Provider :

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class UtilisateurProvider with ChangeNotifier {
 final ApiService _apiService = ApiService();
 List<Utilisateur> _utilisateurs = [];
 bool _isLoading = false;
 String? _erreur;

 List<Utilisateur> get utilisateurs => _utilisateurs;
 bool get isLoading => _isLoading;
 String? get erreur => _erreur;

 Future<void> chargerUtilisateurs() async {
 _isLoading = true;
 notifyListeners();

 try {
 _utilisateurs = await _apiService.fetchUtilisateurs();
 _erreur = null;
 } catch (e) {
 _erreur = e.toString();
 }
 }
}
```

```
 _isLoading = false;
 notifyListeners();
 }
}

class UtilisateurListPage extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
 return ChangeNotifierProvider(
 create: (_) => UtilisateurProvider()..chargerUtilisateurs(),
 child: Scaffold(
 appBar: AppBar(title: Text('Liste Utilisateurs')),
 body: Consumer<UtilisateurProvider>(
 builder: (context, provider, child) {
 if (provider.isLoading) {
 return Center(child: CircularProgressIndicator());
 } else if (provider.erreur != null) {
 return Center(child: Text('Erreur :
${provider.erreur}'));
 } else {
 return ListView.builder(
 itemCount: provider.utilisateurs.length,
 itemBuilder: (context, index) {
 final utilisateur = provider.utilisateurs[index];
 return ListTile(
 title: Text(utilisateur.nom),
 subtitle: Text(utilisateur.email),
);
 },
);
 }
 },
),
),
);
 }
}
```

### Gestion des états asynchrones avec Riverpod :

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

final utilisateursProvider = FutureProvider<List<Utilisateur>>((ref)
 async {
 return await ApiService().fetchUtilisateurs();
 });

class UtilisateurRiverpodPage extends ConsumerWidget {
 @override
 Widget build(BuildContext context, WidgetRef ref) {
 final asyncUtilisateurs = ref.watch(utilisateursProvider);

 return Scaffold(
 appBar: AppBar(title: Text('Liste Utilisateurs Riverpod')),
);
 }
}
```

```
body: asyncUtilisateurs.when(
 data: (utilisateurs) {
 return ListView.builder(
 itemCount: utilisateurs.length,
 itemBuilder: (context, index) {
 final utilisateur = utilisateurs[index];
 return ListTile(
 title: Text(utilisateur.nom),
 subtitle: Text(utilisateur.email),
);
 },
);
 },
 loading: () => Center(child: CircularProgressIndicator()),
 error: (err, stack) => Center(child: Text('Erreur : $err')),
),
);
}
```

### Bonnes pratiques :

- **Optimisation des appels API** : Évitez les appels redondants et implémentez des mécanismes de caching.
- **Gestion des erreurs asynchrones** : Assurez-vous de capturer et de gérer les erreurs de manière appropriée pour améliorer l'expérience utilisateur.
- **Feedback utilisateur** : Informez l'utilisateur des états de chargement et des erreurs pour une meilleure interactivité.

## Module 5 : Fonctionnalités Avancées et Plugins

### Leçon 1 : Intégration de cartes avec Google Maps

**Introduction à Google Maps dans Flutter** : L'intégration de cartes permet d'afficher des emplacements géographiques, des itinéraires et d'autres données spatiales. Flutter propose le package `google_maps_flutter` pour intégrer Google Maps.

#### Installation du package `google_maps_flutter` :

1. **Ajouter la dépendance** dans le fichier `pubspec.yaml` :
2. `dependencies:`
3. `google_maps_flutter: ^2.2.3`
4. **Installer les packages** avec la commande `flutter pub get`.

#### Configuration pour Android :

- **Obtenir une clé API Google Maps** et l'ajouter dans `AndroidManifest.xml`.
- `<meta-data android:name="com.google.android.geo.API_KEY" android:value="YOUR_API_KEY"/>`

### Configuration pour iOS :

- **Obtenir une clé API Google Maps et l'ajouter dans AppDelegate.swift.**
- `import GoogleMaps`
- 
- `@UIApplicationMain`
- `@objc class AppDelegate: FlutterAppDelegate {`
- `override func application(`
- `_ application: UIApplication,`
- `didFinishLaunchingWithOptions launchOptions:`
- `[UIApplication.LaunchOptionsKey: Any]?`
- `) -> Bool {`
- `GMSServices.provideAPIKey("YOUR_API_KEY")`
- `GeneratedPluginRegistrant.register(with: self)`
- `return super.application(application,`
- `didFinishLaunchingWithOptions: launchOptions)`
- `}`
- `}`

### Utilisation de Google Maps dans Flutter :

```
import 'package:flutter/material.dart';
import 'package:google_maps_flutter/google_maps_flutter.dart';

class GoogleMapsPage extends StatefulWidget {
 @override
 _GoogleMapsPageState createState() => _GoogleMapsPageState();
}

class _GoogleMapsPageState extends State<GoogleMapsPage> {
 late GoogleMapController _mapController;
 final LatLng _centre = LatLng(48.8566, 2.3522); // Paris

 void _onMapCreated(GoogleMapController controller) {
 _mapController = controller;
 }

 @override
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Google Maps Integration'),
),
 body: GoogleMap(
 onMapCreated: _onMapCreated,
 initialCameraPosition: CameraPosition(
 target: _centre,
 zoom: 12,
),
 markers: {
 Marker(
 markerId: MarkerId('paris'),
 position: _centre,
)
 }
)
);
 }
}
```

```
 infoWindow: InfoWindow(title: 'Paris', snippet: 'Ville
Lumière'),
),
 },
),
 floatingActionButton: FloatingActionButton(
 onPressed: () {
 _mapController.animateCamera(
 CameraUpdate.newCameraPosition(
 CameraPosition(
 target: LatLng(51.5074, -0.1278), // Londres
 zoom: 12,
),
),
);
 },
 child: Icon(Icons.location_city),
),
);
}
```

### Personnalisation des cartes :

- **Markers** : Ajoutez des marqueurs personnalisés pour indiquer des emplacements spécifiques.
- **Polylines** : Dessinez des itinéraires ou des lignes sur la carte.
- **Polygons** : Dessinez des zones géographiques définies.

### Exemple d'ajout de Polyline :

```
Set<Polyline> _polylines = {};

void _ajouterItineraire() {
 setState(() {
 _polylines.add(
 Polyline(
 polylineId: PolylineId('itineraire1'),
 points: [
 LatLng(48.8566, 2.3522), // Paris
 LatLng(50.1109, 8.6821), // Francfort
],
 color: Colors.blue,
 width: 5,
),
);
 });
}

@override
Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Google Maps avec Polyline'),
),
);
}
```

```
),
body: GoogleMap(
 onMapCreated: _onMapCreated,
 initialCameraPosition: CameraPosition(
 target: _centre,
 zoom: 5,
),
 polylines: _polylines,
),
floatingActionButton: FloatingActionButton(
 onPressed: _ajouterItineraire,
 child: Icon(Icons.directions),
),
);
}
```

### Gestion des permissions de localisation :

- Utilisez le package **permission\_handler** pour demander les permissions de localisation nécessaires.
- Implémentez des fonctionnalités comme le suivi de la position en temps réel.

### Exemple de demande de permission de localisation :

```
import 'package:permission_handler/permission_handler.dart';

Future<void> _demanderPermissionLocalisation() async {
 var status = await Permission.location.status;
 if (!status.isGranted) {
 await Permission.location.request();
 }
}
```

### Bonnes pratiques :

- **Limiter les permissions** : Demandez uniquement les permissions nécessaires pour réduire les risques de refus.
- **Optimisation des rendus** : Utilisez des optimisations comme le clustering des marqueurs pour améliorer les performances sur les grandes cartes.
- **Gestion des erreurs** : Gérez les cas où les permissions sont refusées ou les services de localisation sont indisponibles.

## Leçon 2 : Utilisation des capteurs de l'appareil (GPS, gyroscope, etc.)

**Introduction aux capteurs de l'appareil** : Les capteurs comme le **GPS**, le **gyroscope**, l'**accéléromètre**, et le **magnétomètre** permettent de créer des applications interactives et réactives aux mouvements et à la localisation de l'utilisateur.

### Installation du package **sensor\_plus** :

1. **Ajouter la dépendance** dans le fichier `pubspec.yaml` :
2. `dependencies:`
3. `sensors_plus: ^2.0.0`
4. **Installer les packages** avec la commande `flutter pub get`.

### Utilisation du GPS avec le package `geolocator` :

```
dependencies:
 geolocator: ^9.0.0
import 'package:flutter/material.dart';
import 'package:geolocator/geolocator.dart';

class GPSPage extends StatefulWidget {
 @override
 _GPSPageState createState() => _GPSPageState();
}

class _GPSPageState extends State<GPSPage> {
 Position? _position;

 Future<void> _obtenirPosition() async {
 bool serviceEnabled;
 LocationPermission permission;

 // Vérifier si les services de localisation sont activés
 serviceEnabled = await Geolocator.isLocationServiceEnabled();
 if (!serviceEnabled) {
 ScaffoldMessenger.of(context).showSnackBar(
 SnackBar(content: Text('Les services de localisation sont
désactivés.')),
);
 return;
 }

 // Vérifier les permissions
 permission = await Geolocator.checkPermission();
 if (permission == LocationPermission.denied) {
 permission = await Geolocator.requestPermission();
 if (permission == LocationPermission.denied) {
 ScaffoldMessenger.of(context).showSnackBar(
 SnackBar(content: Text('Permissions de localisation
refusées.')),
);
 return;
 }
 }

 if (permission == LocationPermission.deniedForever) {
 ScaffoldMessenger.of(context).showSnackBar(
 SnackBar(content: Text('Permissions de localisation
définitivement refusées.')),
);
 return;
 }
 }
}
```

```
// Obtenir la position actuelle
Position position = await Geolocator.getCurrentPosition();
setState(() {
 _position = position;
});

}

@override
Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Utilisation du GPS'),
),
 body: Center(
 child: Column(
 mainAxisAlignment: MainAxisAlignment.center,
 children: [
 _position == null
 ? Text('Aucune position récupérée')
 : Text('Latitude: ${_position!.latitude}, Longitude:
${_position!.longitude}'),
 SizedBox(height: 20),
 ElevatedButton(
 onPressed: _obtenirPosition,
 child: Text('Obtenir Position'),
),
],
),
),
);
}
```

### Utilisation du gyroscope et de l'accéléromètre avec `sensors_plus` :

```
import 'package:flutter/material.dart';
import 'package:sensors_plus/sensors_plus.dart';

class CapteursPage extends StatefulWidget {
 @override
 _CapteursPageState createState() => _CapteursPageState();
}

class _CapteursPageState extends State<CapteursPage> {
 double _accelX = 0.0, _accelY = 0.0, _accelZ = 0.0;
 double _gyroX = 0.0, _gyroY = 0.0, _gyroZ = 0.0;

 @override
 void initState() {
 super.initState();

 // Écouter les données de l'accéléromètre
 accelerometerEvents.listen((AccelerometerEvent event) {
 setState(() {
 _accelX = event.x;
 _accelY = event.y;
 _accelZ = event.z;
 });
 });
 }
}
```

```
 });
 });

 // Écouter les données du gyroscope
 gyroscopeEvents.listen((GyroscopeEvent event) {
 setState(() {
 _gyroX = event.x;
 _gyroY = event.y;
 _gyroZ = event.z;
 });
 });
}

@override
Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Utilisation des Capteurs'),
),
 body: Padding(
 padding: const EdgeInsets.all(16.0),
 child: Column(
 children: [
 Text('Accéléromètre'),
 Text('X: $_accelX'),
 Text('Y: $_accelY'),
 Text('Z: $_accelZ'),
 SizedBox(height: 20),
 Text('Gyroscope'),
 Text('X: $_gyroX'),
 Text('Y: $_gyroY'),
 Text('Z: $_gyroZ'),
],
),
),
);
}
```

### Bonnes pratiques :

- **Optimisation de la consommation d'énergie** : Écoutez les capteurs seulement lorsque nécessaire pour économiser la batterie.
- **Gestion des flux de données** : Utilisez des stream controllers ou des techniques de throttling pour gérer les données des capteurs sans surcharger l'interface.
- **Respect de la vie privée** : Informez les utilisateurs de l'utilisation des capteurs et obtenez les permissions nécessaires.

## Leçon 3 : Notification Push avec Firebase Cloud Messaging

**Introduction à Firebase Cloud Messaging (FCM)** : FCM permet d'envoyer des notifications push aux utilisateurs sur différentes plateformes. Cette leçon détaille l'intégration de FCM dans une application Flutter pour envoyer et recevoir des notifications.

### Configuration initiale de FCM :

1. **Suivre les étapes de configuration de FCM** pour Android et iOS comme décrit dans la [documentation officielle](#).
2. **Assurer la configuration correcte des clés API** et des fichiers de configuration.

### Recevoir des notifications push :

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_messaging/firebase_messaging.dart';

Future<void> _firebaseMessagingBackgroundHandler(RemoteMessage message)
async {
 await Firebase.initializeApp();
 print('Message reçu en arrière-plan : ${message.messageId}');
}

void main() async {
 WidgetsFlutterBinding.ensureInitialized();
 await Firebase.initializeApp();

 FirebaseMessaging.onBackgroundMessage(_firebaseMessagingBackgroundHandler);
 runApp(MyApp());
}

class MyApp extends StatefulWidget {
 @override
 _MyAppState createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
 late FirebaseMessaging _messaging;

 @override
 void initState() {
 super.initState();
 _messaging = FirebaseMessaging.instance;

 // Demander les permissions (iOS)
 _messaging.requestPermission();

 // Obtenir le token FCM
 _messaging.getToken().then((token) {
 print('Token FCM : $token');
 });

 // Gérer les messages en premier plan
 FirebaseMessaging.onMessage.listen((RemoteMessage message) {
 print('Message reçu en premier plan : ${message.messageId}');
 if (message.notification != null) {
 print('Titre : ${message.notification!.title}');
 print('Corps : ${message.notification!.body}');
 }
 });
 }
}
```

```
});

// Gérer les interactions avec les notifications
FirebaseMessaging.onMessageOpenedApp.listen((RemoteMessage message)
{
 print('Notification ouverte : ${message.messageId}');
 // Naviguer vers une page spécifique
});
}

@override
Widget build(BuildContext context) {
 return MaterialApp(
 title: 'FCM Notifications Push',
 home: HomePage(),
);
}

class HomePage extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Accueil Notifications Push'),
),
 body: Center(
 child: Text('Attendez une notification push'),
),
);
 }
}
```

### Envoyer des notifications push depuis Firebase Console :

1. **Accéder à Firebase Console.**
2. **Naviguer vers Cloud Messaging.**
3. **Créer une nouvelle notification** en définissant le titre, le corps et la cible (appareil ou groupe d'appareils).

### Personnalisation des notifications :

- **Données personnalisées :** Ajoutez des données supplémentaires pour une gestion spécifique des notifications.
- {
  - "notification": {
    - "title": "Nouveau Message",
    - "body": "Vous avez un nouveau message."
  - },
  - "data": {
    - "route": "/messages",
    - "id": "12345"
  - },

- "to": "<FCM\_TOKEN>"
- }

### Gestion des notifications interactives :

- **Actions** : Ajoutez des actions personnalisées aux notifications pour permettre aux utilisateurs d'interagir directement.
- **Naviguer vers des écrans spécifiques** en fonction des données reçues avec la notification.

### Exemple de navigation basée sur les données de notification :

```
FirebaseMessaging.onMessageOpenedApp.listen((RemoteMessage message) {
 if (message.data['route'] != null) {
 Navigator.pushNamed(context, message.data['route'], arguments:
 message.data['id']);
 }
});
```

### Bonnes pratiques :

- **Limiter la fréquence des notifications** : Évitez de spammer les utilisateurs avec trop de notifications.
- **Personnalisation** : Envoyez des notifications pertinentes et personnalisées pour augmenter l'engagement.
- **Respecter la vie privée** : Informez les utilisateurs sur les types de notifications qu'ils recevront et obtenez leur consentement.

## Leçon 4 : Implémentation de l'authentification (Google, Facebook, Email)

**Introduction à l'authentification** : L'authentification permet de sécuriser l'accès à l'application et de personnaliser l'expérience utilisateur. Cette leçon couvre l'implémentation de différentes méthodes d'authentification avec Firebase Auth.

### Installation des dépendances :

1. **Ajouter les dépendances** dans le fichier `pubspec.yaml` :
2. `dependencies:`
3. `firebase_auth: ^4.0.0`
4. `google_sign_in: ^5.4.0`
5. `flutter_facebook_auth: ^5.0.0`
6. **Installer les packages** avec la commande `flutter pub get`.

### Authentification par Email et Mot de Passe :

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';

class EmailAuthPage extends StatefulWidget {
```

## Développement d'Applications Flutter

```
@override
_EmailAuthPageState createState() => _EmailAuthPageState();
}

class _EmailAuthPageState extends State<EmailAuthPage> {
 final FirebaseAuth _auth = FirebaseAuth.instance;
 final TextEditingController _emailController =
 TextEditingController();
 final TextEditingController _motDePasseController =
 TextEditingController();
 String? _message;

 Future<void> _inscrire() async {
 try {
 UserCredential userCredential = await
 _auth.createUserWithEmailAndPassword(
 email: _emailController.text,
 password: _motDePasseController.text,
);
 setState(() {
 _message = 'Inscription réussie :
 ${userCredential.user?.email}';
 });
 } catch (e) {
 setState(() {
 _message = 'Erreur : ${e.toString()}';
 });
 }
 }

 Future<void> _connecter() async {
 try {
 UserCredential userCredential = await
 _auth.signInWithEmailAndPassword(
 email: _emailController.text,
 password: _motDePasseController.text,
);
 setState(() {
 _message = 'Connexion réussie : ${userCredential.user?.email}';
 });
 } catch (e) {
 setState(() {
 _message = 'Erreur : ${e.toString()}';
 });
 }
 }

 @override
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Authentification Email'),
),
 body: Padding(
 padding: const EdgeInsets.all(16.0),
 child: Column(
 children: [
```

```
TextField(
 controller: _emailController,
 decoration: InputDecoration(labelText: 'Email'),
),
TextField(
 controller: _motDePasseController,
 decoration: InputDecoration(labelText: 'Mot de Passe'),
 obscureText: true,
),
 SizedBox(height: 20),
 ElevatedButton(
 onPressed: _inscrire,
 child: Text('Inscription'),
),
 ElevatedButton(
 onPressed: _connecter,
 child: Text('Connexion'),
),
 SizedBox(height: 20),
 Text(_message ?? ''),
],
),
));
}
```

### Authentification avec Google :

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:google_sign_in/google_sign_in.dart';

class GoogleAuthPage extends StatelessWidget {
 final FirebaseAuth _auth = FirebaseAuth.instance;
 final GoogleSignIn _googleSignIn = GoogleSignIn();

 Future<void> _signInWithGoogle() async {
 try {
 // Déclencher le processus de connexion Google
 final GoogleSignInAccount? googleUser = await
 _googleSignIn.signIn();
 if (googleUser == null) return; // L'utilisateur a annulé la
 connexion

 // Obtenir les informations d'authentification
 final GoogleSignInAuthentication googleAuth = await
 googleUser.authentication;

 // Créer une nouvelle credential pour Firebase
 final credential = GoogleAuthProvider.credential(
 accessToken: googleAuth.accessToken,
 idToken: googleAuth.idToken,
);

 // Se connecter avec le credential Google
```

```
 UserCredential userCredential = await
_auth.signInWithCredential(credential);
 print('Connexion réussie : ${userCredential.user?.displayName}');
 } catch (e) {
 print('Erreur : $e');
 }
 }

Future<void> _signOut() async {
 await _auth.signOut();
 await _googleSignIn.signOut();
 print('Déconnexion réussie');
}

@override
Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Authentification Google'),
),
 body: Center(
 child: Column(
 mainAxisAlignment: MainAxisAlignment.center,
 children: <Widget>[
 ElevatedButton(
 onPressed: _signInWithGoogle,
 child: Text('Se connecter avec Google'),
),
 ElevatedButton(
 onPressed: _signOut,
 child: Text('Se déconnecter'),
),
],
),
),
);
}
```

### Authentification avec Facebook :

```
import 'package:flutter/material.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter_facebook_auth/flutter_facebook_auth.dart';

class FacebookAuthPage extends StatelessWidget {
 final FirebaseAuth _auth = FirebaseAuth.instance;

 Future<void> _signInWithFacebook() async {
 try {
 // Déclencher le processus de connexion Facebook
 final LoginResult result = await FacebookAuth.instance.login();

 if (result.status == LoginStatus.success) {
 // Obtenir le credential Facebook
 }
 }
 }
}
```

## Développement d'Applications Flutter

```
final OAuthCredential credential =
FacebookAuthProvider.credential(result.accessToken!.token);

// Se connecter avec le credential Facebook
UserCredential userCredential = await
_auth.signInWithCredential(credential);
print('Connexion réussie :
${userCredential.user?.displayName}');
} else {
print('Erreur de connexion Facebook : ${result.status}');
}
} catch (e) {
print('Erreur : $e');
}
}

Future<void> _signOut() async {
await _auth.signOut();
await FirebaseAuth.instance.logout();
print('Déconnexion réussie');
}

@override
Widget build(BuildContext context) {
return Scaffold(
 appBar: AppBar(
 title: Text('Authentification Facebook'),
),
 body: Center(
 child: Column(
 mainAxisAlignment: MainAxisAlignment.center,
 children: <Widget>[
 ElevatedButton(
 onPressed: _signInWithFacebook,
 child: Text('Se connecter avec Facebook'),
),
 ElevatedButton(
 onPressed: _signOut,
 child: Text('Se déconnecter'),
),
],
),
),
);
}
```

### Bonnes pratiques :

- **Sécurité** : Assurez-vous de gérer correctement les tokens et les credentials pour éviter les vulnérabilités.
- **Expérience utilisateur** : Fournissez des feedbacks clairs lors des processus d'authentification (chargement, erreurs, succès).
- **Gestion des erreurs** : Gérez les différents scénarios d'erreur (annulation, échec de la connexion) de manière appropriée.

## Leçon 5 : Création et gestion d'une base de données en temps réel

**Introduction à Firebase Realtime Database :** Firebase Realtime Database est une base de données NoSQL en temps réel qui synchronise les données entre les clients en temps réel. Elle est idéale pour les applications nécessitant des mises à jour instantanées.

### Configuration de Firebase Realtime Database :

1. **Activer Realtime Database** dans Firebase Console.
2. **Configurer les règles de sécurité** pour restreindre l'accès aux données.

### Installation du package `firebase_database` :

1. **Ajouter la dépendance** dans le fichier `pubspec.yaml` :
2. `dependencies:`
3. `firebase_database: ^10.0.0`
4. **Installer les packages** avec la commande `flutter pub get`.

### Utilisation de Firebase Realtime Database :

```
import 'package:flutter/material.dart';
import 'package:firebase_database/firebase_database.dart';

class RealtimeDatabasePage extends StatefulWidget {
 @override
 _RealtimeDatabasePageState createState() =>
 _RealtimeDatabasePageState();
}

class _RealtimeDatabasePageState extends State<RealtimeDatabasePage> {
 final DatabaseReference _dbRef =
 FirebaseDatabase.instance.ref('messages');
 final TextEditingController _messageController =
 TextEditingController();
 List<String> _messages = [];

 @override
 void initState() {
 super.initState();
 _ecouterMessages();
 }

 void _ecouterMessages() {
 _dbRef.onValue.listen((DatabaseEvent event) {
 final data = event.snapshot.value as Map<dynamic, dynamic>?;
 if (data != null) {
 setState(() {
 _messages = data.values.map((value) => value['texte'] as
String).toList();
 });
 }
 });
 }
}
```

```

 }

 Future<void> _envoyerMessage(String texte) async {
 await _dbRef.push().set({'texte': texte, 'timestamp':
DateTime.now().toIso8601String()});
 }

 @override
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Realtime Database'),
),
 body: Column(
 children: [
 Expanded(
 child: ListView.builder(
 itemCount: _messages.length,
 itemBuilder: (context, index) {
 return ListTile(
 title: Text(_messages[index]),
);
 },
),
),
 Padding(
 padding: const EdgeInsets.all(8.0),
 child: Row(
 children: [
 Expanded(
 child: TextField(
 controller: _messageController,
 decoration: InputDecoration(labelText: 'Votre
message'),
),
),
 IconButton(
 icon: Icon(Icons.send),
 onPressed: () {
 if (_messageController.text.isNotEmpty) {
 _envoyerMessage(_messageController.text);
 _messageController.clear();
 }
 },
),
],
),
),
],
),
);
 }
 }
}

```

**Gestion des états en temps réel :**

- **Écoute des changements** : Utilisez des écouteurs pour réagir en temps réel aux modifications des données.
- **Optimisation des mises à jour** : Limitez les mises à jour aux données pertinentes pour améliorer les performances.

### Exemple avec filtres et limites :

```
void _ecouterMessagesFiltres() {

 _dbRef.orderByChild('timestamp').limitToLast(50).onValue.listen((DatabaseEvent event) {
 final data = event.snapshot.value as Map<dynamic, dynamic>;
 if (data != null) {
 setState(() {
 _messages = data.values.map((value) => value['texte'] as String).toList();
 });
 }
 });
}
```

### Intégration avec d'autres services :

- **Authentification** : Restreignez l'accès aux données en fonction de l'utilisateur connecté.
- **Notifications** : Envoyez des notifications push en réponse aux changements de données.

### Bonnes pratiques :

- **Sécurité des données** : Configurez les règles de sécurité pour restreindre l'accès et protéger les données sensibles.
- **Optimisation des performances** : Limitez les données transférées et optimisez les requêtes pour améliorer les performances.
- **Gestion des erreurs** : Implémentez une gestion robuste des erreurs pour traiter les échecs de connexion ou les interruptions réseau.

## Leçon 6 : Gestion de l'appareil photo et des fichiers médias

**Introduction à la gestion des fichiers médias** : La gestion des fichiers médias comme les photos, vidéos et audios permet de créer des applications riches en fonctionnalités. Cette leçon couvre l'utilisation des packages `image_picker` et `firebase_storage` pour gérer les médias.

### Installation des dépendances :

1. **Ajouter les dépendances** dans le fichier `pubspec.yaml` :
2. `dependencies:`
3. `image_picker: ^0.8.5+3`

4. `firebase_storage: ^11.0.0`
5. `firebase_core: ^2.0.0`
6. **Installer les packages avec la commande `flutter pub get`.**

### Utilisation du package `image_picker` :

```
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';
import 'dart:io';

class CameraPage extends StatefulWidget {
 @override
 _CameraPageState createState() => _CameraPageState();
}

class _CameraPageState extends State<CameraPage> {
 File? _image;

 Future<void> _prendrePhoto() async {
 final ImagePicker _picker = ImagePicker();
 final XFile? photo = await _picker.pickImage(source:
 ImageSource.camera);

 if (photo != null) {
 setState(() {
 _image = File(photo.path);
 });
 }
 }

 Future<void> _choisirImage() async {
 final ImagePicker _picker = ImagePicker();
 final XFile? image = await _picker.pickImage(source:
 ImageSource.gallery);

 if (image != null) {
 setState(() {
 _image = File(image.path);
 });
 }
 }

 @override
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Gestion de l\'Appareil Photo'),
),
 body: Center(
 child: Column(
 children: [
 _image == null
 ? Text('Aucune image sélectionnée')
 : Image.file(_image!),
 SizedBox(height: 20),
 ElevatedButton(
```

```
 onPressed: _prendrePhoto,
 child: Text('Prendre une Photo'),
),
 ElevatedButton(
 onPressed: _choisirImage,
 child: Text('Choisir une Image'),
),
],
),
));
}
```

### Télécharger les fichiers médias sur Firebase Storage :

```
import 'package:flutter/material.dart';
import 'package:firebase_storage/firebase_storage.dart';
import 'dart:io';

class UploadMediaPage extends StatefulWidget {
 final File imageFile;

 UploadMediaPage({required this.imageFile});

 @override
 _UploadMediaPageState createState() => _UploadMediaPageState();
}

class _UploadMediaPageState extends State<UploadMediaPage> {
 final FirebaseStorage _storage = FirebaseStorage.instance;
 String? _downloadUrl;

 Future<void> _telechargerImage() async {
 try {
 String chemin =
 'images/${DateTime.now().millisecondsSinceEpoch}.png';
 TaskSnapshot snapshot = await
 _storage.ref().child(chemin).putFile(widget.imageFile);
 String url = await snapshot.ref.getDownloadURL();
 setState(() {
 _downloadUrl = url;
 });
 print('Image téléchargée : $url');
 } catch (e) {
 print('Erreur lors du téléchargement : $e');
 }
 }

 @override
 void initState() {
 super.initState();
 _telechargerImage();
 }

 @override
 Widget build(BuildContext context) {
```

```
return Scaffold(
 appBar: AppBar(
 title: Text('Téléchargement de l\'Image'),
),
 body: Center(
 child: _downloadUrl == null
 ? CircularProgressIndicator()
 : Image.network(_downloadUrl!),
));
}
```

### Gestion des fichiers médias :

- **Affichage des médias** : Utilisez des widgets comme **Image**, **VideoPlayer** pour afficher les médias.
- **Gestion des permissions** : Demandez les permissions nécessaires pour accéder à la caméra et à la galerie.

### Exemple d'affichage de vidéos avec `video_player` :

```
dependencies:
 video_player: ^2.4.0
import 'package:flutter/material.dart';
import 'package:video_player/video_player.dart';

class VideoPage extends StatefulWidget {
 final String videoUrl;

 VideoPage({required this.videoUrl});

 @override
 _VideoPageState createState() => _VideoPageState();
}

class _VideoPageState extends State<VideoPage> {
 late VideoPlayerController _controller;

 @override
 void initState() {
 super.initState();
 _controller = VideoPlayerController.network(widget.videoUrl)
 ..initialize().then((_) {
 setState(() {});
 _controller.play();
 });
 }

 @override
 void dispose() {
 _controller.dispose();
 super.dispose();
 }
}
```

```
@override
Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Lecture Vidéo'),
),
 body: Center(
 child: _controller.value.isInitialized
 ? AspectRatio(
 aspectRatio: _controller.value.aspectRatio,
 child: VideoPlayer(_controller),
)
 : CircularProgressIndicator(),
));
}
```

### Bonnes pratiques :

- **Gestion de la mémoire** : Disposez correctement des contrôleurs de médias pour éviter les fuites de mémoire.
- **Compression des médias** : Optimisez les fichiers médias pour réduire la taille et améliorer les temps de chargement.
- **Sécurité** : Restreignez l'accès aux fichiers sensibles stockés dans Firebase Storage avec des règles de sécurité appropriées.

## Leçon 6 : Publication d'un plugin Flutter personnalisé

**Introduction aux plugins Flutter** : Les **plugins Flutter** permettent d'étendre les fonctionnalités de Flutter en intégrant du code natif (Android/iOS) avec du code Dart. Cette leçon couvre la création et la publication d'un plugin Flutter personnalisé.

### Création d'un plugin Flutter :

1. **Créer un plugin** avec la commande Flutter CLI :
2. `flutter create --template=plugin --platforms=android,ios custom_plugin`
3. **Structure du plugin** :
4. `custom_plugin/`
5. | `lib/`
6. | | `custom_plugin.dart`
7. | `android/`
8. | `ios/`
9. | `example/`
10. | `pubspec.yaml`

### Développer le code Dart :

```
// lib/custom_plugin.dart
import 'dart:async';
import 'package:flutter/services.dart';
```

## Développement d'Applications Flutter

```
class CustomPlugin {
 static const MethodChannel _channel = MethodChannel('custom_plugin');

 static Future<String?> get platformVersion async {
 final String? version = await
_channel.invokeMethod('getPlatformVersion');
 return version;
 }

 static Future<void> envoyerMessage(String message) async {
 await _channel.invokeMethod('envoyerMessage', {'message':
message});
 }
}
```

### Développer le code natif Android :

```
// android/src/main/kotlin/com/example/custom_plugin/CustomPlugin.kt
package com.example.custom_plugin

import androidx.annotation.NonNull
import io.flutter.embedding.engine.plugins.FlutterPlugin
import io.flutter.plugin.common.MethodChannel

class CustomPlugin: FlutterPlugin {
 private lateinit var channel : MethodChannel

 override fun onAttachedToEngine(@NonNull flutterPluginBinding:
FlutterPlugin.FlutterPluginBinding) {
 channel = MethodChannel(flutterPluginBinding.binaryMessenger,
"custom_plugin")
 channel.setMethodCallHandler { call, result ->
 if (call.method == "getPlatformVersion") {
 result.success("Android ${android.os.Build.VERSION.RELEASE}")
 } else if (call.method == "envoyerMessage") {
 val message = call.argument<String>("message")
 // Traitement du message
 println("Message reçu : $message")
 result.success(null)
 } else {
 result.notImplemented()
 }
 }
 }

 override fun onDetachedFromEngine(@NonNull binding:
FlutterPlugin.FlutterPluginBinding) {
 channel.setMethodCallHandler(null)
 }
}
```

### Développer le code natif iOS :

```
// ios/Classes/CustomPlugin.swift
```

## Développement d'Applications Flutter

```
import Flutter
import UIKit

public class SwiftCustomPlugin: NSObject, FlutterPlugin {
 public static func register(with registrar: FlutterPluginRegistrar) {
 let channel = FlutterMethodChannel(name: "custom_plugin",
binaryMessenger: registrar.messenger())
 let instance = SwiftCustomPlugin()
 registrar.addMethodCallDelegate(instance, channel: channel)
 }

 public func handle(_ call: FlutterMethodCall, result: @escaping
FlutterResult) {
 if call.method == "getPlatformVersion" {
 result("iOS " + UIDevice.current.systemVersion)
 } else if call.method == "envoyerMessage" {
 if let args = call.arguments as? [String: Any],
 let message = args["message"] as? String {
 // Traitement du message
 print("Message reçu : \(message)")
 result(nil)
 } else {
 result(FlutterError(code: "INVALID_ARGUMENT", message: "Message
manquant", details: nil))
 }
 } else {
 result(FlutterMethodNotImplemented)
 }
 }
}
```

### Tester le plugin dans l'exemple :

```
// example/lib/main.dart
import 'package:flutter/material.dart';
import 'package:custom_plugin/custom_plugin.dart';

void main() {
 runApp(MyApp());
}

class MyApp extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
 return MaterialApp(
 title: 'Custom Plugin Example',
 home: Scaffold(
 appBar: AppBar(
 title: Text('Custom Plugin'),
),
 body: Center(
 child: Column(
 mainAxisAlignment: MainAxisAlignment.center,
 children: [
 FutureBuilder<String?>(
 future: CustomPlugin.platformVersion,
```

## Développement d'Applications Flutter

```
builder: (context, snapshot) {
 if (snapshot.connectionState ==
 ConnectionState.waiting) {
 return CircularProgressIndicator();
 } else if (snapshot.hasError) {
 return Text('Erreur: ${snapshot.error}');
 } else {
 return Text('Version Plateforme:
 ${snapshot.data}');
 }
},
),
ElevatedButton(
 onPressed: () {
 CustomPlugin.envoyerMessage('Hello from Flutter!');
 },
 child: Text('Envoyer Message'),
),
],
),
),
),
);
}
```

### Préparer le plugin pour la publication :

1. **Ajouter une documentation** détaillée dans le fichier **README.md**.
2. **Définir les métadonnées** dans le fichier **pubspec.yaml** :
3. name: custom\_plugin
4. description: Un plugin Flutter personnalisé pour envoyer des messages.
5. version: 1.0.0
6. author: Votre Nom <votre.email@example.com>
7. homepage: [https://github.com/votreusername/custom\\_plugin](https://github.com/votreusername/custom_plugin)
8. environment:
9. sdk: ">=2.17.0 <3.0.0"

### Publier le plugin sur Pub.dev :

1. **Créer un compte** sur [Pub.dev](https://pub.dev).
2. **Authentifier** votre terminal avec la clé API de Pub.dev :
3. flutter pub login
4. **Publier le plugin** avec la commande :
5. flutter pub publish --dry-run
6. flutter pub publish
7. **Suivre les instructions** pour valider et publier le plugin.

### Bonnes pratiques :

- **Documentation claire** : Fournissez une documentation complète et des exemples d'utilisation.

- **Versioning** : Utilisez une gestion de version sémantique pour faciliter les mises à jour.
- **Tests** : Implémentez des tests unitaires et d'intégration pour garantir la fiabilité du plugin.
- **Maintenabilité** : Maintenez le plugin à jour avec les nouvelles versions de Flutter et les exigences des plateformes.

### Leçon 7 : Publication d'un plugin Flutter personnalisé

**Introduction à la publication d'un plugin** : Publier un plugin Flutter permet de partager vos solutions avec la communauté et d'enrichir l'écosystème Flutter. Cette leçon détaille les étapes nécessaires pour créer, tester et publier un plugin Flutter personnalisé sur **Pub.dev**.

#### Étapes de création d'un plugin Flutter :

1. **Créer le plugin** :
2. `flutter create --template=plugin --platforms=android,ios custom_plugin`
3. **Développer le plugin** :
  - **Code Dart** : Écrivez l'API publique du plugin dans `lib/custom_plugin.dart`.
  - **Code natif** : Implémentez les fonctionnalités spécifiques à chaque plateforme dans les dossiers `android/` et `ios/`.
4. **Tester le plugin** :
  - **Exemple** : Utilisez le projet `example/` pour tester votre plugin.
  - **Ajouter des tests unitaires** dans le dossier `test/`.

#### Exemple de code Dart pour le plugin :

```
// lib/custom_plugin.dart
import 'dart:async';
import 'package:flutter/services.dart';

class CustomPlugin {
 static const MethodChannel _channel = MethodChannel('custom_plugin');

 /// Récupère la version de la plateforme.
 static Future<String?> get platformVersion async {
 final String? version = await
_channel.invokeMethod('getPlatformVersion');
 return version;
 }

 /// Envoie un message natif.
 static Future<void> envoyerMessage(String message) async {
 await _channel.invokeMethod('envoyerMessage', {'message':
message});
 }
}
```

### Tests unitaires pour le plugin :

```
// test/custom_plugin_test.dart
import 'package:flutter/services.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:custom_plugin/custom_plugin.dart';

void main() {
 const MethodChannel channel = MethodChannel('custom_plugin');

 TestWidgetsFlutterBinding.ensureInitialized();

 group('CustomPlugin', () {

 TestDefaultBinaryMessengerBinding.instance.defaultBinaryMessenger.setMockMethodCallHandler(channel, (MethodCall methodCall) async {
 if (methodCall.method == 'getPlatformVersion') {
 return 'Android 10';
 }
 return null;
 });

 test('getPlatformVersion returns correct version', () async {
 expect(await CustomPlugin.platformVersion, 'Android 10');
 });

 test('envoyerMessage', () async {
 await CustomPlugin.envoyerMessage('Test Message');
 // Vérifiez les effets secondaires ou les logs si nécessaire
 });
 });
}
```

### Préparer le plugin pour la publication :

1. **Ajouter une documentation complète** dans le fichier **README.md**.
  - **Description** : Expliquez les fonctionnalités du plugin.
  - **Installation** : Montrez comment ajouter le plugin dans un projet Flutter.
  - **Exemples d'utilisation** : Fournissez des exemples de code pour illustrer l'utilisation du plugin.
  - **API Reference** : Documentez les méthodes et les propriétés disponibles.
2. **Définir les métadonnées** dans le fichier **pubspec.yaml** :
3. name: custom\_plugin
4. description: Un plugin Flutter personnalisé pour envoyer des messages.
5. version: 1.0.0
6. author: Votre Nom <votre.email@example.com>
7. homepage: [https://github.com/votreusername/custom\\_plugin](https://github.com/votreusername/custom_plugin)
8. repository: [https://github.com/votreusername/custom\\_plugin](https://github.com/votreusername/custom_plugin)
9. environment:
10. sdk: ">=2.17.0 <3.0.0"
11. dependencies:
12. flutter:
13. sdk: flutter

```
14. dev_dependencies:
15. flutter_test:
16. sdk: flutter
17. mockito: ^5.0.0
18. flutter:
19. plugin:
20. platforms:
21. android:
22. package: com.example.custom_plugin
23. pluginClass: CustomPlugin
24. ios:
25. pluginClass: SwiftCustomPlugin
```

### Validation avant la publication :

1. **Exécuter une vérification à sec :**
2. `flutter pub publish --dry-run`
  - **Corrigez les erreurs** signalées lors de la vérification.
3. **Publier le plugin :**
4. `flutter pub publish`
  - **Suivez les instructions** pour authentifier et confirmer la publication.

### Bonnes pratiques :

- **Documentation claire et complète** : Facilitez l'adoption de votre plugin en fournissant une documentation détaillée.
- **Tests robustes** : Assurez-vous que votre plugin fonctionne correctement sur toutes les plateformes supportées.
- **Gestion des versions** : Utilisez un système de versionnage sémantique pour gérer les mises à jour et les compatibilités.
- **Support et maintenance** : Soyez réactif aux feedbacks et aux issues des utilisateurs pour maintenir la qualité du plugin.
- **Sécurité** : Assurez-vous que votre plugin ne présente pas de vulnérabilités et respecte les meilleures pratiques de sécurité.

## Module 6 : Optimisation, Tests et Déploiement

### Leçon 1 : Optimiser les performances d'une application Flutter

**Introduction à l'optimisation des performances** : L'optimisation des performances est cruciale pour garantir une expérience utilisateur fluide et réactive. Une application Flutter bien optimisée consomme moins de ressources, démarre plus rapidement et répond mieux aux interactions utilisateur.

#### Principes de base :

- **Minimiser les reconstructions inutiles** : Évitez de reconstruire l'arborescence des widgets sans nécessité.

- **Optimiser les images et les assets** : Utilisez des formats adaptés et compressez les images pour réduire la charge.
- **Gérer efficacement l'état** : Utilisez des solutions de gestion d'état appropriées pour éviter les recalculs et les reconstructions excessives.

### Techniques d'optimisation :

#### 1. Réduction des reconstructions de widgets :

- Utilisez des widgets **const** lorsque c'est possible pour éviter les reconstructions.
- `const Text('Texte constant')`
- Utilisez **shouldRebuild** dans les widgets personnalisés pour contrôler les reconstructions.

#### 2. Optimisation des ListViews :

- Utilisez **ListView.builder** ou **ListView.separated** pour des listes dynamiques et performantes.
- `ListView.builder(`
- `itemCount: 1000,`
- `itemBuilder: (context, index) {`
- `return ListTile(title: Text('Item $index'));`
- `},`
- `)`

#### 3. Gestion efficace des images :

- Utilisez des images **lazy-loaded** avec des packages comme **cached\_network\_image**.
- `CachedNetworkImage(`
- `imageUrl: 'https://example.com/image.jpg',`
- `placeholder: (context, url) =>`
- `CircularProgressIndicator(),`
- `errorWidget: (context, url, error) => Icon(Icons.error),`
- `)`
- Comprimez et redimensionnez les images pour réduire la taille des fichiers.

#### 4. Utilisation de **const** :

- Déclarez les widgets **const** pour améliorer les performances en permettant à Flutter de les réutiliser sans les reconstruire.
- `const Icon(Icons.star)`

#### 5. Optimisation de la gestion d'état :

- Utilisez des solutions de gestion d'état efficaces comme **Provider**, **Riverpod**, ou **BLoC** pour minimiser les reconstructions.
- Gardez l'état local quand c'est possible pour réduire la portée des reconstructions.

#### 6. Lazy Loading et Pagination :

- Implémentez le **lazy loading** pour charger les données au fur et à mesure que l'utilisateur fait défiler la liste.
- `ListView.builder(`
- `controller: _scrollController,`
- `itemCount: _items.length,`
- `itemBuilder: (context, index) {`
- `if (index == _items.length - 1) {`

```
o // Charger plus de données
o }
o return ListTile(title: Text(_items[index]));
o },
o)
```

### 7. Utilisation de **RepaintBoundary** :

- o Encapsulez les parties de l'interface utilisateur qui changent souvent dans des **RepaintBoundary** pour limiter les zones de redessin.
- o `RepaintBoundary`(
  - o `child: Container`(
    - o `color: Colors.blue`,
    - o `child: Text('Contenu isolé')`,
  - o ),
- o )

### 8. Profilage et Debugging :

- o Utilisez **Flutter DevTools** pour profiler les performances de l'application, identifier les goulots d'étranglement, et optimiser le rendu.
  - **Timeline** : Analysez les frames et identifiez les problèmes de performance.
  - **Widget Inspector** : Inspectez la hiérarchie des widgets et leur état.
  - **Memory Profiler** : Surveillez l'utilisation de la mémoire pour éviter les fuites.

### Bonnes pratiques :

- **Minimiser les reconstructions** : Utilisez des widgets immuables et des méthodes efficaces pour limiter les reconstructions.
- **Optimiser les assets** : Compressez les images et utilisez des formats adaptés pour réduire la taille des fichiers.
- **Profilage régulier** : Intégrez des sessions de profilage dans votre cycle de développement pour identifier et corriger les problèmes de performance tôt.
- **Utiliser des packages optimisés** : Choisissez des packages maintenus et optimisés pour éviter les ralentissements.

## Leçon 2 : Introduction aux tests unitaires et tests widgets

**Introduction aux tests dans Flutter** : Les tests sont essentiels pour garantir la qualité, la fiabilité et la maintenabilité des applications Flutter. Flutter supporte différents types de tests : **tests unitaires**, **tests widgets**, et **tests d'intégration**.

### Types de tests :

#### 1. Tests unitaires :

- o Testent des unités de code isolées, comme des fonctions ou des classes.
- o Utilisés pour vérifier la logique métier et les calculs.
- o **Exemple** :

```
o void main() {
o test('Addition devrait retourner la somme de deux
nombres', () {
```

```
o final result = addition(2, 3);
o expect(result, 5);
o });
o }
o
o int addition(int a, int b) => a + b;
```

### 2. Tests widgets :

- o Testent les widgets individuels et leur interaction.
- o Utilisés pour vérifier l'interface utilisateur et le comportement des widgets.
- o **Exemple :**

```
o void main() {
o testWidgets('Bouton incrémente le compteur',
o (WidgetTester tester) async {
o await tester.pumpWidget(
o ProviderScope(
o child: MaterialApp(home: CompteurPage()),
o),
o);
o
o expect(find.text('Compteur : 0'), findsOneWidget);
o
o await tester.tap(find.byType(ElevatedButton));
o await tester.pump();
o
o expect(find.text('Compteur : 1'), findsOneWidget);
o });
o }
```

### Configuration des tests :

#### 1. Ajouter les dépendances de test :

- o Assurez-vous que le fichier **pubspec.yaml** contient les dépendances nécessaires.
- o dev\_dependencies:
- o flutter\_test:
- o sdk: flutter
- o mockito: ^5.0.0

#### 2. Structure des fichiers de test :

- o Placez les tests dans le répertoire **test/**, en imitant la structure des fichiers de l'application.

```
o test/
o └─ unit/
o └─ addition_test.dart
o └─ widgets/
o └─ compteur_test.dart
o └─ integration/
o └─ navigation_test.dart
```

### Écrire des tests unitaires :

```
// test/unit/addition_test.dart
import 'package:flutter_test/flutter_test.dart';
```

```
int addition(int a, int b) => a + b;

void main() {
 test('Addition devrait retourner la somme de deux nombres', () {
 final result = addition(2, 3);
 expect(result, 5);
 });

 test('Addition avec zéro', () {
 final result = addition(0, 5);
 expect(result, 5);
 });

 test('Addition de nombres négatifs', () {
 final result = addition(-2, -3);
 expect(result, -5);
 });
}
```

### Écrire des tests widgets :

```
// test/widgets/compteur_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:provider/provider.dart';
import 'package:custom_plugin/custom_plugin.dart'; // Remplacez par
votre package réel
import 'package:your_app/compteur_page.dart'; // Remplacez par votre
fichier réel

class MockCompteurProvider with ChangeNotifier {
 int _compteur = 0;
 int get compteur => _compteur;

 void incrementer() {
 _compteur++;
 notifyListeners();
 }
}

void main() {
 testWidgets('Bouton incrémente le compteur', (WidgetTester tester)
 async {
 await tester.pumpWidget(
 ChangeNotifierProvider<MockCompteurProvider>(
 create: (_) => MockCompteurProvider(),
 child: MaterialApp(home: CompteurPage()),
),
);

 expect(find.text('Compteur : 0'), findsOneWidget);

 await tester.tap(find.byType(ElevatedButton));
 await tester.pump();
 });
}
```

```
 expect(find.text('Compteur : 1'), findsOneWidget);
 });

 testWidgets('Affichage initial du compteur', (WidgetTester tester)
 async {
 await tester.pumpWidget(
 ChangeNotifierProvider<MockCompteurProvider>(
 create: (_) => MockCompteurProvider(),
 child: MaterialApp(home: CompteurPage()),
),
);

 expect(find.text('Compteur : 0'), findsOneWidget);
 expect(find.text('Compteur : 1'), findsNothing);
 });
}
```

### Bonnes pratiques :

- **Isoler les tests** : Chaque test doit être indépendant des autres pour éviter les interférences.
- **Couvrir tous les cas d'utilisation** : Assurez-vous que les tests couvrent les différents scénarios et cas limites.
- **Automatiser les tests** : Intégrez les tests dans un pipeline CI/CD pour garantir une qualité continue.
- **Utiliser des mocks** : Utilisez des mocks pour isoler les tests des dépendances externes et améliorer la fiabilité des tests.

## Leçon 3 : Tests d'intégration et automatisation des tests

**Introduction aux tests d'intégration** : Les **tests d'intégration** vérifient l'ensemble de l'application ou de grandes parties de celle-ci pour s'assurer que les différents modules interagissent correctement. Ils simulent des scénarios réels d'utilisation pour garantir la cohérence et la fonctionnalité de l'application.

### Utiliser le package `integration_test` :

1. **Ajouter les dépendances** dans le fichier `pubspec.yaml` :
2. `dev_dependencies:`
3. `integration_test:`
4.  `sdk: flutter`
5. `flutter_test:`
6.  `sdk: flutter`
7. **Configurer l'environnement de test** :
  - Créez un dossier `integration_test/` à la racine de votre projet.
  - Ajoutez un fichier de test, par exemple `app_test.dart`.

### Écrire un test d'intégration :

```
// integration_test/app_test.dart
```

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:integration_test/integration_test.dart';
import 'package:your_app/main.dart' as app;

void main() {
 IntegrationTestWidgetsFlutterBinding.ensureInitialized();

 testWidgets('Naviguer de la page d\'accueil à la page de détails',
 (WidgetTester tester) async {
 app.main();
 await tester.pumpAndSettle();

 // Vérifier la présence de la page d'accueil
 expect(find.text('Accueil'), findsOneWidget);

 // Cliquer sur un bouton pour naviguer
 await tester.tap(find.text('Aller à détail'));
 await tester.pumpAndSettle();

 // Vérifier la présence de la page de détails
 expect(find.text('Page de Détail'), findsOneWidget);
 });

 testWidgets('Ajouter une tâche et vérifier la liste', (WidgetTester
 tester) async {
 app.main();
 await tester.pumpAndSettle();

 // Naviguer vers la page de gestion de tâches
 await tester.tap(find.text('Gestion de Tâches'));
 await tester.pumpAndSettle();

 // Taper du texte dans le champ de saisie
 await tester.enterText(find.byType(TextField), 'Nouvelle Tâche');
 await tester.tap(find.byIcon(Icons.add));
 await tester.pumpAndSettle();

 // Vérifier que la tâche est ajoutée
 expect(find.text('Nouvelle Tâche'), findsOneWidget);
 });
}
```

### Exécuter les tests d'intégration :

```
flutter test integration_test/app_test.dart
```

### Automatisation des tests :

- **CI/CD** : Intégrez les tests d'intégration dans votre pipeline CI/CD pour exécuter automatiquement les tests lors des commits ou des déploiements.
- **GitHub Actions** : Utilisez des workflows comme **GitHub Actions** pour automatiser l'exécution des tests.

### Exemple de configuration GitHub Actions pour les tests d'intégration :

```
.github/workflows/flutter.yml
name: Flutter CI

on: [push, pull_request]

jobs:
 build:
 runs-on: ubuntu-latest

 steps:
 - uses: actions/checkout@v2
 - uses: subosito/flutter-action@v2
 with:
 flutter-version: '3.0.0'

 - name: Install dependencies
 run: flutter pub get

 - name: Run unit tests
 run: flutter test

 - name: Run integration tests
 run: flutter test integration_test/app_test.dart
```

### Bonnes pratiques :

- **Couvrir les scénarios critiques** : Assurez-vous que les tests d'intégration couvrent les fonctionnalités essentielles de l'application.
- **Maintenir les tests à jour** : Mettez à jour les tests en même temps que le code pour éviter les échecs de tests dus à des changements de fonctionnalités.
- **Optimiser les temps de test** : Écrivez des tests efficaces pour minimiser les temps d'exécution tout en garantissant une couverture complète.

## Leçon 4 : Debugging et résolution des problèmes courants

**Introduction au Debugging** : Le **debugging** est une étape essentielle du développement qui permet d'identifier et de corriger les bugs dans l'application. Flutter offre divers outils et techniques pour faciliter le processus de debugging.

### Outils de debugging dans Flutter :

#### 1. Flutter DevTools :

- Un ensemble d'outils de développement intégré qui offre des fonctionnalités de profiling, inspection des widgets, et analyse des performances.
- **Accès** : Exécutez `flutter pub global activate devtools` puis `flutter pub global run devtools`.

#### 2. Print Statements :

- Utilisez des **print** pour afficher des informations dans la console.
- `print('Valeur de la variable : $_variable');`
- 3. **Assertions :**
  - Utilisez des **assert** pour vérifier les conditions pendant le développement.
  - `assert(_variable != null, 'La variable ne doit pas être nulle');`
- 4. **Breakpoints :**
  - Utilisez les breakpoints dans votre IDE (VS Code, Android Studio) pour suspendre l'exécution et inspecter l'état de l'application.

### Techniques de debugging :

1. **Inspecter la hiérarchie des widgets :**
  - Utilisez le **Widget Inspector** dans Flutter DevTools pour visualiser et inspecter l'arborescence des widgets.
  - **Fonctionnalités :** Sélectionner des widgets, inspecter leurs propriétés, naviguer dans la hiérarchie.
2. **Analyser les logs :**
  - Utilisez la console pour suivre les logs et identifier les erreurs ou les comportements inattendus.
  - `print('Erreur : $e');`
3. **Profilage des performances :**
  - Utilisez les outils de **profilage** pour identifier les goulots d'étranglement et optimiser les performances de l'application.
  - **Fonctionnalités :** Analyse des frames, mesure des temps de rendu, surveillance de l'utilisation de la mémoire.
4. **Gérer les exceptions :**
  - Utilisez des blocs **try-catch** pour capturer et gérer les exceptions.
  - ```
try {  
  // Code susceptible de générer une exception  
} catch (e) {  
  print('Exception capturée : $e');
```
 - }

Résolution des problèmes courants :

1. **Erreurs de compilation :**
 - **Messages d'erreur clairs :** Lisez attentivement les messages d'erreur pour comprendre la cause.
 - **Recherchez les erreurs :** Utilisez les ressources en ligne comme Stack Overflow pour trouver des solutions.
2. **Problèmes de performance :**
 - **Identifiez les widgets lourds :** Utilisez Flutter DevTools pour trouver les widgets qui consomment le plus de ressources.
 - **Optimisez les images :** Compressez et redimensionnez les images pour réduire la charge de rendu.
3. **Problèmes de navigation :**

- **Vérifiez les routes** : Assurez-vous que les routes sont correctement définies et que les arguments sont passés correctement.
 - **Gérez les erreurs de navigation** : Implémentez des pages 404 pour gérer les routes inconnues.
4. **Problèmes d'état** :
- **Gérez correctement l'état** : Utilisez des solutions de gestion d'état appropriées et évitez les états redondants.
 - **Debuguez l'état** : Utilisez des outils comme Provider DevTools ou Riverpod DevTools pour inspecter l'état de l'application.

Bonnes pratiques :

- **Adopter une approche méthodique** : Isoler les problèmes en testant les différentes parties de l'application.
- **Documenter les bugs** : Tenez un journal des bugs rencontrés et des solutions appliquées pour faciliter les futures résolutions.
- **Utiliser le versionnage** : Utilisez le contrôle de version (Git) pour suivre les changements et revenir à des versions antérieures si nécessaire.
- **Collaborer avec l'équipe** : Partagez les problèmes et les solutions avec votre équipe pour améliorer la qualité globale du code.

Leçon 5 : Préparation pour le déploiement sur le Play Store et l'App Store

Introduction au déploiement des applications Flutter : Le déploiement de l'application sur les stores (Play Store pour Android et App Store pour iOS) est une étape cruciale pour rendre l'application accessible aux utilisateurs. Cette leçon couvre les étapes nécessaires pour préparer et publier une application Flutter sur ces plateformes.

Préparation pour Android (Play Store) :

1. **Configurer le projet pour la release** :
 - Modifiez le fichier **android/app/build.gradle** pour définir les versions de la version.
 - ```
defaultConfig {
 applicationId "com.example.votre_app"
 minSdkVersion 21
 targetSdkVersion 33
 versionCode 1
 versionName "1.0.0"
}
```
2. **Générer une clé de signature** :
  - Créez un fichier de clé **keystore** :  

```
keytool -genkey -v -keystore ~/votre_app_keystore.jks -
alias votre_alias -keyalg RSA -keysize 2048 -validity 10000
```
  - Ajoutez la configuration de la signature dans **android/app/build.gradle** :  

```
android {
 ...
```

```
o signingConfigs {
o release {
o keyAlias 'votre_alias'
o keyPassword 'votre_mot_de_passe'
o storeFile file('/chemin/vers/votre_app_keystore.jks')
o storePassword 'votre_mot_de_passe_store'
o }
o }
o buildTypes {
o release {
o signingConfig signingConfigs.release
o minifyEnabled false
o shrinkResources false
o proguardFiles getDefaultProguardFile('proguard-
android-optimize.txt'), 'proguard-rules.pro'
o }
o }
o }
```

### 3. Construire l'APK ou l'AAB :

- o **APK :**
- o `flutter build apk --release`
- o **AAB (recommandé pour le Play Store) :**
- o `flutter build appbundle --release`

### 4. Créer une fiche Play Store :

- o **Titre, description, captures d'écran, icône de l'application, vidéo de présentation, etc.**

### 5. Uploader l'application sur le Play Store :

- o Accédez à la [Google Play Console](https://play.google.com/console).
- o Créez un nouveau projet ou utilisez un projet existant.
- o Téléversez l'AAB ou l'APK, remplissez les informations requises et soumettez pour révision.

## Préparation pour iOS (App Store) :

### 1. Configurer le projet pour la release :

- o Ouvrez le projet iOS dans **Xcode**.
- o Configurez les paramètres de version et de build dans **Runner > General**.

### 2. Générer un certificat et un profil de provisionnement :

- o Utilisez le **Apple Developer Portal** pour créer un certificat de distribution et un profil de provisionnement.

### 3. Construire l'application pour iOS :

#### 4. `flutter build ios --release`

- o Ouvrez le projet iOS dans **Xcode** et archivez l'application (**Product > Archive**).

### 5. Créer une fiche App Store :

- o **Titre, description, captures d'écran, icône de l'application, vidéo de présentation, etc.**

### 6. Uploader l'application sur l'App Store :

- o Utilisez **Xcode** ou **Transporter** pour téléverser l'archive.

- Accédez à l'[App Store Connect](#), créez un nouveau build, remplissez les informations requises et soumettez pour révision.

### Optimisation pour le déploiement :

- **Réduction de la taille de l'application** : Supprimez les ressources inutilisées et optimisez les images.
- **Testez l'application sur différents appareils** : Assurez-vous de la compatibilité et des performances sur diverses tailles d'écran et configurations matérielles.
- **Implémentez des fonctionnalités spécifiques aux plateformes** : Par exemple, des icônes adaptées, des paramètres de permissions, etc.

### Bonnes pratiques :

- **Vérifiez les exigences des stores** : Assurez-vous que votre application respecte les [politiques du Play Store](#) et de l'[App Store](#).
- **Implémentez des analyses et des crash reports** : Utilisez des outils comme **Firebase Analytics** et **Firebase Crashlytics** pour surveiller les performances et les erreurs après le déploiement.
- **Communiquez avec les utilisateurs** : Fournissez des notes de mise à jour claires et répondez aux feedbacks pour améliorer l'application continuellement.

## Leçon 6 : Bonnes pratiques de maintenance et mises à jour d'application

**Introduction à la maintenance des applications** : La maintenance continue est essentielle pour assurer la stabilité, la sécurité et l'amélioration des fonctionnalités de l'application. Cette leçon couvre les bonnes pratiques pour maintenir et mettre à jour une application Flutter.

### Planification des mises à jour :

- **Roadmap produit** : Établissez une feuille de route claire pour les futures fonctionnalités et les mises à jour.
- **Cycle de release** : Définissez un cycle de release régulier (mensuel, trimestriel) pour planifier et publier les mises à jour.

### Gestion des versions :

- **Versioning sémantique** : Utilisez un système de versionnage sémantique (MAJOR.MINOR.PATCH) pour gérer les versions de l'application.
- `version: 1.0.0+1`
  - **MAJOR** : Changements incompatibles.
  - **MINOR** : Ajout de fonctionnalités compatibles.
  - **PATCH** : Corrections de bugs compatibles.

### Suivi des bugs et des feedbacks :

- **Outils de suivi** : Utilisez des outils comme **GitHub Issues**, **JIRA**, ou **Trello** pour suivre les bugs et les demandes de fonctionnalités.
- **Collecte de feedbacks** : Intégrez des mécanismes pour collecter les feedbacks des utilisateurs, comme des formulaires de feedback ou des systèmes de notation.

### Gestion des dépendances :

- **Mises à jour régulières** : Maintenez les dépendances à jour pour bénéficier des dernières fonctionnalités et des correctifs de sécurité.
- `flutter pub upgrade`
- **Vérification de compatibilité** : Assurez-vous que les mises à jour des packages ne cassent pas les fonctionnalités existantes.

### Refactoring du code :

- **Nettoyez le code** : Supprimez le code mort, simplifiez les structures complexes et améliorez la lisibilité.
- **Modularité** : Organisez le code en modules ou en packages pour faciliter la maintenance et la réutilisation.

### Tests continus :

- **Automatisation des tests** : Intégrez des tests automatisés (unitaires, widgets, intégration) dans votre pipeline CI/CD pour détecter les régressions.
- **Revue de code** : Mettez en place des processus de revue de code pour assurer la qualité et la cohérence du codebase.

### Gestion des performances :

- **Surveillance continue** : Utilisez des outils comme **Firebase Performance Monitoring** pour surveiller les performances en temps réel.
- **Optimisations régulières** : Identifiez et corrigez les problèmes de performance au fur et à mesure qu'ils apparaissent.

### Sécurité continue :

- **Mises à jour de sécurité** : Appliquez rapidement les mises à jour de sécurité pour les dépendances et le code de l'application.
- **Audit de sécurité** : Effectuez des audits réguliers pour identifier et corriger les vulnérabilités potentielles.

### Documentation :

- **Documentation interne** : Maintenez une documentation claire et à jour pour faciliter la compréhension et la maintenance du code.

- **Documentation utilisateur** : Fournissez des guides et des FAQ pour aider les utilisateurs à naviguer et utiliser l'application efficacement.

### Exemple de mise à jour :

```
// Mise à jour de la version dans pubspec.yaml
version: 1.1.0+2

// Ajout d'une nouvelle fonctionnalité
class NouveautePage extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Nouvelle Fonctionnalité'),
),
 body: Center(
 child: Text('Bienvenue dans la nouvelle fonctionnalité!'),
),
);
 }
}

// Mise à jour du routeur
class AppRouter {
 static const String accueil = '/';
 static const String detailTache = '/detailTache';
 static const String nouveaute = '/nouveaute';

 Route<dynamic> onGenerateRoute(RouteSettings settings) {
 switch (settings.name) {
 case accueil:
 return MaterialPageRoute(builder: (context) => Accueil());
 case detailTache:
 final tache = settings.arguments as Tache;
 return MaterialPageRoute(builder: (context) =>
DetailTache(tache: tache));
 case nouveaute:
 return MaterialPageRoute(builder: (context) =>
NouveautePage());
 default:
 return MaterialPageRoute(builder: (context) => Page404());
 }
 }
}

// Ajout du bouton de navigation vers la nouvelle fonctionnalité
class Accueil extends StatelessWidget {
 final List<Tache> _taches = [
 Tache(titre: 'Tâche 1', description: 'Description 1'),
 Tache(titre: 'Tâche 2', description: 'Description 2'),
];

 @override
 Widget build(BuildContext context) {
 return Scaffold(
```

```
appBar: AppBar(
 title: Text('Accueil'),
) ,
body: ListView.builder(
 itemCount: _taches.length,
 itemBuilder: (context, index) {
 final tache = _taches[index];
 return ListTile(
 title: Text(tache.titre),
 onTap: () {
 Navigator.pushNamed(
 context,
 AppRouter.detailTache,
 arguments: tache,
);
 },
),
),
) ,
floatingActionButton: FloatingActionButton(
 onPressed: () {
 Navigator.pushNamed(context, AppRouter.nouveaute);
 },
 child: Icon(Icons.new_releases),
) ,
) ;
}
```

### Bonnes pratiques :

- **Planifiez les mises à jour** pour éviter les interruptions majeures et garantir une transition en douceur pour les utilisateurs.
- **Testez exhaustivement** avant de déployer les mises à jour pour minimiser les risques de bugs.
- **Communiquez clairement** avec les utilisateurs sur les nouvelles fonctionnalités, les corrections de bugs et les améliorations de l'application.

## Leçon 7 : Gestion de la sécurité et des permissions sur les stores

**Introduction à la sécurité des applications :** La sécurité est un aspect crucial du développement d'applications Flutter pour protéger les données des utilisateurs et garantir l'intégrité de l'application. Cette leçon couvre la gestion des permissions et les meilleures pratiques de sécurité pour les applications Flutter avant leur déploiement sur les stores.

### Gestion des permissions :

1. **Identifier les permissions nécessaires :**
  - Déterminez quelles permissions sont nécessaires pour les fonctionnalités de votre application (caméra, localisation, stockage, etc.).

### 2. Déclarer les permissions dans Android :

- Ajoutez les permissions nécessaires dans le fichier **AndroidManifest.xml**.
- `<uses-permission android:name="android.permission.CAMERA"/>`
- `<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>`

### 3. Déclarer les permissions dans iOS :

- Ajoutez les descriptions des permissions dans le fichier **Info.plist**.
- `<key>NSCameraUsageDescription</key>`
- `<string>Cette application nécessite l'accès à la caméra.</string>`
- `<key>NSLocationWhenInUseUsageDescription</key>`
- `<string>Cette application nécessite l'accès à votre localisation.</string>`

### 4. Demander les permissions dynamiquement :

- Utilisez le package **permission\_handler** pour demander et gérer les permissions à l'exécution.
- `import 'package:permission_handler/permission_handler.dart';`
- 
- `Future<void> _demanderPermissions() async {`
- `var statusCamera = await Permission.camera.status;`
- `if (!statusCamera.isGranted) {`
- `await Permission.camera.request();`
- `}`
- 
- `var statusLocation = await Permission.location.status;`
- `if (!statusLocation.isGranted) {`
- `await Permission.location.request();`
- `}`
- `}`

## Sécurisation des données :

### 1. Chiffrement des données sensibles :

- Chiffrez les données sensibles stockées localement avec des packages comme **flutter\_secure\_storage** ou **encrypt**.
- `import 'package:flutter_secure_storage/flutter_secure_storage.dart';`
- `;`
- 
- `final storage = FlutterSecureStorage();`
- 
- `// Sauvegarder une donnée`
- `await storage.write(key: 'token', value: 'votre_token_securise');`
- 
- `// Lire une donnée`
- `String? token = await storage.read(key: 'token');`

### 2. Utilisation de HTTPS :

- Assurez-vous que toutes les communications réseau utilisent le protocole **HTTPS** pour sécuriser les données en transit.

### 3. Protection contre les injections :

- Validez et nettoyez toutes les entrées utilisateur pour prévenir les attaques par injection.
- 4. **Gestion des tokens d'authentification :**
  - Stockez les tokens d'authentification de manière sécurisée et implémentez des mécanismes de rafraîchissement des tokens.

### Meilleures pratiques de sécurité :

1. **Minimiser les permissions :**
  - Demandez uniquement les permissions nécessaires pour les fonctionnalités de l'application afin de réduire les risques de sécurité.
2. **Validation des entrées utilisateur :**
  - Validez toutes les entrées utilisateur pour éviter les attaques par injection et les comportements inattendus.
3. **Obfuscation du code :**
  - Obfuscatez le code Dart pour rendre plus difficile la rétro-ingénierie et la compréhension du code source.
  - `flutter build apk --release --obfuscate --split-debug-info=/<répertoire>`
4. **Mises à jour régulières :**
  - Maintenez les dépendances à jour pour bénéficier des correctifs de sécurité et des améliorations.
5. **Audit de sécurité :**
  - Effectuez régulièrement des audits de sécurité pour identifier et corriger les vulnérabilités potentielles.

### Préparation pour le déploiement sécurisé :

1. **Vérifier les règles de sécurité :**
  - Assurez-vous que les règles de sécurité Firebase (Firestore, Realtime Database, Storage) sont correctement configurées pour restreindre l'accès aux données.
2. **Tester les permissions et la sécurité :**
  - Testez l'application pour vérifier que les permissions sont correctement gérées et que les données sensibles sont sécurisées.
3. **Utiliser des outils de scan de sécurité :**
  - Utilisez des outils comme **Static Code Analysis** pour identifier les vulnérabilités dans le code.

### Exemple de sécurisation des règles Firebase Firestore :

```
rules_version = '2';
service cloud.firestore {
 match /databases/{database}/documents {
 // Règles pour la collection utilisateurs
 match /utilisateurs/{userId} {
 allow read, write: if request.auth != null && request.auth.uid ==
userId;
```

```
}

// Règles par défaut
match /{document=**} {
 allow read, write: if false;
}
}
}
```

### Bonnes pratiques :

- **Respecter les politiques des stores** : Assurez-vous que votre application respecte les [politiques de sécurité de Google Play](#) et de l'[App Store](#).
- **Informers les utilisateurs** : Fournissez des informations claires sur l'utilisation des permissions et des données collectées.
- **Surveillance continue** : Mettez en place des systèmes de surveillance pour détecter et réagir rapidement aux incidents de sécurité.

## Module 7 : Projet Final : Création d'une Application Complète

### Leçon 1 : Planification et structuration de l'application finale

**Introduction à la planification d'un projet Flutter** : La planification est une étape cruciale qui permet de définir les objectifs, les fonctionnalités et la structure de l'application. Une bonne planification facilite le développement, réduit les risques et améliore la qualité du produit final.

#### Étapes de planification :

1. **Définir les objectifs de l'application** :
  - Identifiez le problème que l'application résout.
  - Déterminez les objectifs principaux et secondaires.
2. **Lister les fonctionnalités** :
  - Énumérez toutes les fonctionnalités que l'application doit inclure.
  - Classez les fonctionnalités par priorité (Must-have, Nice-to-have).
3. **Créer des user stories** :
  - Rédigez des descriptions courtes et simples des fonctionnalités du point de vue de l'utilisateur.
  - Exemple : "En tant qu'utilisateur, je veux pouvoir m'inscrire avec mon email pour accéder à l'application."
4. **Définir la structure de l'application** :
  - Organisez l'application en modules ou en sections logiques.
  - Déterminez la navigation entre les différentes sections.
5. **Établir un calendrier de développement** :
  - Définissez des échéances pour chaque phase du développement.

- Allouez des ressources en fonction des priorités et des compétences de l'équipe.

### Création d'un diagramme de flux :

- Utilisez des outils comme **Figma**, **Draw.io**, ou **Lucidchart** pour créer des diagrammes de flux illustrant la navigation et les interactions dans l'application.

### Exemple de structure de projet :

```
lib/
├── main.dart
├── models/
│ ├── utilisateur.dart
│ └── tache.dart
├── views/
│ ├── accueil.dart
│ ├── detail_tache.dart
│ ├── formulaire_tache.dart
│ └── profil.dart
├── controllers/
│ └── tache_controller.dart
├── services/
│ ├── api_service.dart
│ └── auth_service.dart
├── providers/
│ ├── compteur_provider.dart
│ └── utilisateur_provider.dart
├── widgets/
│ ├── header.dart
│ └── custom_button.dart
└── utils/
 ├── constantes.dart
 └── helpers.dart
```

### Bonnes pratiques :

- **Clarté et simplicité** : Gardez la structure du projet claire et simple pour faciliter la navigation et la maintenance.
- **Modularité** : Organisez le code en modules réutilisables pour améliorer la maintenabilité et la scalabilité.
- **Documentation** : Documentez la structure et les composants clés de l'application pour faciliter la collaboration et la compréhension du code.

## Leçon 2 : Définition de la charte graphique et des maquettes

**Introduction à la conception UI/UX** : La charte graphique et les maquettes définissent l'apparence et l'expérience utilisateur de l'application. Une bonne conception UI/UX améliore l'engagement des utilisateurs et facilite la navigation dans l'application.

### Définir la charte graphique :

1. **Palette de couleurs :**
  - Choisissez une palette de couleurs cohérente qui reflète l'identité de l'application.
  - Utilisez des outils comme **Adobe Color** ou **Coolors** pour créer des palettes harmonieuses.
2. **Typographie :**
  - Sélectionnez des polices de caractères lisibles et adaptées à l'application.
  - Définissez des styles typographiques pour les titres, les sous-titres, le corps du texte, etc.
3. **Iconographie :**
  - Utilisez des icônes cohérentes et intuitives pour améliorer la navigation et l'interaction.
  - Utilisez des packs d'icônes comme **Material Icons** ou **FontAwesome**.
4. **Éléments de design :**
  - Définissez les styles des boutons, des champs de texte, des cartes, etc.
  - Assurez-vous que tous les éléments respectent la charte graphique pour une apparence uniforme.

### Création des maquettes :

1. **Wireframes :**
  - Créez des wireframes pour définir la structure de chaque écran sans se concentrer sur les détails visuels.
  - Utilisez des outils comme **Figma**, **Sketch**, ou **Adobe XD**.
2. **Maquettes haute-fidélité :**
  - Développez des maquettes détaillées en incorporant la charte graphique.
  - Incluez des éléments interactifs pour simuler l'expérience utilisateur.
3. **Prototypage :**
  - Créez des prototypes interactifs pour tester la navigation et les interactions avant de développer l'application.
  - Utilisez des outils comme **Figma** ou **InVision** pour créer des prototypes interactifs.

### Exemple de maquette avec Figma :

- **Accueil :** Affiche la liste des tâches avec un bouton d'ajout.
- **Détail de la tâche :** Affiche les détails de la tâche sélectionnée avec des options de modification et de suppression.
- **Formulaire de tâche :** Formulaire pour ajouter ou modifier une tâche.
- **Profil utilisateur :** Affiche les informations de l'utilisateur avec des options de déconnexion.

### Bonnes pratiques :

- **Consistance :** Assurez-vous que tous les écrans et éléments suivent la même charte graphique pour une expérience utilisateur cohérente.

- ### con 3 : Mise en place de la navigation et de la gestion de l'état

## Introduction à la navigation et à la gestion de l'état : La navigation et la gestion de

### Configuration de la navigation :

- Déclarez les routes dans le fichier **MaterialApp** pour gérer la navigation entre les différentes pages.

```

class AppRouter {
 static const String accueil = '/';
 static const String detailTache = '/detailTache';
 static const String formulaireTache = '/formulaireTache';
 static const String profil = '/profil';

 Route<dynamic> onGenerateRoute(RouteSettings settings) {
 switch (settings.name) {
 case accueil:
 return MaterialPageRoute(builder: (context) =>
 Accueil());
 case detailTache:
 final tache = settings.arguments as Tache;
 return MaterialPageRoute(builder: (context) =>
 DetailTache(tache: tache));
 case formulaireTache:
 return MaterialPageRoute(builder: (context) =>
 FormulaireTache());
 case profil:
 return MaterialPageRoute(builder: (context) =>
 ProfilPage());
 default:
 return MaterialPageRoute(builder: (context) =>
 Page404());
 }
 }
}

void main() {
 runApp(MyApp());
}

class MyApp extends StatelessWidget {
 final AppRouter _appRouter = AppRouter();

 @override
 Widget build(BuildContext context) {

```

```
o return MaterialApp(
o title: 'Gestion de Tâches',
o onGenerateRoute: _appRouter.onGenerateRoute,
o initialRoute: AppRouter.accueil,
o);
o }
o }
```

### 2. Naviguer vers une route nommée :

```
3. Navigator.pushNamed(
4. context,
5. AppRouter.detailTache,
6. arguments: tacheInstance,
7.);
```

### 8. Naviguer avec des transitions personnalisées :

```
9. Navigator.push(
10. context,
11. PageRouteBuilder(
12. pageBuilder: (context, animation, secondaryAnimation) =>
 DetailTache(tache: tacheInstance),
13. transitionsBuilder: (context, animation, secondaryAnimation,
 child) {
14. var begin = Offset(0.0, 1.0);
15. var end = Offset.zero;
16. var curve = Curves.ease;
17.
18. var tween = Tween(begin: begin, end:
 end).chain(CurveTween(curve: curve));
19. var offsetAnimation = animation.drive(tween);
20.
21. return SlideTransition(
22. position: offsetAnimation,
23. child: child,
24.);
25. },
26.),
27.);
```

## Gestion de l'état avec Provider :

### 1. Créer un provider :

```
2. import 'package:flutter/material.dart';
3. import 'package:provider/provider.dart';
4.
5. class UtilisateurProvider with ChangeNotifier {
6. Utilisateur? _utilisateur;
7.
8. Utilisateur? get utilisateur => _utilisateur;
9.
10. void setUtilisateur(Utilisateur utilisateur) {
11. _utilisateur = utilisateur;
12. notifyListeners();
13. }
14.
15. void deconnecter() {
16. _utilisateur = null;
```

```
17. notifyListeners();
18. }
19. }
```

### 20. Fournir le provider dans l'arborescence des widgets :

```
21. void main() {
22. runApp(
23. ChangeNotifierProvider(
24. create: (_) => UtilisateurProvider(),
25. child: MyApp(),
26.),
27.);
28. }
```

### 29. Consommer le provider dans les widgets :

```
30. import 'package:flutter/material.dart';
31. import 'package:provider/provider.dart';
32.
33. class ProfilPage extends StatelessWidget {
34. @override
35. Widget build(BuildContext context) {
36. final utilisateur =
37. Provider.of<UtilisateurProvider>(context).utilisateur;
38.
39. return Scaffold(
40. appBar: AppBar(
41. title: Text('Profil'),
42.),
43. body: utilisateur == null
44. ? Center(child: Text('Utilisateur non connecté'))
45. : Center(
46. child: Column(
47. mainAxisAlignment: MainAxisAlignment.center,
48. children: [
49. Text('Nom: ${utilisateur.nom}'),
50. Text('Email: ${utilisateur.email}'),
51. ElevatedButton(
52. onPressed: () {
53. Provider.of<UtilisateurProvider>(context,
54. listen: false).deconnecter();
55. Navigator.pushNamedAndRemoveUntil(context,
56. AppRouter.accueil, (route) => false);
57. },
58. child: Text('Se déconnecter'),
59.),
60.],
61.),
62.);
63. }
64. }
```

### Gestion de l'état avec Riverpod :

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

final utilisateurProvider = StateProvider<Utilisateur?>((ref) => null);
```

```
class ProfilRiverpodPage extends ConsumerWidget {
 @override
 Widget build(BuildContext context, WidgetRef ref) {
 final utilisateur = ref.watch(utilisateurProvider).state;

 return Scaffold(
 appBar: AppBar(
 title: Text('Profil Riverpod'),
),
 body: utilisateur == null
 ? Center(child: Text('Utilisateur non connecté'))
 : Center(
 child: Column(
 mainAxisAlignment: MainAxisAlignment.center,
 children: [
 Text('Nom: ${utilisateur.nom}'),
 Text('Email: ${utilisateur.email}'),
 ElevatedButton(
 onPressed: () {
 ref.read(utilisateurProvider).state = null;
 Navigator.pushNamedAndRemoveUntil(context,
AppRouter.accueil, (route) => false);
 },
 child: Text('Se déconnecter'),
),
],
),
);
 }
}
```

### Bonnes pratiques :

- **Séparation claire** : Séparez la logique de navigation de la logique de gestion d'état pour une meilleure maintenabilité.
- **Utiliser des providers appropriés** : Choisissez la solution de gestion d'état qui correspond le mieux aux besoins de votre application (Provider, Riverpod, BLoC, etc.).
- **Gérer les états globaux et locaux** : Utilisez des solutions de gestion d'état pour les états partagés et gardez les états locaux proches des widgets qui en dépendent.

## Leçon 4 : Connexion avec une base de données et des API

**Introduction à la connexion avec des bases de données et des APIs** : La connexion avec des bases de données et des APIs permet d'obtenir, de stocker et de manipuler des données dynamiques dans une application Flutter. Cette leçon couvre l'intégration des bases de données (locales et cloud) et l'utilisation des APIs REST pour interagir avec des services externes.

### Connexion avec Cloud Firestore :

## Développement d'Applications Flutter

```
import 'package:flutter/material.dart';
import 'package:cloud_firestore/cloud_firestore.dart';

class FirestoreConnectionPage extends StatelessWidget {
 final FirebaseFirestore _firestore = FirebaseFirestore.instance;

 Future<void> ajouterUtilisateur(String nom, String email) async {
 await _firestore.collection('utilisateurs').add({
 'nom': nom,
 'email': email,
 'timestamp': FieldValue.serverTimestamp(),
 });
 }

 Stream<List<Utilisateur>> getUtilisateurs() {
 return _firestore.collection('utilisateurs').orderBy('timestamp',
 descending: true).snapshots().map((snapshot) =>
 snapshot.docs.map((doc) =>
 Utilisateur.fromFirestore(doc)).toList());
 }

 @override
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Connexion Firestore'),
),
 body: Column(
 children: [
 Expanded(
 child: StreamBuilder<List<Utilisateur>>(
 stream: getUtilisateurs(),
 builder: (context, snapshot) {
 if (snapshot.hasError) {
 return Center(child: Text('Erreur :
${snapshot.error}'));
 }
 if (!snapshot.hasData) {
 return Center(child: CircularProgressIndicator());
 }
 final utilisateurs = snapshot.data!;
 return ListView.builder(
 itemCount: utilisateurs.length,
 itemBuilder: (context, index) {
 final utilisateur = utilisateurs[index];
 return ListTile(
 title: Text(utilisateur.nom),
 subtitle: Text(utilisateur.email),
);
 },
);
),
),
),
 Padding(
 padding: const EdgeInsets.all(16.0),
 child: ElevatedButton(
```

```
 onPressed: () {
 ajouterUtilisateur('Utilisateur Firestore',
'firestore@example.com');
 },
 child: Text('Ajouter Utilisateur'),
),
),
],
),
);
}
}

class Utilisateur {
 final String id;
 final String nom;
 final String email;

 Utilisateur({required this.id, required this.nom, required
this.email});

 factory Utilisateur.fromFirestore(DocumentSnapshot doc) {
 Map data = doc.data() as Map<String, dynamic>;
 return Utilisateur(
 id: doc.id,
 nom: data['nom'] ?? '',
 email: data['email'] ?? '',
);
 }
}
```

### Connexion avec une API REST :

```
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

class ApiConnectionPage extends StatefulWidget {
 @override
 _ApiConnectionPageState createState() => _ApiConnectionPageState();
}

class _ApiConnectionPageState extends State<ApiConnectionPage> {
 List<dynamic> _data = [];
 bool _isLoading = false;
 String? _erreur;

 Future<void> _fetchData() async {
 setState(() {
 _isLoading = true;
 _erreur = null;
 });

 try {
 final response = await
http.get(Uri.parse('https://api.example.com/data'));
 }
 }
}
```

```
if (response.statusCode == 200) {
 setState(() {
 _data = json.decode(response.body);
 });
} else {
 setState(() {
 _erreur = 'Erreur : ${response.statusCode}';
 });
}
} catch (e) {
 setState(() {
 _erreur = 'Exception : $e';
 });
} finally {
 setState(() {
 _isLoading = false;
 });
}
}

@override
void initState() {
 super.initState();
 _fetchData();
}

@override
Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Connexion API REST'),
),
 body: _isLoading
 ? Center(child: CircularProgressIndicator())
 : _erreur != null
 ? Center(child: Text(_erreur!))
 : ListView.builder(
 itemCount: _data.length,
 itemBuilder: (context, index) {
 final item = _data[index];
 return ListTile(
 title: Text(item['titre']),
 subtitle: Text(item['description']),
);
 },
);
}
}
```

### Gestion des états asynchrones :

- **Utilisation des solutions de gestion d'état** (Provider, Riverpod, BLoC) pour gérer les données asynchrones et les mises à jour de l'interface utilisateur.

- **Utilisation des FutureBuilder et StreamBuilder** pour intégrer les données asynchrones directement dans l'interface utilisateur.

### Exemple avec FutureBuilder :

```
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

class FutureBuilderPage extends StatelessWidget {
 Future<List<dynamic>> fetchData() async {
 final response = await
http.get(Uri.parse('https://api.example.com/data'));
 if (response.statusCode == 200) {
 return json.decode(response.body);
 } else {
 throw Exception('Erreur lors du chargement des données');
 }
 }

 @override
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('FutureBuilder Exemple'),
),
 body: FutureBuilder<List<dynamic>>(
 future: fetchData(),
 builder: (context, snapshot) {
 if (snapshot.connectionState == ConnectionState.waiting) {
 return Center(child: CircularProgressIndicator());
 } else if (snapshot.hasError) {
 return Center(child: Text('Erreur : ${snapshot.error}'));
 } else if (!snapshot.hasData || snapshot.data!.isEmpty) {
 return Center(child: Text('Aucune donnée disponible'));
 } else {
 final data = snapshot.data!;
 return ListView.builder(
 itemCount: data.length,
 itemBuilder: (context, index) {
 final item = data[index];
 return ListTile(
 title: Text(item['titre']),
 subtitle: Text(item['description']),
);
 },
);
 }
 },
),
);
 }
}
```

### Bonnes pratiques :

- **Séparer la logique de la présentation** : Gardez la logique de connexion et de traitement des données séparée des widgets pour une meilleure maintenabilité.
- **Gestion robuste des erreurs** : Gérez toutes les exceptions et les erreurs de manière appropriée pour améliorer l'expérience utilisateur.
- **Optimisation des requêtes** : Limitez la quantité de données récupérées et optimisez les requêtes pour améliorer les performances.

### Leçon 5 : Intégration des fonctionnalités avancées (authentification, notifications, etc.)

**Introduction aux fonctionnalités avancées** : Les fonctionnalités avancées enrichissent l'application en offrant des interactions plus sophistiquées et des services supplémentaires. Cette leçon couvre l'intégration de l'authentification, des notifications, et d'autres fonctionnalités avancées.

#### Intégration de l'authentification :

- **Se référer à la Leçon 4 du Module 3** pour les détails sur l'implémentation de l'authentification avec Email, Google, et Facebook.

#### Intégration des notifications push :

- **Se référer à la Leçon 2 du Module 5** pour les détails sur l'implémentation des notifications push avec Firebase Cloud Messaging.

#### Utilisation de Firebase Crashlytics :

```
dependencies:
 firebase_crashlytics: ^3.0.0
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_crashlytics/firebase_crashlytics.dart';

void main() async {
 WidgetsFlutterBinding.ensureInitialized();
 await Firebase.initializeApp();

 FlutterError.onError =
 FirebaseCrashlytics.instance.recordFlutterFatalError;

 runApp(MyApp());
}

class MyApp extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
 return MaterialApp(
 title: 'Crashlytics Exemple',
 home: CrashlyticsPage(),
);
 }
}
```

```
 }
 }

class CrashlyticsPage extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Crashlytics Exemple'),
),
 body: Center(
 child: ElevatedButton(
 onPressed: () {
 // Déclencher un crash pour tester Crashlytics
 throw StateError('Test Crashlytics');
 },
 child: Text('Déclencher un crash'),
),
),
);
 }
}
```

### Intégration de l'analyse des performances :

```
dependencies:
 firebase_performance: ^1.0.0
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_performance/firebase_performance.dart';

void main() async {
 WidgetsFlutterBinding.ensureInitialized();
 await Firebase.initializeApp();
 runApp(MyApp());
}

class MyApp extends StatelessWidget {
 final FirebasePerformance _performance =
 FirebasePerformance.instance;

 @override
 Widget build(BuildContext context) {
 return MaterialApp(
 title: 'Performance Exemple',
 home: PerformancePage(),
);
 }
}

class PerformancePage extends StatelessWidget {
 final FirebasePerformance _performance =
 FirebasePerformance.instance;

 Future<void> _mesurerPerformance() async {
 Trace trace = _performance.newTrace('test_trace');
```

```
 trace.start();
 await Future.delayed(Duration(seconds: 2));
 trace.stop();
 }

 @override
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Performance Exemple'),
),
 body: Center(
 child: ElevatedButton(
 onPressed: _mesurerPerformance,
 child: Text('Mesurer la performance'),
),
),
);
 }
}
```

### Intégration de la géolocalisation avancée :

```
import 'package:flutter/material.dart';
import 'package:geolocator/geolocator.dart';
import 'package:google_maps_flutter/google_maps_flutter.dart';

class GeolocalisationPage extends StatefulWidget {
 @override
 _GeolocalisationPageState createState() =>
 _GeolocalisationPageState();
}

class _GeolocalisationPageState extends State<GeolocalisationPage> {
 Position? _position;
 late GoogleMapController _mapController;
 final LatLng _initialPosition = LatLng(48.8566, 2.3522); // Paris

 Future<void> _obtenirPosition() async {
 Position position = await
 Geolocator.getCurrentPosition(desiredAccuracy: LocationAccuracy.high);
 setState(() {
 _position = position;
 });
 _mapController.animateCamera(
 CameraUpdate.newLatLng(LatLng(position.latitude,
 position.longitude)),
);
 }

 void _onMapCreated(GoogleMapController controller) {
 _mapController = controller;
 }

 @override
 void initState() {
```

```
super.initState();
_obtenirPosition();
}

@override
Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Géolocalisation Avancée'),
),
 body: GoogleMap(
 onMapCreated: _onMapCreated,
 initialCameraPosition: CameraPosition(
 target: _initialPosition,
 zoom: 12,
),
 markers: _position == null
 ? {}
 : {
 Marker(
 markerId: MarkerId('position_actuelle'),
 position: LatLng(_position!.latitude,
 _position!.longitude),
 infoWindow: InfoWindow(title: 'Votre Position'),
),
 },
));
}
```

### Bonnes pratiques :

- **Utiliser des services appropriés** : Intégrez des services tiers de manière sécurisée et optimisée.
- **Optimisation des performances** : Surveillez et optimisez l'utilisation des ressources pour éviter les ralentissements et les fuites de mémoire.
- **Gestion des permissions** : Demandez et gérez les permissions de manière appropriée pour respecter la vie privée des utilisateurs.

## Leçon 6 : Tests et débogage complet de l'application

**Introduction aux tests et débogage complets** : Un débogage et des tests complets garantissent que l'application fonctionne comme prévu dans divers scénarios et configurations. Cette leçon couvre les stratégies avancées pour tester et déboguer une application Flutter de manière exhaustive.

### Stratégies de tests avancées :

1. **Tests unitaires** :
  - Couvrez toutes les fonctions et méthodes critiques.

- Utilisez des **mocks** pour isoler les unités de code des dépendances externes.
- 2. **Tests widgets :**
  - Testez l'interaction des utilisateurs avec les widgets.
  - Utilisez des **tester extensions** pour simuler des gestes complexes.
- 3. **Tests d'intégration :**
  - Testez l'application entière dans un environnement simulé.
  - Utilisez des outils comme **Flutter Driver** ou **Integration Test**.

### Exemple de test widget avancé :

```
// test/widgets/formulaire_tache_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:provider/provider.dart';
import 'package:your_app/formulaire_tache.dart'; // Remplacez par votre
fichier réel
import 'package:your_app/providers/tache_provider.dart'; // Remplacez
par votre provider réel

class MockTacheProvider with ChangeNotifier implements TacheProvider {
 List<Tache> _taches = [];

 @override
 List<Tache> get taches => _taches;

 @override
 Future<void> ajouterTache(String titre, String description) async {
 _taches.add(Tache(titre: titre, description: description));
 notifyListeners();
 }
}

void main() {
 testWidgets('Formulaire de tâche ajoute une nouvelle tâche',
 (WidgetTester tester) async {
 await tester.pumpWidget(
 ChangeNotifierProvider<TacheProvider>(
 create: (_) => MockTacheProvider(),
 child: MaterialApp(home: FormulaireTache()),
),
);

 // Taper du texte dans les champs
 await tester.enterText(find.byType(TextField).at(0), 'Nouvelle
 Tâche');
 await tester.enterText(find.byType(TextField).at(1), 'Description
 de la tâche');

 // Cliquer sur le bouton d'ajout
 await tester.tap(find.byType(ElevatedButton));
 await tester.pumpAndSettle();

 // Vérifier que la tâche a été ajoutée
```

```
final provider =
tester.state<ChangeNotifierProviderState<TacheProvider>>(find.byType(ChangeNotifierProvider<TacheProvider>)).notifier!;
 expect(provider.taches.length, 1);
 expect(provider.taches.first.titre, 'Nouvelle Tâche');
});
}
```

### Techniques de débogage avancées :

#### 1. Debugger dans l'IDE :

- Utilisez les breakpoints, les watches, et les inspections d'état pour analyser le comportement de l'application en temps réel.

#### 2. Utilisation des logs avancés :

- Intégrez des logs détaillés pour suivre le flux de l'application et identifier les problèmes.
- `import 'package:logger/logger.dart';`
- `var logger = Logger();`
- `logger.d('Données chargées : $_data');`
- `logger.e('Erreur lors du chargement', e);`

#### 3. Analyse des performances :

- Utilisez Flutter DevTools pour identifier les problèmes de performance comme les frames lentes, les fuites de mémoire, et les reconstructions excessives.

#### 4. Gestion des exceptions globales :

- Capturez les exceptions non gérées pour éviter les plantages de l'application et enregistrer les erreurs.
- `FlutterError.onError =`  
`FlutterError.onError = FirebaseCrashlytics.instance.recordFlutterFatalError;`
- `runZonedGuarded(() {`  
 `runApp(MyApp());`  
`}, FirebaseCrashlytics.instance.recordError);`

### Optimisation des builds :

- **Minification et obfuscation** : Réduisez la taille de l'application et protégez le code source.
- `flutter build apk --release --obfuscate --split-debug-info=/<répertoire>`
- `flutter build ios --release --obfuscate --split-debug-info=/<répertoire>`

### Bonnes pratiques :

- **Testez sur plusieurs appareils** : Assurez-vous que l'application fonctionne correctement sur différentes tailles d'écran et configurations matérielles.

- **Automatisez les tests** : Intégrez les tests dans un pipeline CI/CD pour détecter les régressions rapidement.
- **Documentez les bugs** : Tenez un journal des bugs rencontrés et des solutions appliquées pour faciliter la maintenance future.
- **Suivi des performances** : Surveillez continuellement les performances de l'application et optimisez-les en conséquence.

### Leçon 7 : Optimisation, finalisation et publication de l'application

**Introduction à l'optimisation et à la finalisation** : Avant de publier une application, il est essentiel d'optimiser le code, de tester exhaustivement toutes les fonctionnalités, et de s'assurer que l'application est prête pour les utilisateurs. Cette leçon couvre les étapes finales avant la publication de l'application sur les stores.

#### Optimisation finale du code :

1. **Révision du code** :
  - Passez en revue le code pour identifier et supprimer le code mort, améliorer la lisibilité, et optimiser les performances.
  - Utilisez des outils comme **Dart Analyzer** pour détecter les problèmes potentiels.
2. **Minification et obfuscation** :
  - Réduisez la taille de l'application et protégez le code source en utilisant la minification et l'obfuscation.
  - `flutter build apk --release --obfuscate --split-debug-info=/<répertoire>`
  - `flutter build ios --release --obfuscate --split-debug-info=/<répertoire>`
3. **Optimisation des assets** :
  - Compressez et optimisez toutes les images, vidéos et autres fichiers médias pour réduire la taille de l'application.
  - Utilisez des outils comme **ImageOptim** ou **TinyPNG** pour compresser les images.
4. **Gestion des dépendances** :
  - Supprimez les dépendances inutilisées et assurez-vous que toutes les dépendances sont à jour.
  - Utilisez `flutter pub outdated` pour vérifier les mises à jour disponibles.

#### Finalisation des tests :

1. **Tests complets** :
  - Exécutez tous les tests unitaires, widgets et d'intégration pour vous assurer que l'application fonctionne correctement.
  - Corrigez tous les bugs détectés pendant les tests.
2. **Tests sur des appareils réels** :

- Testez l'application sur plusieurs appareils physiques pour vérifier la compatibilité et les performances réelles.
  - Utilisez des plateformes comme **Firebase Test Lab** pour tester sur une large gamme d'appareils.
3. **Vérification des fonctionnalités clés :**
- Assurez-vous que toutes les fonctionnalités principales de l'application fonctionnent comme prévu.
  - Effectuez des tests manuels pour vérifier les scénarios complexes et les interactions utilisateur.

### Préparation pour le déploiement :

1. **Configurer les métadonnées de l'application :**
  - Mettez à jour les informations de l'application dans le fichier `pubspec.yaml` (nom, description, version, etc.).
  - `name: votre_app`
  - `description: Une application Flutter complète.`
  - `version: 1.0.0+1`
2. **Créer des icônes et des splash screens :**
  - Utilisez des outils comme **Flutter Launcher Icons** pour générer les icônes de l'application.
  - `dev_dependencies:`
  - `flutter_launcher_icons: ^0.9.2`
  - 
  - `flutter_icons:`
  - `android: true`
  - `ios: true`
  - `image_path: "assets/icon.png"`
  - `flutter pub run flutter_launcher_icons:main`
  - Créez des splash screens adaptés pour Android et iOS.
3. **Configurer les paramètres de build :**
  - Assurez-vous que les configurations de build (gradle pour Android, Xcode pour iOS) sont correctement définies pour la release.

### Publication sur le Play Store et l'App Store :

1. **Play Store :**
  - **Téléverser l'AAB ou l'APK** via la [Google Play Console](https://play.google.com/console).
  - **Remplir la fiche produit** avec les informations requises (titre, description, captures d'écran, icône, etc.).
  - **Configurer les paramètres de distribution** (régions, prix, etc.).
  - **Soumettre pour révision** et attendre l'approbation.
2. **App Store :**
  - **Téléverser l'application** via **Xcode** ou **Transporter**.
  - **Remplir la fiche produit** sur [App Store Connect](https://connect.apple.com).
  - **Configurer les informations de l'application** (titre, description, captures d'écran, icône, etc.).
  - **Soumettre pour révision** et attendre l'approbation.

### Optimisation pour le référencement (ASO) :

- **Titre et description optimisés** : Utilisez des mots-clés pertinents dans le titre et la description de l'application.
- **Captures d'écran attractives** : Fournissez des captures d'écran de haute qualité qui mettent en valeur les fonctionnalités principales.
- **Vidéo de présentation** : Ajoutez une vidéo démontrant les fonctionnalités de l'application.
- **Avis et notes** : Encouragez les utilisateurs à laisser des avis et des notes pour améliorer la visibilité de l'application.

### Surveillance post-déploiement :

- **Utiliser des outils d'analyse** comme **Firebase Analytics** pour suivre l'engagement et les comportements des utilisateurs.
- **Surveiller les crash reports** avec **Firebase Crashlytics** pour identifier et corriger rapidement les bugs.
- **Publier des mises à jour régulières** pour introduire de nouvelles fonctionnalités et améliorer l'expérience utilisateur.

### Bonnes pratiques :

- **Préparez une stratégie de déploiement** : Planifiez les dates de lancement et les phases de release pour gérer les mises à jour de manière efficace.
- **Communiquez avec les utilisateurs** : Informez les utilisateurs des nouvelles fonctionnalités et des améliorations via les notes de version et les canaux de communication.
- **Réagissez rapidement aux feedbacks** : Surveillez les avis et les commentaires pour identifier les problèmes et les améliorer continuellement.

**Conclusion** : L'optimisation, les tests approfondis et une préparation minutieuse sont essentielles pour garantir le succès d'une application Flutter sur les stores. En suivant les bonnes pratiques décrites dans cette leçon, vous pouvez publier des applications performantes, sécurisées et de haute qualité, offrant une expérience utilisateur optimale et répondant aux attentes des utilisateurs.

## Leçon 8 : Bonnes pratiques de maintenance et mises à jour d'application

**Introduction à la maintenance continue** : La maintenance continue assure que l'application reste fonctionnelle, sécurisée et pertinente au fil du temps. Cette leçon explore les meilleures pratiques pour maintenir et mettre à jour une application Flutter après son déploiement.

### Planification des mises à jour :

- **Établir une feuille de route** : Définissez les futures fonctionnalités, les améliorations et les corrections de bugs.
- **Cycle de release régulier** : Planifiez des mises à jour régulières (mensuelles, trimestrielles) pour introduire des améliorations et corriger les problèmes.

### Gestion des versions :

- **Versioning sémantique** : Utilisez le versioning sémantique (MAJOR.MINOR.PATCH) pour gérer les versions de l'application.
- `version: 1.1.0+2`
  - **MAJOR** : Changements incompatibles.
  - **MINOR** : Ajout de fonctionnalités compatibles.
  - **PATCH** : Corrections de bugs compatibles.

### Suivi des bugs et feedbacks :

- **Outils de suivi** : Utilisez des outils comme **GitHub Issues**, **JIRA**, ou **Trello** pour suivre les bugs et les demandes de fonctionnalités.
- **Collecte de feedbacks** : Intégrez des mécanismes pour collecter les feedbacks des utilisateurs, comme des formulaires de feedback ou des systèmes de notation.

### Gestion des dépendances :

- **Mises à jour régulières** : Gardez les dépendances à jour pour bénéficier des dernières fonctionnalités et des correctifs de sécurité.
- `flutter pub upgrade`
- **Vérification de compatibilité** : Assurez-vous que les mises à jour des packages ne cassent pas les fonctionnalités existantes.

### Refactoring du code :

- **Nettoyage du code** : Supprimez le code mort, simplifiez les structures complexes et améliorez la lisibilité.
- **Modularité** : Organisez le code en modules ou en packages pour faciliter la maintenance et la réutilisation.

### Tests continus :

- **Automatisation des tests** : Intégrez des tests automatisés (unitaires, widgets, intégration) dans votre pipeline CI/CD pour détecter les régressions.
- **Revue de code** : Mettez en place des processus de revue de code pour assurer la qualité et la cohérence du codebase.

### Gestion des performances :

- **Surveillance continue** : Utilisez des outils comme **Firebase Performance Monitoring** pour surveiller les performances en temps réel.
- **Optimisations régulières** : Identifiez et corrigez les problèmes de performance au fur et à mesure qu'ils apparaissent.

### Sécurité continue :

- **Mises à jour de sécurité** : Appliquez rapidement les mises à jour de sécurité pour les dépendances et le code de l'application.
- **Audit de sécurité** : Effectuez des audits réguliers pour identifier et corriger les vulnérabilités potentielles.

### Documentation :

- **Documentation interne** : Maintenez une documentation claire et à jour pour faciliter la compréhension et la maintenance du code.
- **Documentation utilisateur** : Fournissez des guides et des FAQ pour aider les utilisateurs à naviguer et utiliser l'application efficacement.

### Exemple de mise à jour :

```
// Mise à jour de la version dans pubspec.yaml
version: 1.2.0+3

// Ajout d'une nouvelle fonctionnalité : ajout de tags aux tâches
class Tache {
 final String titre;
 final String description;
 final List<String> tags;

 Tache({required this.titre, required this.description, this.tags =
const []});

 factory Tache.fromMap(Map<String, dynamic> map) {
 return Tache(
 titre: map['titre'],
 description: map['description'],
 tags: List<String>.from(map['tags'] ?? []),
);
 }

 Map<String, dynamic> toMap() {
 return {
 'titre': titre,
 'description': description,
 'tags': tags,
 };
 }
}

// Mise à jour de l'interface utilisateur pour gérer les tags
class FormulaireTache extends StatefulWidget {
```

## Développement d'Applications Flutter

```
@override
_FormulaireTacheState createState() => _FormulaireTacheState();
}

class _FormulaireTacheState extends State<FormulaireTache> {
 final TextEditingController _titreController =
 TextEditingController();
 final TextEditingController _descriptionController =
 TextEditingController();
 final TextEditingController _tagsController =
 TextEditingController();
 List<String> _tags = [];

 @override
 Widget build(BuildContext context) {
 return Scaffold(
 appBar: AppBar(
 title: Text('Formulaire de Tâche'),
),
 body: Padding(
 padding: const EdgeInsets.all(16.0),
 child: Column(
 children: [
 TextField(
 controller: _titreController,
 decoration: InputDecoration(labelText: 'Titre'),
),
 TextField(
 controller: _descriptionController,
 decoration: InputDecoration(labelText: 'Description'),
),
 TextField(
 controller: _tagsController,
 decoration: InputDecoration(labelText: 'Tags (séparés par
des virgules)'),
 onSubmitted: (value) {
 setState(() {
 _tags = value.split(',').map((tag) =>
tag.trim()).toList();
 });
 },
),
 Wrap(
 spacing: 8.0,
 children: _tags.map((tag) => Chip(label:
Text(tag))).toList(),
),
 SizedBox(height: 20),
 ElevatedButton(
 onPressed: () {
 final tache = Tache(
 titre: _titreController.text,
 description: _descriptionController.text,
 tags: _tags,
);
 // Ajouter la tâche via le provider ou le service
 },
),
],
),
),
);
 }
}
```

```
 child: Text('Ajouter Tâche'),
),
],
),
),
);
}
}
```

### Bonnes pratiques :

- **Documentez les mises à jour** : Fournissez des notes de version claires décrivant les nouvelles fonctionnalités, les améliorations et les corrections de bugs.
- **Tester exhaustivement** : Assurez-vous que les mises à jour n'introduisent pas de régressions et que toutes les fonctionnalités fonctionnent correctement.
- **Impliquer les utilisateurs** : Utilisez les feedbacks des utilisateurs pour guider les futures mises à jour et les améliorations de l'application.
- **Maintenir la cohérence** : Gardez une cohérence dans le design, la navigation et les interactions utilisateur lors des mises à jour.

**Conclusion** : La maintenance et les mises à jour régulières sont essentielles pour assurer la longévité et le succès de votre application Flutter. En adoptant les bonnes pratiques de maintenance décrites dans cette leçon, vous pouvez garantir que votre application reste performante, sécurisée et alignée avec les besoins et les attentes des utilisateurs. Une approche proactive de la maintenance permet d'anticiper les problèmes, d'améliorer continuellement l'application et de maintenir un haut niveau de satisfaction utilisateur.

## Conclusion du Cours Flutter Complet

Félicitations ! Vous avez maintenant complété un parcours exhaustif à travers les **7 modules** de ce cours Flutter, vous dotant des compétences essentielles pour concevoir, développer, optimiser et déployer des applications mobiles robustes et performantes. Voici un récapitulatif des connaissances acquises et des compétences développées tout au long de ce cours.

### Récapitulatif des Modules

1. **Introduction à Flutter et Configuration de l'Environnement**
  - **Fondamentaux de Flutter** : Compréhension de la structure des applications Flutter, du langage Dart, et des principes de la programmation réactive.
  - **Configuration de l'environnement** : Installation des outils nécessaires, création de projets Flutter, et utilisation des émulateurs et appareils physiques pour le développement.
2. **Widgets, Mise en Page et Navigation**
  - **Widgets de base et avancés** : Maîtrise des widgets essentiels pour construire des interfaces utilisateur intuitives et réactives.

- **Mise en page** : Utilisation efficace des widgets de mise en page comme Row, Column, Stack, et gestion des layouts complexes.
- **Navigation** : Implémentation de la navigation entre différentes pages avec des routes nommées et des transitions personnalisées.
- 3. **Gestion de l'État et Architecture**
  - **Gestion de l'état** : Introduction à des solutions comme Provider et Riverpod pour gérer l'état de l'application de manière efficace et scalable.
  - **Architecture propre** : Adoption de bonnes pratiques architecturales pour séparer la logique de présentation, la logique métier et l'accès aux données.
- 4. **Accès aux Services Externes et Intégrations**
  - **Intégration des APIs** : Connexion avec des services RESTful, gestion des requêtes réseau, et manipulation des données JSON.
  - **Bases de données** : Utilisation de solutions comme Firebase Firestore et Realtime Database pour le stockage et la synchronisation des données en temps réel.
  - **Authentification et Sécurité** : Mise en place de systèmes d'authentification sécurisés avec Firebase Auth, gestion des permissions et protection des données sensibles.
- 5. **Fonctionnalités Avancées et Plugins**
  - **Intégration de fonctionnalités complexes** : Ajout de cartes interactives avec Google Maps, gestion des capteurs de l'appareil (GPS, gyroscope), et implémentation de notifications push avec Firebase Cloud Messaging.
  - **Développement de plugins personnalisés** : Création, test et publication de plugins Flutter pour étendre les fonctionnalités de vos applications et contribuer à la communauté Flutter.
- 6. **Optimisation, Tests et Déploiement**
  - **Optimisation des performances** : Techniques pour améliorer la fluidité et la réactivité de l'application, réduction de la taille des builds et gestion efficace des ressources.
  - **Tests** : Mise en place de tests unitaires, tests widgets et tests d'intégration pour garantir la fiabilité et la robustesse de l'application.
  - **Déploiement** : Préparation et publication des applications sur le Google Play Store et l'App Store, gestion des certificats de signature, et optimisation pour le référencement des applications (ASO).
- 7. **Maintenance, Sécurité et Bonnes Pratiques**
  - **Maintenance continue** : Stratégies pour maintenir l'application à jour, gérer les versions, et intégrer les feedbacks utilisateurs.
  - **Sécurité** : Implémentation de meilleures pratiques pour sécuriser l'application, protéger les données des utilisateurs, et respecter les politiques des stores.
  - **Documentation et Collaboration** : Importance de la documentation claire et de la collaboration efficace au sein d'une équipe de développement pour assurer la qualité et la maintenabilité du code.

## Compétences Acquis

- **Maîtrise de Flutter et Dart** : Capacité à créer des interfaces utilisateur sophistiquées et réactives en utilisant les widgets Flutter.
- **Gestion efficace de l'état** : Compétence à utiliser des solutions de gestion d'état modernes pour construire des applications scalables et maintenables.
- **Intégration de services externes** : Aptitude à connecter des applications Flutter avec des APIs, bases de données cloud, et autres services tiers.
- **Développement de plugins** : Capacité à étendre les fonctionnalités de Flutter en développant et en publiant des plugins personnalisés.
- **Optimisation des performances** : Compétence à identifier et à résoudre les problèmes de performance pour offrir une expérience utilisateur fluide.
- **Tests rigoureux** : Habilité à écrire et à exécuter des tests automatisés pour garantir la qualité et la fiabilité de l'application.
- **Déploiement professionnel** : Connaissance approfondie des processus de publication sur les principaux stores mobiles, assurant une distribution efficace et conforme aux standards.

### Perspectives et Prochaines Étapes

Maintenant que vous avez acquis une solide base en développement Flutter, voici quelques recommandations pour continuer à progresser :

1. **Pratique Continue** : Créez des projets personnels ou contribuez à des projets open-source pour appliquer vos connaissances et découvrir de nouveaux défis.
2. **Exploration Avancée** : Approfondissez des sujets spécifiques comme le développement d'applications multiplateformes (web, desktop), l'animation avancée, ou l'intégration de technologies émergentes comme l'IA et le machine learning.
3. **Certification** : Envisagez de passer des certifications Flutter pour valider vos compétences et renforcer votre profil professionnel.
4. **Participation à la Communauté** : Engagez-vous avec la communauté Flutter via des forums, des groupes de discussion, des conférences et des hackathons pour échanger des idées et rester à jour avec les dernières tendances.
5. **Apprentissage Continu** : Restez informé des mises à jour de Flutter et Dart, et continuez à apprendre de nouvelles bibliothèques et outils pour améliorer vos applications.

### Conclusion Finale

Le développement Flutter offre une plateforme puissante et flexible pour créer des applications mobiles modernes et performantes. En complétant ce cours, vous avez acquis les compétences nécessaires pour transformer vos idées en applications fonctionnelles, engageantes et optimisées. Que vous souhaitiez développer des applications pour des clients, lancer votre propre startup, ou simplement élargir votre expertise en développement mobile, les connaissances acquises ici vous fourniront une base solide pour réussir dans le monde dynamique du développement Flutter.

N'oubliez pas que le domaine de la technologie évolue rapidement. Continuez à explorer, à apprendre et à vous adapter pour rester à la pointe et tirer le meilleur parti de vos compétences en Flutter. Bonne continuation dans votre parcours de développeur Flutter !